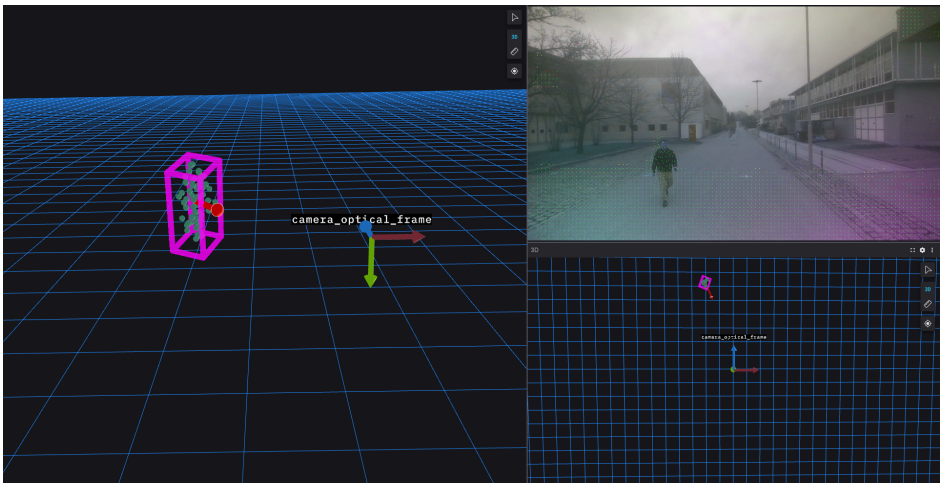
Stereo Camera-Based Environment Perception for Automated Vehicles

Stereokamera-Basierte Umfeldwahrnemung für automatisierte Fahrzeuge



Scientific work for obtaining the academic degree
Master of Science (M.Sc.)
at the Department Mobility Systems Engineering
of the TUM School of Engineering and Design
at the Technical University of Munich

Supervised by	Prof. Dr.-Ing. Markus Lienkamp David Brecht, M.Sc. Chair of Automotive Technology
Submitted by	Adrian Rieker, B.Sc. Schleißheimer Str. 126 80797 München
Submitted on	July 01, 2025

Chair of Automotive Technology
TUM School of Engineering and Design
Technical University of Munich

Project description

Titel der Arbeit

Hier steht ein zum Thema hinführender Text.

In einer konstruktiven (oder theoretischen) Bachelor/-Semester/-Masterarbeit sollen (...)

Folgende Punkte sind durch Herrn/Frau Martin Musterstudent zu bearbeiten:

- Punkt 1
- Punkt 2

Die Ausarbeitung soll die einzelnen Arbeitsschritte in übersichtlicher Form dokumentieren. Der Kandidat/Die Kandidatin verpflichtet sich, die Bachelor/-Semester/-Masterarbeit selbständig durchzuführen und die von ihm verwendeten wissenschaftlichen Hilfsmittel anzugeben.

Die eingereichte Arbeit verbleibt als Prüfungsunterlage im Eigentum des Lehrstuhls.

Announcement date: April 1, 2021

Submission date: October 1, 2021

Prof. Dr.-Ing. Markus Lienkamp

Max Musterbetreuer, M.Sc.

Chair of Automotive Technology
TUM School of Engineering and Design
Technical University of Munich

Geheimhaltungsverpflichtung

Herr: **Rieker, Adrian**

Gegenstand der Geheimhaltungsverpflichtung sind alle mündlichen, schriftlichen und digitalen Informationen und Materialien die der Unterzeichner vom Lehrstuhl oder von Dritten im Rahmen seiner Tätigkeit am Lehrstuhl erhält. Dazu zählen vor allem Daten, Simulationswerkzeuge und Programmcode sowie Informationen zu Projekten, Prototypen und Produkten.

Der Unterzeichner verpflichtet sich, alle derartigen Informationen und Unterlagen, die ihm während seiner Tätigkeit am Lehrstuhl für Fahrzeugtechnik zugänglich werden, strikt vertraulich zu behandeln.

Er verpflichtet sich insbesondere:

- derartige Informationen betriebsintern zum Zwecke der Diskussion nur dann zu verwenden, wenn ein ihm erteilter Auftrag dies erfordert,
- keine derartigen Informationen ohne die vorherige schriftliche Zustimmung des Betreuers an Dritte weiterzuleiten,
- ohne Zustimmung eines Mitarbeiters keine Fotografien, Zeichnungen oder sonstige Darstellungen von Prototypen oder technischen Unterlagen hierzu anzufertigen,
- auf Anforderung des Lehrstuhls für Fahrzeugtechnik oder unaufgefordert spätestens bei seinem Ausscheiden aus dem Lehrstuhl für Fahrzeugtechnik alle Dokumente und Datenträger, die derartige Informationen enthalten, an den Lehrstuhl für Fahrzeugtechnik zurückzugeben.

Eine besondere Sorgfalt gilt im Umgang mit digitalen Daten:

- Für den Dateiaustausch dürfen keine Dienste verwendet werden, bei denen die Daten über einen Server im Ausland geleitet oder gespeichert werden (Es dürfen nur Dienste des LRZ genutzt werden (Lehrstuhllaufwerke, Sync&Share, GigaMove).
- Vertrauliche Informationen dürfen nur in verschlüsselter Form per E-Mail versendet werden.
- Nachrichten des geschäftlichen E-Mail Kontos, die vertrauliche Informationen enthalten, dürfen nicht an einen externen E-Mail Anbieter weitergeleitet werden.
- Die Kommunikation sollte nach Möglichkeit über die (my)TUM-Mailadresse erfolgen.

Die Verpflichtung zur Geheimhaltung endet nicht mit dem Ausscheiden aus dem Lehrstuhl für Fahrzeugtechnik, sondern bleibt 5 Jahre nach dem Zeitpunkt des Ausscheidens in vollem Umfang bestehen. Die eingereichte schriftliche Ausarbeitung darf der Unterzeichner nach Bekanntgabe der Note frei veröffentlichen.

Der Unterzeichner willigt ein, dass die Inhalte seiner Studienarbeit in darauf aufbauenden Studienarbeiten und Dissertationen mit der nötigen Kennzeichnung verwendet werden dürfen.

Datum: 1. Juli 2025

Unterschrift: _____

Chair of Automotive Technology
TUM School of Engineering and Design
Technical University of Munich

Erklärung

Ich versichere hiermit, dass ich die von mir eingereichte Abschlussarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Garching, den 1. Juli 2025

Adrian Rieker, B.Sc.

Chair of Automotive Technology
TUM School of Engineering and Design
Technical University of Munich

Declaration of Consent, Open Source

Hereby I, Rieker, Adrian, born on August 14, 2000, make the software I developed during my Masterthesis Thesis available to the Institute of Automotive Technology under the terms of the license below.

Garching, July 1, 2025

Adrian Rieker, B.Sc.

Contents

Formula Symbols

Formula Symbols	Unit	Description
f_x	px	Focal length of the camera in the x direction
f_y	px	Focal length of the camera in the y direction
c_x	px	x coordinate of the principal point
c_y	px	y coordinate of the principal point
$\mathbf{x}_k$		State vector at time k (3D position and velocity)
$\hat{\mathbf{x}}_k$		Predicted state vector at time k
$\mathbf{P}_k$		State covariance matrix
$\hat{\mathbf{P}}_k$		Predicted state covariance matrix
$\mathbf{A}$		State transition matrix
$\mathbf{B}$		Control input matrix
$\mathbf{u}_k$		Control input vector (e.g. ego-motion)
$\boldsymbol{\omega}_k$		Process noise vector
$\mathbf{Q}$		Covariance matrix of process noise
$\mathbf{z}_k$		Measurement vector (e.g. pixel + depth)
$\hat{\mathbf{z}}_k$		Predicted measurement vector
$\mathbf{H}$		Measurement matrix (linear case)
$\mathbf{H}_k$		Jacobian of the nonlinear measurement function
$\mathbf{h}(\cdot)$		Nonlinear measurement function
$\mathbf{R}$		Measurement noise covariance matrix
$\hat{\mathbf{T}}_k$		Predicted measurement covariance
$\mathbf{s}_k$		Innovation vector (residual)
$\mathbf{S}_k$		Innovation covariance matrix
$\mathbf{K}_k$		Kalman gain
$\mathbf{I}$		Identity matrix
u_k	px	Horizontal pixel coordinate

v_k	px	Vertical pixel coordinate
d_k	m	Depth value
Δt	s	Time difference between frames

Todo list

1 Introduction

Autonomous vehicles are quickly heading towards a widespread deployment, with the potential to significantly enhance traffic safety, alleviate congestion, and increase overall transportation efficiency.

At the heart of this transformation lies the ability to perceive the surrounding environment with high accuracy and robustness. Perception systems enable vehicles to detect obstacles, interpret the behavior of other traffic participants, and understand the overall context of the driving scene. However, the real world is dynamic and often unstructured, presenting substantial challenges to perception algorithms.

Failures in perception can lead to *disengagements*, situations in which the automated driving system can no longer identify a safe course of action and must transfer control to a human operator, often via teleoperation. These events typically occur outside the nominal operating conditions of the syste, and are especially critical. Even in such scenarios, the vehicle must maintain a degree of situational awareness to support fallback strategies or assist the operator in resolving the situation safely.

Current state-of-the-art perception systems are predominantly learning-based, with deep neural networks trained on large annotated datasets. These methods perform well in structured, well-represented environments but struggle to generalize to rare or previously unseen scenarios. Their lack of transparency also raises concerns about interpretability and systematic validation—key requirements in safety-critical applications.

To address these limitations, this thesis explores a complementary, deterministic approach using stereo camera data. The goal is to design and implement a perception system capable of estimating the position and motion of surrounding objects without reliance on prior training or semantic annotations. By focusing on geometric cues such as disparity and optical flow, the system aims to achieve robust performance across diverse and challenging environments.

The system processes synchronized stereo images from a vehicle mounted camera setup to produce 6D motion estimates: three dimensional positions and the corresponging velocities of selected scene points. A Kalman filter-based framework, inspired by the Daimler 6D Vision approach, integrates depth, optical flow, and ego-motion information to compute these estimates. The resulting motion field is clustered using the DBSCAN algorithm to identify coherent moving objects.

To assess potential hazards, a criticality evaluation module calculates safety metrics based on predicted trajectories of both the ego-vehicle and the detected objects. One key metric, Time-To-Collision (TTC), provides insight into the urgency of evasive actions. These evaluations are performed in real time to support both autonomous fallback strategies and teleoperated decision-making.

This work builds on previous research at the Chair of Automotive Technology, which investigated stereo vision and deterministic motion estimation. The present study focuses on integrating these methods into a modular, reusable software component. A major contribution is the development of an independent ROS2 servie, based on the Humble distribution, designed for seamless integration with the **EDGAR research vehicle** and adaptability to other platforms.

The system is implemented in **C++**, with performance-critical components accelerated using **CUDA** and parallelized via **OpenMP**. This ensures that real-time constraints are met while maintaining scalability and

robustness. Each major function—depth estimation, optical flow, Kalman filtering, clustering, and criticality analysis—is encapsulated in its own ROS 2 node, supporting modular development, testing, and integration. The use of **Docker** for containerization ensures portability and simplifies deployment across heterogeneous systems.

The system architecture consists of two main processing pipelines:

- **Perception pipeline:** Combines stereo vision, visual odometry, and deterministic motion estimation to infer 6D trajectories of scene points. Moving objects are identified by clustering these trajectories based on spatial and motion consistency.
- **Criticality assessment pipeline:** Analyzes interactions between the ego-vehicle and detected objects to compute risk-related metrics that support fallback decisions and driver assistance.

Unlike purely learning-based systems, this approach emphasizes interpretability, generalization to novel scenarios, and independence from large-scale labeled data. It also complements neural-network-based methods by enhancing robustness in adverse or edge-case conditions.

System evaluation is carried out in two stages. First, individual components are tested on public benchmarks such as KITTI, which provide ground truth for depth, optical flow, and odometry. Second, the complete system is deployed on the EDGAR vehicle and assessed in real-world driving situations. Its performance is compared to that of the existing learning-based perception stack to evaluate reliability and robustness.

In summary, this thesis presents a deterministic, modular, and real-time capable perception system for automated driving. By leveraging stereo vision and geometric algorithms, the system enhances environmental understanding in situations where deep learning methods may fall short. Its deployment as a ROS2 service, optimized for performance and portability, enables easy integration into diverse research and development workflows.

2 State of the Art

Autonomous and automated vehicles must have a comprehesive and accurate understanding of their surroundings at all times to ensure safe and reliable operation. This process, known as environment perception, is responsible for detecting, identifying, and tracking objects such as other vehicles, cyclists, pedestrians or other road elements. The quality of the measurement of the state of this objects directly influence the vehicle's ability to make safe and informed driving decisions.

Failures in the perception can lead to hazarous situations, such as collisions or unexpected navigation decisions. This is particularly critical in high-speed scenarios or complex urban environments, where is a high density of high dynamic objects which make more complex the operation in real time.

A robust perception system must provide the following capabilities:

1. **Object detection:** Identify and locate objects in the vehicle's surroundings, such as vehicles, pedestrians, cyclists, or other road elements.
2. **Motion estimation:** Determining the position and velocity of moving objects
3. **Scene understanding:** Classifiying the detected objects and assessing their criticality for the vehicle's navigation
4. **Reliability in edge cases:** Ensuring correct operation in complex and unforeseen scenarios, such as occlusions, adverse weather conditions, or sensor failures.
5. **Real-time operation:** Providing timely and accurate information to the vehicle's control system to make informed driving decisions.

While the perception concept includes different aspects, this work will focus specifically on the detection of moving object in the vehicle's surroundings and the estimation of their 6D motion state (position and velocity) using a deterministic approach. In the following sections, we will provide an overview of the state-of-the-art of the different sensors and algorithms used in the perception systems of autonomous vehicles, focusing on the deterministic image based approaches.

2.1 Perception Systems for Autonomous Vehicles

Perception systems are a core component of autonomous and automated vehicles. They enable the continuous interpretation of the environment by converting raw sensor data into structured information, which is subsequently used by planning and control modules. The accuracy, reliability, and completeness of this information directly influence the vehicle's ability to operate safely and efficiently in various driving conditions [1].

To obtain a comprehensive representation of the surrounding environment, modern autonomous systems integrate data from multiple sensor modalities. These include LiDAR, radar, and camera systems, each offering distinct capabilities and constraints. The fusion of these sensors allows the vehicle to mitigate individual

sensor limitations and achieve a more robust perception. However, understanding the characteristics of each sensor independently remains essential, especially in system design or in the development of fallback strategies based on deterministic algorithms.

The following subsections provide an overview of the most common sensor types used in autonomous vehicles, including their working principles, strengths, and limitations. This review lays the foundation for the sensor selection and processing strategies discussed later in this work.

LiDAR-Based Perception

LiDAR (Light Detection and Ranging) is widely employed in automotive perception for its ability to generate dense and geometrically accurate 3D representations of the environment [1]. By measuring the round-trip time of emitted laser pulses, LiDAR produces a point cloud that encodes spatial structure with high resolution.

This technology is particularly effective in tasks such as object detection, 3D mapping, lane boundary identification, and free-space estimation. Its ability to provide absolute distance measurements makes it well suited for geometric computations without dependence on visual features. Automotive companies including BMW [2], Mercedes-Benz [3], and Waymo [4] incorporate LiDAR in their perception stacks, particularly for high-level automation.

Working principle A LiDAR unit emits a sequence of laser pulses into the environment and detects the reflected signals using photodetectors. By calculating the time interval between emission and reception, and knowing the speed of light, the distance to each object surface is computed [5]. Modern automotive LiDARs rotate or scan across multiple directions to generate a full 3D point cloud of the scene.

Advantages and Limitations The main advantage of LiDAR is its precise depth sensing and high spatial resolution. This allows robust obstacle detection independent of ambient light conditions. However, LiDAR is sensitive to weather disturbances such as rain, fog, or snow [6], where light scattering can degrade data quality. In addition, LiDAR devices typically incur higher costs than camera or radar systems. Although prices are decreasing, LiDAR adoption in production vehicles remains limited to higher-end or research platforms.

Radar-Based Perception

Radar (Radio Detection and Ranging) sensors use electromagnetic waves in the radio frequency spectrum to detect objects and estimate their distance and relative velocity. These sensors have long been used in automotive systems, particularly for functions such as adaptive cruise control and collision avoidance.

Working principle Radar sensors emit radio signals that reflect off surfaces in the environment. The time delay between emission and reception, combined with the known wave propagation speed, enables the computation of object distances. The Doppler shift in the frequency of the reflected waves further allows the estimation of relative velocities [7]. Automotive radars typically operate in the 77 GHz band, balancing resolution and penetration capabilities .

reference here

Advantages and Limitations Radar systems are robust under adverse environmental conditions. Their signals are less affected by rain, fog, or dust, making them a reliable source of detection even in degraded visual environments [1, 8]. Moreover, their ability to directly measure relative velocity provides useful dynamic information.

However, the spatial resolution of radar is lower than that of LiDAR or cameras, which can complicate object classification or detailed localization. Additionally, radar signals may produce multipath effects or suffer from interference in complex environments.

Image-Based Perception

Camera-based systems provide rich visual information that can be used for a variety of perception tasks, including classification, semantic segmentation, visual odometry, and depth estimation. They are an essential part of most autonomous driving stacks and, in some designs, serve as the primary perception modality—as exemplified by Tesla's Autopilot system [9].

Monocular vision Monocular setups involve a single camera capturing 2D projections of the environment. These systems are effective for detecting traffic signs, identifying objects, and inferring semantic information. They benefit from low cost and ease of integration but have limited capacity to infer depth without additional cues or motion-based estimation methods [1].

Stereo vision Stereo camera systems use two synchronized cameras mounted at a known baseline. By applying geometric constraints from epipolar geometry and the pinhole camera model [10], the system computes disparity maps that can be converted into depth information. This enables passive 3D reconstruction without active illumination, making stereo vision particularly attractive for dense, low-cost depth estimation in structured environments.

Advantages and Limitations Image-based perception provides access to texture, color, and appearance features, enabling detailed scene understanding. Cameras are compact, power-efficient, and inexpensive relative to other sensors. Nonetheless, their performance is susceptible to lighting variations, shadows, glare, and occlusions. Depth estimation from stereo vision requires careful calibration and performs poorly in textureless regions or when the baseline is insufficient. Computational demands can also be significant, particularly when processing high-resolution frames or running complex algorithms in real time.

The following table summarizes the strengths and limitations of the three major sensor types discussed:

Table 2.1: Summary of sensor types used in autonomous vehicles

Sensor type	Advantages	Limitations
LiDAR	Accurate 3D representation	Expensive and sensitive to adverse weather conditions
Radar	Operates in degraded weather, velocity estimation	Low spatial resolution
Cameras	Rich scene information, low cost and compact	Sensitive to lighting, limited passive depth estimation

While each sensor type offers distinct advantages, their integration and the interpretation of their data through perception algorithms ultimately determine the system's effectiveness. The next sections will provide a more detailed overview of algorithmic strategies used for detecting and tracking moving objects, with a focus on deterministic approaches relevant to the scope of this thesis.

2.2 Algorithms for Moving Object Detection

Beyond sensor selection, the effectiveness of a perception system depends heavily on the algorithms used to process sensor data and detect dynamic elements in the environment. These algorithms are responsible for interpreting raw measurements to estimate the presence, position, and motion of objects relevant to driving decisions.

Moving object detection is a fundamental task in autonomous vehicles, as it enables the system to anticipate changes in the environment and plan appropriate maneuvers. The algorithms designed for this task can generally be classified into two broad categories: learning-based approaches and deterministic approaches. The former relies on data-driven models, while the latter is based on physical and geometric principles. Each paradigm has its strengths and limitations, and they often complement each other in hybrid systems .

some reference of this

The following sections provide an overview of both categories, highlighting representative methods, their operational characteristics, and their relevance to the goals of this thesis.

Learning-Based Approaches

Learning-based approaches have become predominant in many domains of computer vision, including object detection, tracking, and scene understanding. These methods use models such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformer-based architectures to learn mappings from raw sensor inputs to high-level perception outputs.

In the context of moving object detection, learning-based methods typically combine object detection with instance tracking over time. Networks are trained end-to-end to predict object categories, bounding boxes, and motion parameters. For example, approaches such as CenterTrack [11] and TrackRCNN [12] integrate detection and tracking into a unified framework. Some methods extend this further to 3D detection using point cloud data or camera-LiDAR fusion, as explored in works like PointRCNN [13] and DeepFusion [14].

A comprehensive review of perception techniques based on deep learning is provided by Wen and Jo [15], where architectures are categorized by input modality, task type, and application scope. A simplified summary of these methods is shown in Figure 2.1.

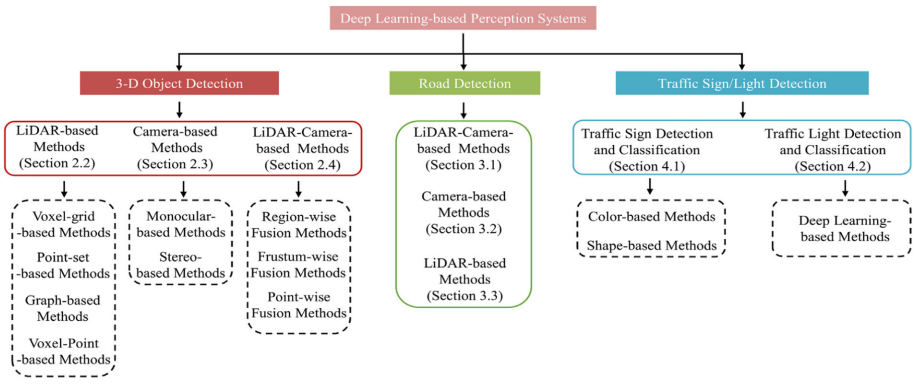


Figure 2.1: Learning-based perception methods. Source: [15]

Despite their high accuracy and generalization across many scenarios, learning-based methods present important challenges. They require large volumes of annotated data for training and validation, which is costly and time-consuming to obtain, especially for edge-case scenarios such as rare weather events, unusual object classes, or emergency maneuvers. Additionally, the black-box nature of neural networks complicates

formal verification and interpretability, which are essential for safety critical applications such as automated driving.

In unstructured or previously unseen environments, these models may fail to generalize correctly. This limitation motivates the use of complementary, model-based approaches, especially as a fallback or redundancy mechanism in high-assurance systems.

Deterministic Approaches

Deterministic approaches rely on physically grounded models and algorithms to estimate object motion from sensor data. They do not require training and are typically more interpretable, making them suitable for applications where transparency and real-time guarantees are essential.

Optical Flow Optical flow refers to the apparent motion of image brightness patterns between consecutive frames. By assuming brightness constancy and small motion between frames, optical flow algorithms estimate the displacement vector for each pixel or image region. This flow field encodes both motion direction and magnitude, which can be used to infer object motion relative to the camera.

Classic methods for optical flow computation include:

- **Lucas–Kanade** [16], which estimates flow using local patches and least-squares optimization,
- **Horn–Schunck** [17], which introduces a global smoothness constraint on the flow field.

Optical flow is often used in combination with depth or disparity information to project 2D motion into 3D space. When paired with stereo cameras, it can provide estimates of object motion in the vehicle's coordinate frame.

6D Vision The 6D Vision framework, introduced by Franke et al. [18], is a deterministic method for jointly estimating the 3D position and velocity of points in a scene using stereo vision and optical flow. Unlike learning-based methods, this approach does not rely on semantic segmentation or object-specific models. Instead, it directly tracks sparse 3D points across time and estimates their motion using filtering techniques.

The key components of 6D Vision are:

1. **Stereo disparity estimation:** Corresponding points are matched between left and right stereo images to estimate depth through triangulation.
2. **Optical flow tracking:** Temporal correspondences are computed using optical flow between successive frames.
3. **Kalman filtering:** Each tracked point is associated with a state vector comprising position and velocity. Kalman filters are used to predict and update this state over time, taking into account sensor noise and model uncertainty.

The result is a sparse but dynamic representation of the scene, where each tracked point has an associated 6D state. These points can then be grouped into moving objects through clustering techniques such as DBSCAN, enabling object-level motion understanding without the need for prior object models.

Because it is model-free and data-independent, 6D Vision is especially useful in scenarios where learning-based systems may fail due to insufficient training data or degraded input quality. However, the method requires accurate calibration and careful handling of uncertainty to remain robust under varying conditions.

In this thesis, we follow the principles of 6D Vision and extend them into a modular and real-time ROS2-based implementation. The proposed system uses stereo camera input to generate a dynamic 3D representation of the environment, which serves as a basis for object detection and criticality analysis. The deterministic nature of the approach ensures transparency and complements learning-based perception stacks, contributing to improved redundancy and safety in autonomous driving.

2.3 State of the Art in the FTM

The Institute of Automotive Technology (FTM) at the Technical University of Munich has conducted previous research in the field of environment perception for autonomous and automated vehicles. This section provides an overview of previous work carried out at the FTM, particularly regarding the EDGAR research vehicle and the development of deterministic perception systems based on 6D vision.

2.3.1 EDGAR Research Vehicle

The EDGAR (Excellent Driving Garching) research vehicle is a modular testbed developed at the Institute of Automotive Technology (FTM) at TUM [19]. It is designed to support the development, integration, and evaluation of autonomous driving software in real-world conditions.

The platform is based on a Volkswagen T7 Multivan 1.4 eHybrid. This vehicle offers sufficient electrical power and internal space for mounting computing units and sensor interfaces. It retains certified drive-by-wire functionality, which allows autonomous control over steering, throttle, and braking while preserving manual override.

EDGAR features a multi-modal sensor suite, including:

- **Cameras (mono, stereo, depth)** for perception and teleoperation
- **LiDARs (short and long range)** for 3D environment modeling
- **Radars** for velocity estimation in all weather conditions
- **GNSS/IMU** for localization and motion estimation
- **Microphones** for ambient sound detection

The sensors are mounted with known extrinsics and stored in a digital twin model, enabling reproducibility in simulation and real-world testing. An example of the sensor layout is shown in Figure 2.2.

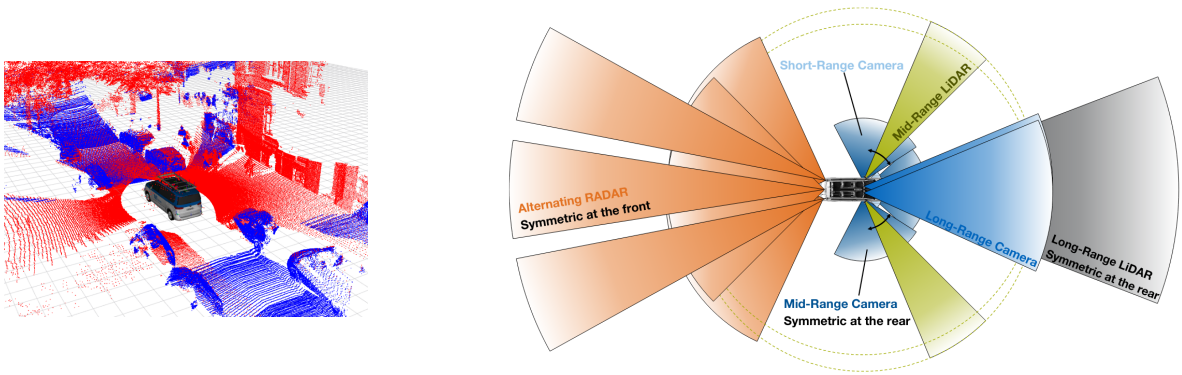


Figure 2.2: EDGAR Vehicle Sensors. Source: [19]

To support sensor fusion and deterministic perception, precise time synchronization is required. For this purpose, the vehicle implements a full Precision Time Protocol (PTP) network. A master clock (GMR5000) provides global time, and all sensors and computing units synchronize as PTP clients. The network infrastructure is based on a Netgear M4250 switch with Time Sensitive Networking (TSN) support and PoE capabilities.

All connected sensors must fulfill the following integration requirements:

- Support for hardware-level PTP time synchronization
- Preferably PoE interface to reduce cabling complexity
- Availability of a stable ROS 2-compatible driver (Humble version)
- Compatibility with high-throughput network architecture (40 Gbit uplink)

The EDGAR system contains two high-performance computers: one x86-based and one ARM-based. Both platforms support GPU acceleration, have CAN interfaces to access the actuator bus, and are synchronized via the internal PTP system. The overall network architecture is shown in Figure 2.3.

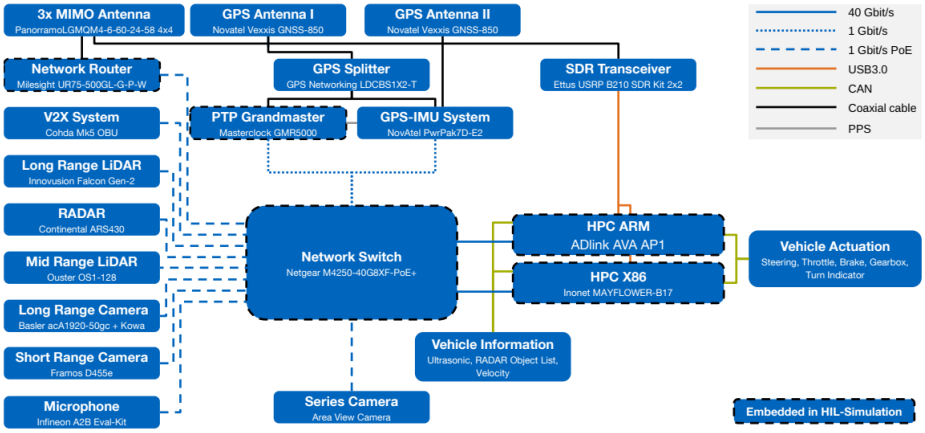


Figure 2.3: EDGAR network architecture. Source: [19]

This architecture imposes specific constraints on any new sensing or perception component. In particular, any new camera to be integrated into the EDGAR platform must be fully PTP-compatible, support a stable ROS 2 driver, and preferably offer PoE functionality to ensure seamless deployment within the existing infrastructure.

The modularity of EDGAR allows rapid software integration, reproducible validation, and testing of deterministic perception systems such as the one developed in this thesis.

2.3.2 Previous Work in the FTM

This project builds upon previous work conducted at the FTM in the field of environment perception for automated vehicles with 6D vision. Some of the key contributions include:

1. **Objektdetektion und Bewegungsprädiktion mittels eines Stereokamerasystems beim teleoperierten Fahren** [20]: Bauer in his master's thesis developed a deterministic system for object detection based on the 6D vision of Daimler [18]. The system was implemented in C++ using the ADTF framework Automotive Data and Time-Triggered Framework and was

capable of detecting and tracking objects in the vehicle's surroundings with some real time limitations.

2. **Weiterentwicklung und echtzeitfähige Umsetzung eines Stereo-Vision-basierten Bewegungsprädiktionssystems für die aktive Sicherheit von teleoperierten Fahrzeugen** Some time later in [21] Rotar continued Bauers work by implementing the system in GPU using CUDA to speed up the computationally intensive tasks.

3. **Validation of an Obstacle Detection Approach for Teleoperation of Automated Vehicles:** Finally Pastor [22] started porting the system to ROS1 to facilitate the integration with other modules in the EDGAR vehicle. The system was implemented without CUDA GPU acceleration and was not deployed in the EDGAR vehicle. The system also has some problem with the object detection.

Building upon these previous contributions, this project aims to further improve deterministic perception by addressing key limitations found in prior implementations. In particular, this work will focus on enhancing computational efficiency, real-time feasibility, and system deployment within ROS2. These improvements will be detailed in the next section on objectives.

2.4 Stereo Camera Technology

This section reviews stereo vision technologies relevant to autonomous driving applications. Stereo cameras provide dense depth estimation, enabling motion perception, obstacle detection, and environment reconstruction without active range sensing. While early systems such as Daimler's 6D Vision relied on passive stereo only [18], modern cameras integrate active lighting, neural inference, and time synchronization for robust deployment.

The classification used here distinguishes passive stereo, active infrared stereo, laser-structured light, and smart stereo systems with onboard processing. The overview is based on the ROS-Industrial survey [23], a general technology review [24], and recent empirical studies in robotics and autonomous navigation.

2.4.1 Passive Stereo Vision

Passive stereo systems compute depth from disparities between synchronized images under ambient illumination. They are well-suited for outdoor driving scenarios, where natural texture is often abundant.

- **MultiSense S30**: A rugged stereo camera with a 30 cm baseline, IP68 enclosure, and FPGA-based depth processing. It has been used in visual odometry pipelines for outdoor navigation, demonstrating high robustness to scene complexity and lighting variations [25].
- **Luxonis OAK-D**: A lightweight smart camera integrating stereo depth with neural inference on-chip. In [26], it was used for real-time gate detection and navigation in autonomous drones, showing potential for embedded vehicle perception tasks without relying on external GPUs.

2.4.2 Active Infrared Stereo

Active infrared stereo cameras enhance depth estimation in low-texture scenes by projecting structured light patterns. They are effective indoors or under controlled illumination, but their performance may degrade under direct sunlight.

- **Intel RealSense D455**: Incorporates a global shutter stereo pair and infrared projector. It supports depth estimation up to 10 m. Its suitability for SLAM was evaluated in [27], which found that depth accuracy decreases significantly with distance and oblique incidence, though it remains competitive in short-range indoor mapping.
- **Luxonis OAK-D Pro**: Combines active stereo with onboard AI. In comparative benchmarks [28], it performed reliably in low-light and textureless environments, offering integrated person detection and object tracking capabilities.

2.4.3 Laser-Structured Stereo

Laser-structured stereo systems project high-contrast patterns to improve depth estimation. These systems are often used in industrial setups where illumination can be controlled, and where high accuracy is required.

- **Zivid 2+**: A high-resolution RGB-D camera employing structured laser light. Its depth noise and uncertainty were characterized in [29], where a model-based compensation scheme improved the performance of 3D reconstruction pipelines. Although designed for industrial robotics, its accuracy and noise profile make it a candidate for infrastructure inspection in autonomous driving.

2.4.4 Smart Stereo Cameras with Onboard Processing

Smart stereo cameras integrate processing units for depth and AI tasks, reducing bandwidth and computation load on host systems. These sensors are increasingly used in embedded perception stacks for autonomous platforms.

- **Stereolabs ZED 2**: Provides real-time stereo depth and inertial tracking. In [30], it was used to detect and track trees from a mobile platform in unstructured outdoor environments. Dense stereo point clouds were processed into pseudo-LiDAR for object-level reasoning, showing applicability in naturalistic driving settings such as forests or parks.
- **Luxonis OAK-D**: Reappears here due to its integrated VPU, which supports neural model inference and stereo depth estimation in real time. Its use in closed-loop autonomous navigation [26] exemplifies its relevance for perception systems with limited computational budgets.

Stereo vision is employed as the primary sensing modality for motion estimation in this work. The Framos D455e, an active stereo camera with infrared projection, is currently deployed on the EDGAR platform. Its real-time capabilities align with the requirements of this project. Alternative sensor options and their impact on performance are discussed in Section **??**.

2.5 Requirements for the Thesis

Robust detection of dynamic objects is a fundamental requirement in automated driving, as it directly affects the capability of the vehicle to avoid hazards and make safe navigation decisions. While deep learning-based systems currently dominate commercial perception stacks, their depencency on extensive labeled datasets, limited generalization in rare scenarios, and lack of transparency introduce challenges in safety-critical contexts. This has led to increased interest in deterministic alternatives, which leverage geometric and physical priors to infer motion from raw sensor data without the need for training.

Among these, LiDAR-based methods have demonstrated strong performance due to their accurate depth estimation. Voxel-based segmentation approaches such as [31] and mapless, distortion-compensated techniques such as [32] show that dynamic object detection can be performed in real time using 3D geometry alone. However, LiDAR sensors remain expensive, weather-sensitive, and difficult to integrate into embedded platforms. These limitations motivate the use of stereo vision, which provides dense spatial information at significantly lower cost and system complexity.

Camera-based deterministic methods have emerged as a promising alternative. In [33], a stereo system detects and tracks moving obstacles in real time using sparse point clouds and inter-frame clustering. The method achieves high tracking accuracy on embedded hardware. Other works such as [34] extend this concept to dynamic pedestrian environments, while [35] proposes efficient filtering of dynamic points to support consistent 3D mapping. Although these methods produce promising results, many rely on heuristic motion classification or operate in simplified 2D spaces.

The 6D Vision framework, described in Section 2.2, provides a more principled approach. It combines stereo disparity and optical flow to estimate the full 3D position and velocity of scene points. This method tracks sparse features over time and uses Kalman filters to estimate their trajectories, enabling object-agnostic motion reasoning through recursive estimation.

This thesis adopts and modernizes the 6D Vision paradigm, introducing updates in both algorithmic design and system-level implementation. The developed system is implemented entirely in C++ and ROS2 and designed for deployment on the EDGAR research vehicle. A key novelty is the full modularization of the pipeline into a reusable and scalable ROS2 service. Each processing stage: depth estimation, optical flow, motion tracking, object clustering, and risk assessment, is encapsulated as an independent component, allowing flexible testing, extension, and integration with additional modules.

In contrast to previous work at the FTM [20–22], which used ADTF or partial ROS1 based implementations with limited deployment, this thesis delivers a fully containerized, real-time capable perception system. The system is tested under real-world conditions using the Framos D455e stereo camera and the EDGAR vehicle's PTP-synchronized infrastructure. Furthermore, a new egomotion compensation module is introduced to enable consistent alignment to the world frame, separating ego movement from external object motion.

Another important contribution is the integration of a motion-based clustering step, which aggregates coherent point trajectories into object-level representations. These clusters form the basis for the computation of runtime criticality metrics, a novel extension not considered in previous FTM work. The criticality module computes indicators such as Time-to-Collision (TTC) by considering the relative velocity and distance between moving obstacles and the ego trajectory. This enables quantitative risk assessment in teleoperated fallback scenarios, where robust scene understanding is required despite degraded sensing or connectivity or even an additional security layer when driving autonomously.

To meet the outlined goals and ensure reliable operation in real-world conditions, the system must fulfill the following functional and technical requirements:

1. **Real-time performance:** Minimum processing frequency of 10 Hz to meet vehicle control latency constraints.
2. **Stereo-only sensing:** Exclusive use of stereo vision to reduce hardware cost and simplify sensor integration.
3. **Semantic independence:** Object detection must operate without class labels or training data, enabling generalization to unknown object types.
4. **Egomotion compensation:** Accurate transformation of point observations into a consistent ego vehicle frame.

5. **6D output:** Estimation of both 3D position and velocity for all detected scene points or objects.
6. **Temporal consistency:** Persistent motion tracking over time through recursive filtering and data association.
7. **Clustering:** Aggregation of dynamic points into coherent object-level clusters for further analysis.
8. **Criticality estimation:** Online computation of risk metrics based on relative motion and ego trajectory.
9. **Modular ROS2 architecture:** Each processing stage implemented as an independent ROS2 service to support reusability and scalability.
10. **Robustness to occlusion:** Reliable operation in presence of partial or intermittent visibility.
11. **Parallelism and acceleration:** Use of CUDA and OpenMP to accelerate compute-intensive components.
12. **Portability and deployment:** Docker-based containerization for reproducible deployment on the EDGAR research platform.

These requirements are derived from both the theoretical analysis in the literature and the practical constraints of the target deployment platform. They reflect the system-level vision of enabling a deployable, deterministic, and transparent perception pipeline as a fallback for learned approaches in automated driving.

3 Methods and Implementation

This chapter presents the methodological and implementation framework that form the core the system developed in this thesis. The system is designed to perform deterministic, stereo camera-based environment perception and criticality assessment for automated vehicles, with an emphasis on real-time applicability and modular deployment.

The chapter begins by outlining the high-level architecture, describing the interaction between core subsystems that process raw visual data and generate semantically structured information regarding the dynamic environment. This architecture is conceptually divided into two main pipelines: the perception pipeline, which estimates the motion of surrounding objects based on the stereo camera input, and the criticality pipeline, which evaluates the potential risk posed by these objects relative to the ego-vehicle trajectory.

Subsequent sections will provide a detailed examination of each of the individual components presented in the overall architecture. Special emphasis will be placed on the design choices made to ensure real-time performance, modularity, and ease of integration with the existing software infrastructure of the EDGAR research vehicle.

The modular decomposition of the system facilitates scalability, systematic testing, and independent development of components. It also ensures compatibility with the software infrastructure of the EDGAR research vehicle, enabling seamless integration with existing modules responsible for data acquisition, actuation, and supervisory control. Through this design, the system provides a reproducible and extensible foundation for further development in the context of teleoperated and autonomous driving research.

3.1 System Architecture

In this section the high level architecture of the system will be presented.

The system is conceptually divided into two main pipelines: the perception pipeline and the criticality pipeline. The **perception pipeline** processes raw sensor data captured by the stereo camera to build a dynamic representation of the environment and estimating the position and speed vectors of the moving objects. The output of this pipeline is a list of object in the scene that is later used by the **criticality pipeline** to compute the criticality score of the detected objects.

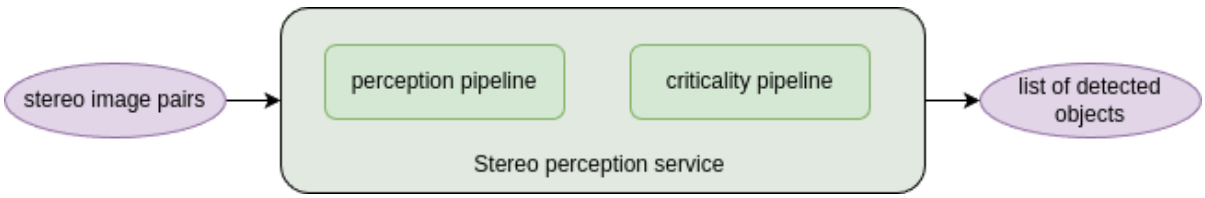


Figure 3.1: High level overall system

This section introduces the high-level architecture of the system developed in this thesis.

The overall system is structured into two principal pipelines: the **perception pipeline** and the **criticality pipeline**. The perception pipeline operates on raw stereo image data, constructing a dynamic model of the environment by estimating the 3D positions and velocity vectors of moving objects. Its output is a structured list of detected objects, which serves as input to the criticality pipeline. The latter evaluates the relative risk posed by each object with respect to the ego-vehicle's planned trajectory by computing relevant criticality metrics.

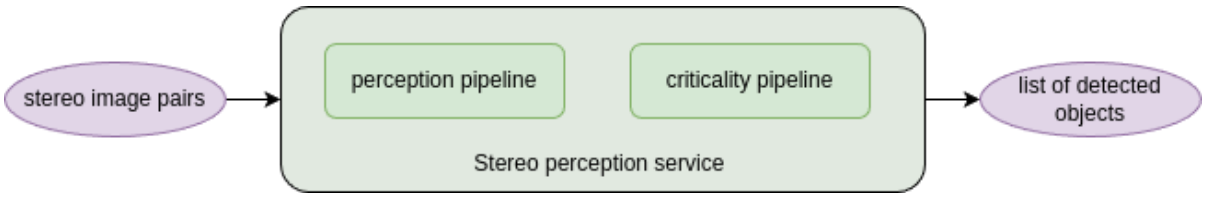


Figure 3.2: High-level overview of the system

Both pipelines are deployed jointly as a single ROS 2 service on the EDGAR research vehicle. This design decision ensures a clean separation between external interfaces and internal implementation, thereby promoting system portability across different vehicle platforms and stereo camera configurations. The service exposes standardized ROS 2 messages for input and output, enabling seamless integration within heterogeneous software stacks. Internally, the system remains agnostic to vehicle-specific details, facilitating reuse and simplifying deployment across varied hardware setups.

3.1.1 Perception pipeline

The perception pipeline will be responsible for detecting moving objects and estimating their position and current velocity (the 6D motion field). The pipeline consists of several submodules as illustrated in figure 3.3.

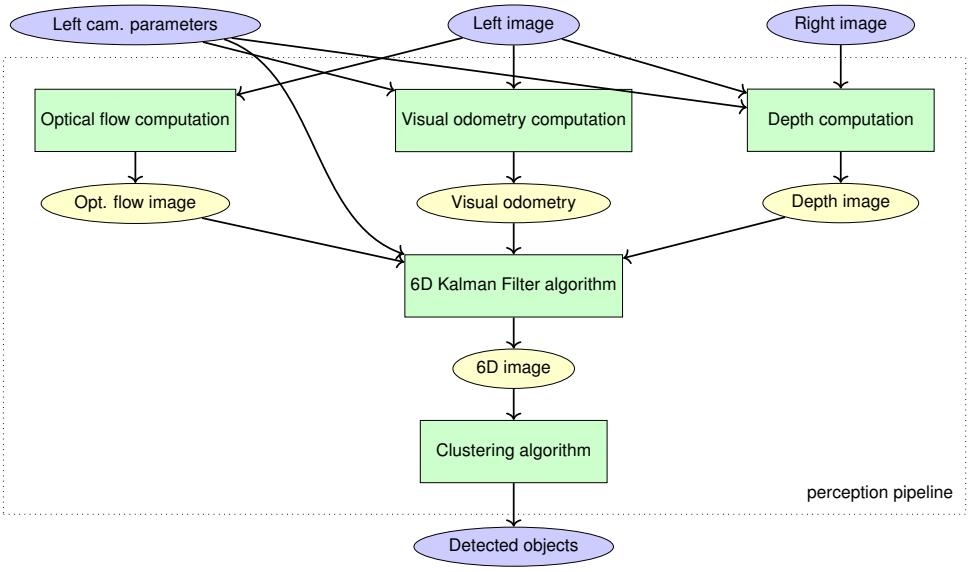


Figure 3.3: High level perception pipeline

As shown in the figure, the perception pipeline comprises the following core modules:

1. **Optical flow computation:** Computes a dense optical flow field from the left image sequence using the Farneback algorithm [36].
2. **Visual odometry:** Estimates the ego-motion of the vehicle using stereo image sequences

3. **Depth computation:** Computes a dense depth image from the stereo image pair using a semi-global matching (SGM) algorithm.
4. **Kalman filter:** Central module of the perception pipeline. It uses depth and optical flow data to estimate the 6D state (position and velocity) of a sparse set of points over time.
5. **Object detection:** Applies a clustering algorithm (DBSCAN [37]) to group similar 6D points into detected objects.

Each of the previous presented modules will be implemented as a separate ROS2 package, allowing for modular development and testing.

3.1.2 Criticallity pipeline

The criticality pipeline (Figure 3.4) assesses the risk posed by each detected object. Based on the output of the perception pipeline and the current trajectory of the ego-vehicle, it computes criticality metrics such as the Time-to-Collision (TTC) [38]. These metrics serve as indicators of potential hazards in the environment.

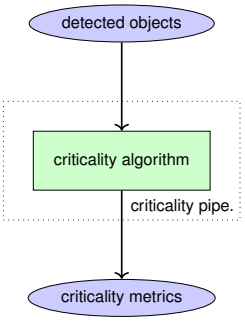


Figure 3.4: High level criticality pipeline

This pipeline is designed to be independent of the perception pipeline, allowing for the possibility of integrating different criticality assessment algorithms in the future on top of the perception pipeline. In this project it only implements the TTC algorithm, which will also be implemented as a separate ROS2 package.

3.1.3 Repository structure

The project is implemented as a monorepo, consolidating all system components into a single, unified repository to simplify development, integration, and deployment.

The repository is structured into three main directories:

1. `src/` Contains all ROS 2 source code.
   (a) `launch_packages/` Contains all launch files used to start the system.
      i. `perception_pipeline_launch/` Package that contains the launch files for the perception pipeline.
      ii. `criticallity_pipeline_launch/` Package that contains the launch files for the criticallity pipeline.
   (b) `perception_pipeline/` Contains all the packages used in the perception pipeline.
      i. `camera_info_publisher/` Package that republishes and syncronizes color image topic and camera info topic with the depth image topic.

   ii. `stereo_computation/` Package to compute a depth image from a pair of stereo images.
   iii. `optical_flow_computation/` Package to compute the optical flow from an image topic.
   iv. `kalman_filter/` Package that performs the 6D state estimation of the points in the scene.
   v. `object_detector/` Package that performs the object detection and clustering of the points in the scene.
(c) `criticallity_pipeline/` Contains all the packages used in the criticality pipeline.
   i. `ttc_calculator/` Package that computes the Time-to-Collision (TTC) of the detected objects.
(d) `utils/` Contains all the packages that are independent of the perception and criticality pipelines.
   i. `stereo_perception_msgs/` Contains all the custom messages used in the repository.
   ii. `disparity_to_depth/` Package that converts disparity images to depth images.
   iii. `stereo_perception_edgar_bridge/` Package that contains a bridge between the custom messages used in the EDGAR vehicle and standard ROS2 messages
   iv. `tod_msgs/` Custom messages used in the EDGAR vehicle (git submodule).
   v. `tum_msgs/` Custom messages used in the EDGAR vehicle (git submodule).

2. `docker/` Contains all Docker image related files, including the Dockerfile and configuration files.

   (a) `perception_pipeline/` Contains all the Dockerfiles and configuration files for the perception pipeline.
   (b) `criticallity_pipeline/` Contains all the Dockerfiles and configuration files for the criticallity pipeline.
   (c) `stereo_perception_edgar_bridge/` Contains all the Dockerfiles and configuration files for the bridge between the custom messages used in the EDGAR vehicle and standard ROS2 messages.

3. `non_ros/` Contains all non ROS2 (e.g. Latex files, scripts, etc.) files.

3.1.4 Containerization and Deployment

Docker is a platform that enables the execution of software in isolated environments called containers. These encapsulate code and dependencies, ensuring consistency across development and deployment environments.

To guarantee reproducibility and simplify integration, the system was deployed using Docker. Initially, all components were combined into a single container to reduce inter-process communication and avoid the latency introduced by message serialization between modules. This design allowed the perception modules, which are tightly coupled, to share memory space and operate with minimal data transfer overhead.

However, to increase modularity and facilitate portability across platforms, the final deployment was structured into three separate Docker images. The first container hosts the entire perception pipeline. It receives stereo

camera data and generates a list of detected objects. The second container contains the criticality pipeline, which computes criticality metrics based on object detection results and ego-vehicle trajectory. The third container serves as a ROS 2 bridge that converts custom messages from the EDGAR platform into standard message types used by the other two containers.

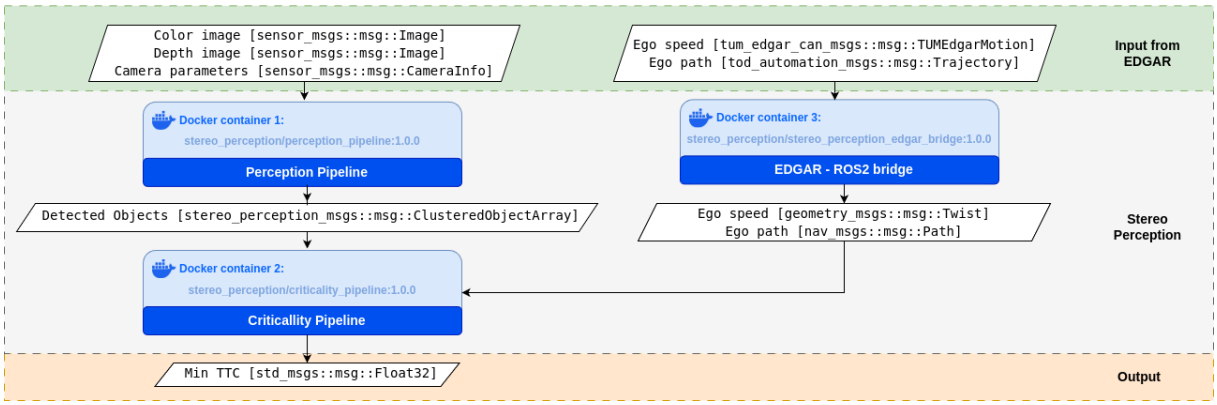


Figure 3.5: Container architecture and data flow

This separation allows both the perception and criticality pipelines to operate independently of EDGAR-specific message definitions, thus making them deployable in other platforms with minimal adjustments.

Each container is organized with a consistent directory structure:

- `config/`: Configuration files in YAML format.
- `scripts/`: Launch and runtime scripts.
- `Dockerfile`: Build instructions for the container image.

The system relies on GPU acceleration for computationally intensive tasks, particularly for processing stereo vision and optical flow using CUDA-enabled OpenCV. The containers are built upon NVIDIA CUDA base images, and OpenCV is compiled from source with CUDA support.

Deployment requires a host system with a compatible NVIDIA GPU and the NVIDIA Container Toolkit. The implementation is optimized for CUDA 12.4 and Compute Capability 8.0, 8.6, and 9.0. Compatibility details can be found in the official NVIDIA documentation [39].

If the host system lacks a compatible GPU, the container must be rebuilt with adjusted CUDA settings in the Dockerfile.

3.2 Perception pipeline: Stereo image computation

The stereo image computation module is responsible for generating a dense depth map from a pair of rectified stereo images and its calibration parameters. This component is particularly relevant when working with datasets or sensors that provide raw stereo image pairs but no precomputed depth information. Although this module is not used in the final deployment of the system on the EDGAR research vehicle, since its stereo camera setup provides depth images directly—it plays a crucial role in training, testing, and benchmarking the system using external datasets. It can also be used in the future to integrate other stereo camera sets that do not provide depth images.

3.2.1 Mathematical Background: Pinhole Camera Model and Disparity

In stereo vision, depth estimation relies on the triangulation of corresponding points in a pair of images captured from slightly different viewpoints. The pinhole camera model (figure: 3.6) is commonly used to describe the relationship between 3D world coordinates and 2D image coordinates. In this model, a 3D point $\mathbf{P} = (X, Y, Z)$ is projected onto the image plane as a 2D point $\mathbf{p} = (x, y)$ using the camera's intrinsic parameters ([10]). These parameters include the focal length f and the principal point (c_x, c_y), which define the camera's optical center and field of view.

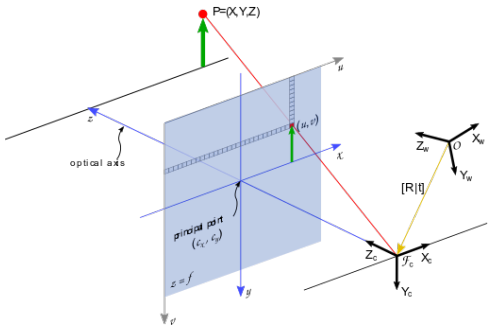


Figure 3.6: Pinhole camera model. Source: Nvidia

The relationship between the 3D point and its 2D projection is given by:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \tag{3.1}$$

where f_x and f_y are the focal lengths in pixels, and (c_x, c_y) is the principal point.

In a typical stereo setup, the two cameras are displaced horizontally along the x-axis, with their optical axes kept parallel. This physical separation between the cameras is known as the **baseline** B, as seen in the figure 3.7 left. The baseline is a key parameter in stereo vision, as it directly influences the triangulation accuracy: a larger baseline improves depth resolution, especially for distant objects, but also increases the difficulty of finding correct correspondences due to greater viewpoint differences.

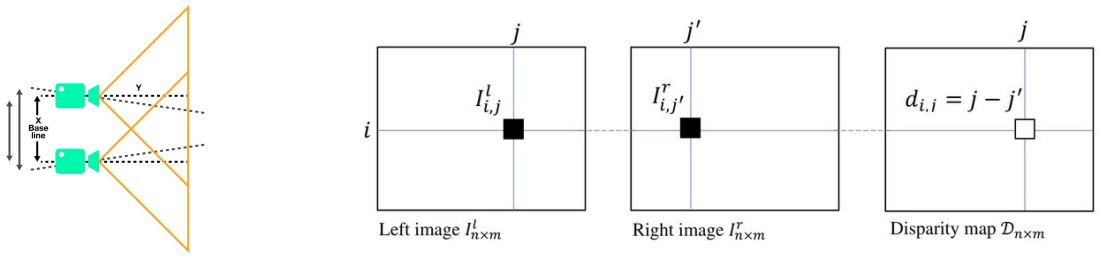


Figure 3.7: Stereo camera setup (left) and disparity concept (right). Source: Framos, [40].

Thanks to this horizontal arrangement, the correspondence search simplifies to a one-dimensional problem along the image rows. After rectification, corresponding points between the two images lie along the same horizontal line (epipolar line), and the **disparity** d (figure 3.7 right) between corresponding points is defined as:

$$d = x_{\text{left}} - x_{\text{right}} \tag{3.2}$$

Disparity is inversely proportional to the depth of the observed point. The relation between disparity and depth Z is given by:

$$Z = \frac{f \cdot B}{d} \tag{3.3}$$

3.2.2 Stereo Matching Algorithms

Stereo matching algorithms are responsible for computing the disparity map between a pair of rectified stereo images. Given that disparity directly relates to depth, these algorithms are a critical component of the stereo vision pipeline. Their main task is to find, for each pixel in the left image, the corresponding pixel in the right image along the same horizontal line.

Several approaches exist to solve this correspondence problem, each with different trade-offs between computational complexity, robustness, and accuracy. Some of the most common implemented stereo matching algorithms include:

- **Block Matching (BM):** A local method that compares fixed-size blocks between the left and right images. While computationally efficient, it often struggles with textureless regions and depth discontinuities ([41]).
- **Semi-Global Matching (SGM):** An algorithm that balances accuracy and computational efficiency by aggregating matching costs along multiple paths through the image ([42]).

Comparative studies ([43]) have shown that SGM offers a good trade-off between accuracy and performance, making it suitable for real-time applications where both speed and quality are essential.

3.2.3 Semi-Global Matching (SGM)

Semi-Global Matching (SGM) is a stereo matching algorithm that offers a compromise between the accuracy of global optimization methods and the computational efficiency of local methods. It was first introduced by Hirschmüller [42] and has become widely implemented in real-time applications.

The underlying principle in SGM is to minimize a global energy function that imposes continuity in the disparity map, having strong depth discontinuities along object boundaries. The energy function $E(D)$ for a disparity map D is defined as:

$$E(D) = \sum_{\mathbf{p}} \left(C(\mathbf{p}, D_{\mathbf{p}}) + \sum_{\mathbf{q} \in \mathcal{N}_{\mathbf{p}}} P_1 \cdot \delta\left(|D_{\mathbf{p}} - D_{\mathbf{q}}| = 1\right) + P_2 \cdot \delta\left(|D_{\mathbf{p}} - D_{\mathbf{q}}| > 1\right) \right) \tag{3.4}$$

where:

- $C(\mathbf{p}, D_{\mathbf{p}})$ is the matching cost at pixel $\mathbf{p}$ for disparity $D_{\mathbf{p}}$.
- $\mathcal{N}_{\mathbf{p}}$ denotes the set of neighboring pixels.
- P_1 and P_2 are penalty terms for disparity changes of 1 and greater than 1, respectively.
- δ is the Kronecker delta function.

The algorithm operates in two main steps:

1. Cost computation: For each possible disparity, a matching cost calculation happens between the left picture plus the right picture at each pixel. This calculation uses methods like the Sum of Absolute Differences or measurements that use census transform.

2. Cost aggregation: SGM does not do a full 2D global optimization. That process demands a lot of computation, so it is not practical for systems that need to run in real time. SGM comes close to global reasoning when it combines costs along several 1D paths. Costs are put together along either 4 or 8 directions from each pixel. A dynamic programming method finds the lowest local cost in each direction. It also adds penalties for significant changes in disparity.

Once the cost values are added together from all directions, the disparity picked for each pixel is the one that has the lowest combined cost.

This balance between accuracy and computational efficiency makes SGM highly attractive for embedded and real-time applications.

3.2.4 ROS2 Implementation

The SGM algorithm has been integrated into the pipeline with a dedicated ROS2 package called `stereo_computation`.It contains a node that subscribes to three topics: the rectified left and right camera images, and the camera info topic of the left camera. After receiving synchronized input, the node computes both the disparity map and the depth map, which are then published on two separate ROS2 topics for downstream processing.

The stereo matching algorithm was implemented using the OpenCV library. In particular, the `cv::cuda::StereoSGM` class was chosen to compute the disparity map. This class offers a CUDA-accelerated version of the Semi-Global Matching algorithm, enabling the computations to be performed directly on the GPU. Taking advantage of GPU acceleration is crucial to meet the real-time performance requirements of the perception pipeline.

The OpenCV implementation of SGM is highly optimized and designed to exploit the parallel architecture of modern GPUs, significantly reducing the runtime compared to CPU-based methods. After computing the disparity map, the depth map is generated by applying the classical pinhole camera equation:

$$Z = \frac{f \cdot B}{d} \tag{3.5}$$

where Z is the depth value, f is the focal length of the camera, B is the baseline distance between the two cameras, and d is the disparity.

This node architecture ensures modularity within the perception pipeline and allows easy integration with other modules, such as the Kalman filtering and object detection components. Furthermore, by publishing both the disparity and depth images, the system maintains flexibility for future extensions or algorithm improvements that may require direct access to raw disparity data.

To make the tuning and implementation easier, the following ROS2 parameters were added to the node:

ROS2 parameter	Definition
in_left_image_topic	Topic name for the left input grayscale image
in_right_image_topic	Topic name for the right input grayscale image
in_camera_info_topic	Topic name for the camera calibration information
out_depth_image_topic	Output topic for the computed depth image
out_disparity_image_topic	Output topic for the computed disparity image
min_disparity	Minimum disparity to consider for the disparity calculation
num_disparities	Number of disparities (disparity range) to compute
P1	Control parameter for the SGM algorithm (penalty for small disparity changes)
P2	Control parameter for the SGM algorithm (penalty for small disparity changes)
uniqueness_ratio	Threshold for uniqueness of matching points (higher means stricter matching)
speckle_window_size	Size of the window used for speckle filtering
speckle_range	Maximum disparity range for speckle filtering
pre_filter_cap	Pre-filtering parameter for the image to improve disparity computation
use_sim_time	Flag to use simulation time instead of system time

and the node can be added to a launch file as seen in the following example:

Code 3.1: Stereo computation launch

```
launch_ros.actions.Node(
            package='stereo_computation',
            executable='stereo_computation_node',
            output='screen',
            parameters=[
              {"in_left_image_topic": input_gray_left_topic},
              {"in_right_image_topic": input_gray_right_topic},
              {"in_camera_info_topic": input_camera_info_topic
 },
              {"out_depth_image_topic": "perception_pipeline/
 depth_image"},
              {"out_disparity_image_topic": "
 perception_pipeline/disparity"},
              {"min_disparity": 0},
              {"num_disparities": 256},
              {"P1": 30},
              {"P2": 200},
              {"uniqueness_ratio": 15},
              {"speckle_window_size": 50},
```

```
17                  {"speckle_range": 2},
18                  {"pre_filter_cap": 31},
19                  {"use_sim_time": use_sim_time}
20                  ],
21              ),
```

3.3 Perception pipeline: Optical flow computation

This section describes the optical flow computation module within the perception pipeline. This module is responsible for estimating the relative motion of the pixels of the image between two consecutive frames.

3.3.1 Optical flow concept

The concept of Optical Flow, first introduced by James Gibson in the 1950s ([44]) refers to the displacements of intensity patterns. In our case it represents the relative motion of the pixels between two consequtive frames of a video stream. The output is a 2D vector field where each vector describes the position of a pixel or a small region in the current frame relative to its position in the previous frame.

This module is crucial for estimating the information of how the pixels in the image are moving, which is one of the essential inputs to the Kalman Filter stage.

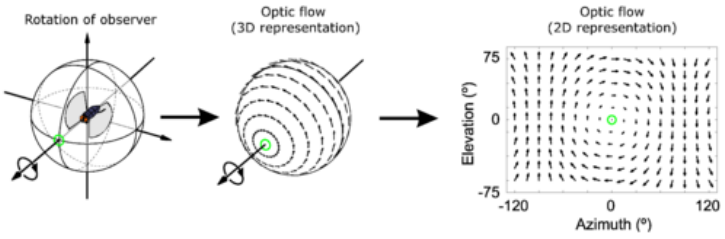


Figure 3.8: Optical flow

Optical flow estimation relies on the assumotion that the brightness of a pixel remains constant over time [45, 46], which allows the system to track the movement of objects based on the changes in pixel intensity. Mathematically, this assumption leads to the optical flow constraint equation:

$$I_x u + I_y v + I_t = 0 \tag{3.6}$$

Where:

- I_x and I_y are the spatial gradients of the image intensity in the x and y directions.
- I_t is the temporal gradient of the image intensity (the difference between two consecutive frames).
- u and v are the optical flow components in the x and y directions.

Since there is one equation and two unknowns, the optical flow constraint equation is underdetermined. To solve this problem, additional constraints or assumptions are needed, leading to different optical flow algorithms.

3.3.2 Optical flow algorithms

Optical flow methods can be classified into two main categories: sparse and dense optical flow.

Sparse optical flow methods focus on estimating motion vectors at a discrete set of key points in the image. These points are typically features such as corners or edges that can be tracked reliably across frames. In contrast, **dense optical flow** methods aim to estimate a motion vector for every pixel in the image, providing a complete 2D displacement field.

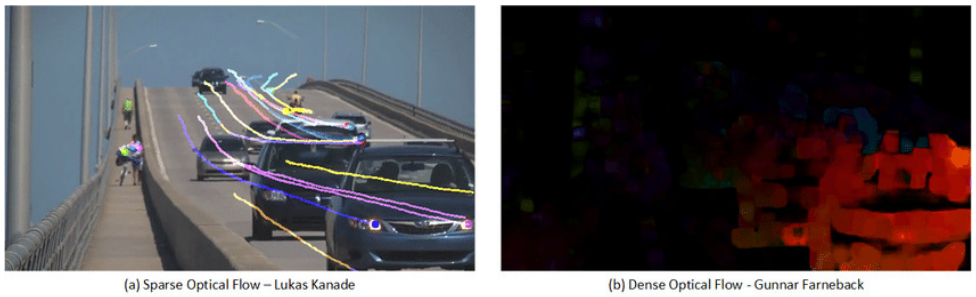


Figure 3.9: Sparse vs dense optical flow. Source: [47]

Recently, deep learning-based methods have gained popularity and demonstrated impressive performance for optical flow estimation. Models such as FlowNet [48] and RAFT [49] leverage convolutional neural networks (CNNs) to learn spatial and temporal features from large datasets, enabling them to predict optical flow fields with high accuracy. However, these methods often require significant computational resources and may suffer from poor generalization to unseen environments.

For these reasons, and due to the necessity of robustness and real-time performance, this work focuses on traditional deterministic optical flow algorithms. These classical methods also offer the advantage of being widely implemented and optimized in existing libraries, which significantly accelerates development and integration time.

Some of the most common optical flow algorithms include [50, 51]:

- **Lucas-Kanade method [52]:**

  Introduces in 1981 by Lucas and Kanade, this os a sparse optical flow method based on the assumption of constant optical flow within a small local neighborhood. It solves the optical flow constraint equation using a least-squares approach. This method is simple and computationally efficient, but it is sensitive to large motions and requires good texture in the tracked regions.

- **Horn-Schunck method [17]:**

  Proposed by Horn and Schunck, this method computed dense optical flow by introducing a global smoothness constraint. It formulates flow estimation as the minimization of an enegy function that combines the brightness constancy constraint and a smoothness term. While robust to moredate noise and capable of estimating full dense flow, the method can oversmooth motion boundaries and struggles with large displacements.

- **TV-L1 method [53]:**

  This approach refines the variational method of Horn and Schunck by introducing a total variation (TV) regularization term and an L1 norm for the data term. The resulting algorithm is more robust and was implemented in the previous work of Bauer [20]. Nevertheless, it is computationally expensive and may not be the best choice for real-time applications.

- **Farneback method [36]:** Farneback introduced a dense optical flow method based on local polynomial expansions. The idea is to approximate the neighborhood of each pixel ith a quadratic polynomial and then estimate how the polynomials change between frames.

  This method offers a good trade-off between accuracy and computational efficiency. When accelerated using GPUs (e.g. via OpenCV CUDA modules), it can achieve real-time performance even on high-resolution images.

In this work, the Farneback method has been selected due to its computational efficiency, robustness for medium-range motions, and the availability of an optimized CUDA implementation.

3.3.3 Farneback algorithm

The Farneback optical flow method is a dense flow estimation algorithm based on polynomial expansion of local image neighborhoods. Its core idea is to approximate the intensity variation in a small image region using a quadratic polynomial, and then infer the motion between frames by analyzing how these polynomials deform over time.

The first step is to approximate the image intensity function $I(x, y)$ in a local neighborhood using a second-order Taylor expansion:

$$I(x, y) \approx x^T Ax + b^T x + c \tag{3.7}$$

where:

- $x = (x, y)^T$ represents the spatial coordinates,
- A is a symmetric 2×2 matrix encoding the second-order spatial derivatives,
- b is a 2×1 vector representing the first-order spatial derivatives,
- c is a scalar term corresponding to the local intensity offset.

This local polynomial approximation can be efficiently computed over the entire image using Gaussian-weighted least squares fitting, a process known as polynomial expansion.

When the scene undergoes a pure translation $d = (d_x, d_y)^T$ between two consecutive frames, the transformed intensity function $I'(x)$ can be related to the original by:

$$I'(x) = I(x - d) \tag{3.8}$$

Substituting the polynomial approximation yields:

$$I'(x) \approx (x - d)^T A(x - d) + b^T(x - d) + c \tag{3.9}$$

Expanding the right-hand side leads to:

$$I'(x) \approx x^T Ax + (b - 2Ad)^T x + (d^T Ad - b^T d + c) \tag{3.10}$$

Comparing terms, we observe that:

$$A' = A \tag{3.11}$$
$$b' = b - 2Ad \tag{3.12}$$
$$c' = d^T Ad - b^T d + c \tag{3.13}$$

where A' and b' represent the new polynomial coefficients after displacement.

Thus, the displacement d can be estimated by solving the linear system:

$$2Ad = b - b' \tag{3.14}$$

assuming that A is non-singular and well-conditioned.

In practice, direct estimation is complicated by noise, local deformations, and the limited validity of polynomial approximations. To address these issues, the following refinements are applied:

- Estimate the displacement over a local window using Gaussian weighting to prioritize central pixels.
- Aggregate local estimates using weighted least squares to improve robustness against noise.
- Employ a multi-scale coarse-to-fine pyramid approach to capture both small and large displacements effectively.

Coarse-to-fine pyramid approach

Large displacements between frames can cause local matching failures if only a single resolution is used. To handle this, the Farneback method employs a coarse-to-fine multi-scale pyramid strategy [54].

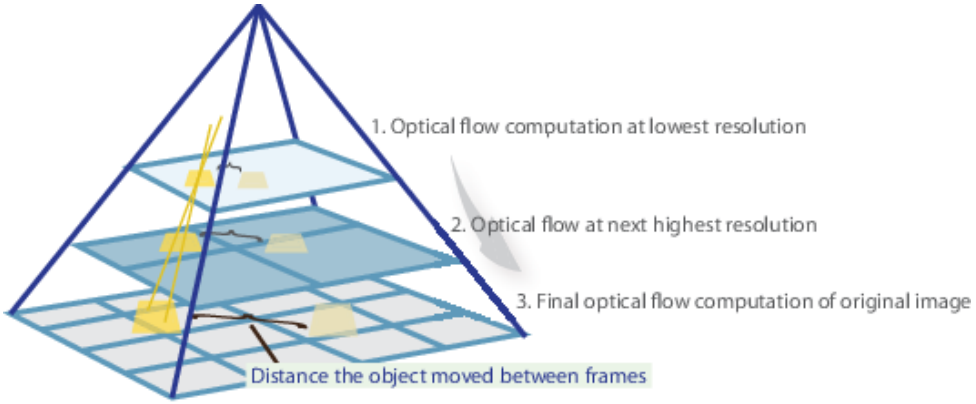


Figure 3.10: Coarse-to-fine pyramid approach. Source: Matlab

In the pyramid approach:

- The original images are repeatedly downsampled by a scaling factor (typically 0.5), producing several lower-resolution versions.
- Optical flow is first estimated at the coarsest (smallest) scale, where large motions appear reduced and easier to estimate.
- The initial flow estimate is then propagated to the next finer scale, where it is refined using the higher-resolution image data.
- This process continues until the finest scale (the original resolution) is reached, producing a dense and detailed optical flow field.

The pyramid-based coarse-to-fine strategy is crucial for ensuring that large motions do not overwhelm the local polynomial approximation, maintaining both the accuracy and robustness of the Farneback method.

This hierarchical refinement significantly enhances the capability of the algorithm to deal with large inter-frame displacements, dynamic scenes, and varying object velocities.

3.3.4 Implementation

This algorithm has been implemented in a ROS2 package called `optical_flow_computation`. This package contains a node responsible for computing the optical flow between two consecutive images. The node subscribes to two topics: the first for the left image and the second for the right image. Once it receives both images, it computes the optical flow and publishes the result on an output topic.

Regarding the algorithm implementation, the `cv::cuda::FarnebackOpticalFlow` class from OpenCV was used. This class provides a CUDA-accelerated version of the Farneback algorithm, allowing computations to be performed directly on the GPU. The OpenCV implementation of the Farneback algorithm is highly optimized and designed to leverage the parallel architecture of modern GPUs, significantly reducing execution time compared to CPU-based methods. This class is included in the header `opencv2/cudaoptflow.hpp`, which generally requires manually building OpenCV with CUDA support.

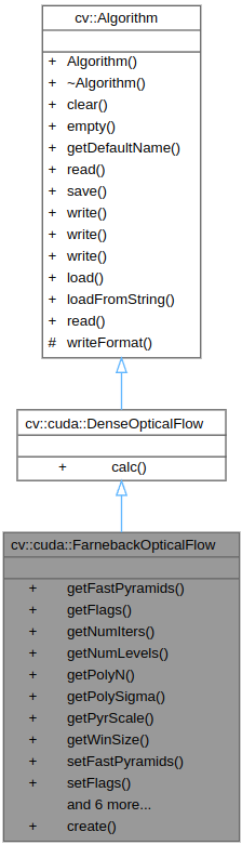


Figure 3.11: CUDA Farneback class. Source: OpenCV

To facilitate tuning and configuration, the following ROS2 parameters were added to the node:

Table 3.1: ROS2 parameters for the optical flow computation node

ROS2 parameter	Definition
image_topic	Input topic name for raw RGB images used in optical flow estimation
optical_flow_topic	Output topic name where the computed optical flow is published
pyr_scale	Scaling factor between pyramid levels
levels	Number of pyramid layers, including the original image
winsize	Averaging window size for smoothing derivatives; larger values improve robustness but may oversmooth motion boundaries
iterations	Number of iterations at each pyramid level; higher values improve convergence in complex motion areas
poly_n	Size of the pixel neighborhood used for polynomial expansion
poly_sigma	Standard deviation of the Gaussian used to smooth derivatives before polynomial expansion
flags	Operation flags passed to the Farneback function
use_sim_time	Boolean indicating whether to use simulation time (`/clock`) instead of system time

The node can be added to a launch file as shown in the following example:

Code 3.2: Optical flow computation launch

```
launch_ros.actions.Node(
            package='optical_flow_computation',
            executable='optical_flow_node',
            output='screen',
            parameters=[
              {"image_topic": input_gray_left_topic},
              {"optical_flow_topic": "perception_pipeline/
    optical_flow"},
              {"pyr_scale": 0.5},
              {"levels": 3},
              {"winsize": 15},
              {"iterations": 3},
              {"poly_n": 5},
              {"poly_sigma": 1.2},
              {"flags": 0},
              {"use_sim_time": use_sim_time}
              ],
          ),
```

The output image is published on the topic `/perception_pipeline/optical_flow`. This image is published as a `sensor_msgs/Image` message with a `32FC2` encoding format, representing a 32-bit floating-point optical flow image with two channels. The first channel corresponds to the displacement along the x-axis, and the second channel corresponds to the displacement along the y-axis.

In addition to the flow image, two debug images are published on the `/debug` topic to facilitate debugging and result visualization. One debug image shows the original image with optical flow vectors superimposed, and the other displays an RGB image where the intensity represents the magnitude of the optical flow vector and the color represents its direction.

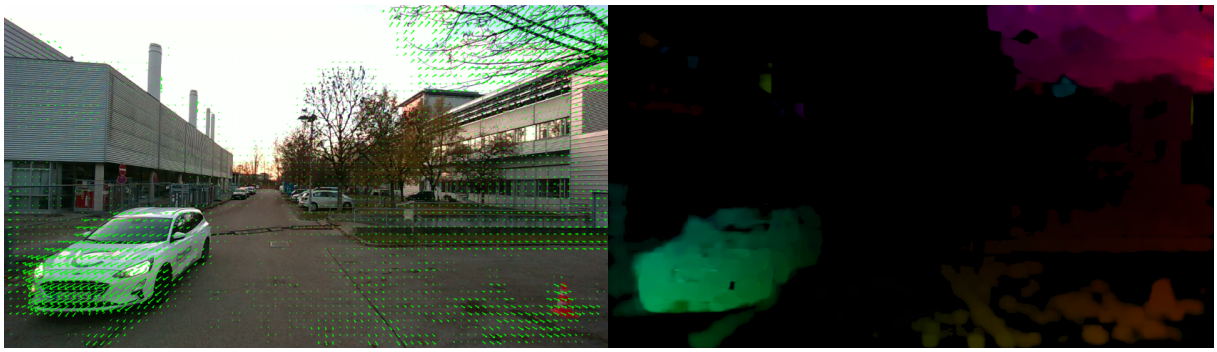


Figure 3.12: Optical flow debug images. Left: Optical flow vectors superimposed on the original image. Right: RGB image where the intensity represents the magnitude and the color represents the direction of the optical flow vector. Source: own work

These additional debug outputs greatly facilitate the qualitative evaluation of the optical flow results and enable faster identification of potential errors during the development and testing phases.

3.4 Perception pipeline: Ego motion estimation

In the context of autonomous and automated vehicles, estimating the ego motion is crucial to understand how the vehicle is moving in the environment. In the scope of this work, ego motion is defined as the relative motion of the vehicle with respect to the environment. Specifically, the 3D position of the camera in one frame, relative to the previous frame.

This information is essential for the Kalman Filter step where the 6D field will be estimated, since the optical flow output is relative to the camera frame, and the Kalman Filter requires knowledge of the camera's position in the world frame to properly compensate for its motion.

For this purpose, visual odometry (VO) techniques are used to estimate the required information using the stereo setup input. Incorporating the depth information from the stereo camera allows the direct recovery of metric information, providing robustness and accuracy.

This section describes the concept behind visual odometry, the coordinate systems used, the specific algorithmic approach that was implemented, as well as the ROS2 implementation details.

3.4.1 Visual odometry

Visual odometry (VO) refers to the process of estimating the motion of a camera by analyzing a sequence of images it captures. Originally introduced by Moravec [55] and later formalized by Nistér et al. [56], VO has become a fundamental technique for mobile robotics and autonomous driving [57].

The core idea behind VO is to track visual features across consecutive frames and infer the relative motion of the camera from these observations. The estimated motion provides a continuous estimate of the vehicle trajectory without relying on external localization systems.

Visual odometry methods are generally categorized based on the type of input and processing approach used:

- **Monocular VO:** Utilizes a single camera as input. It is cost-effective but suffers from scale ambiguity, meaning that the absolute scale of the motion (e.g., in meters) cannot be determined without additional information. Monocular VO is often used in conjunction with other sensors (e.g., IMU) to resolve this ambiguity [57].
- **Stereo VO:** Used two synchronized cameras to estimate depth through triangulation, which allows direct recovery of metric scale. This removes the scale ambiguity present in monocular systems, producing more accurate and robust motion estimation. Stereo VO is especially beneficial in environments where monocular methods struggle, such as in low-texture areas or during large baseline movements, thanks to the additional geometric constraints provided by the known baseline distance between the cameras [57].

Processing approaches are typically divided into two categories:

- **Feature-based methods:** Extract and match features (e.g., corners, edges) across frames, estimating motion from the displacement of these features.
- **Direct methods:** Directly use pixel intensities to estimate motion without explicit feature extraction.

Recently, **deep learning-based visual odometry** approaches have emerged, combining different types neural networks to estimate motion directly from image sequences. These methods aim to improve robustness under challenging conditions, such as illumination changes and textureless environments [57].

This work adopts a **Stereo VO:** and **stereo feature-based** since the depth information is already available from the stereo Camera and using feature matching to estimate motion robustly. This choice provides a good balance between accuracy and computational efficiency, making it suitable for real-time applications

3.4.2 Coordinate systems

To properly understand the visual odometry algorithm and this work, it is important to define the coordinate systems used in the implementation. The following coordinate frames are defined in the context of this project:

- **camera_optical_frame** The optical coordinate frame attached to the camera at the current time step. All of the optical frames follow the ROS2 convention, where the z-axis points forward, the x-axis points to the right of the camera, and the y-axis points downwards.
- **camera_optical_frame_previous** The optical coordinate frame attached to the camera at the previous time step of the visual odometry computation.
- **camera_optical_frame_initial** The coordinate frame attached to the camera at the start of the sequence.
- **world** Coincident with camera camera_optical_frame_initial but rotated. Z-axis points up, x-axis points forward, and y-axis points to the left.

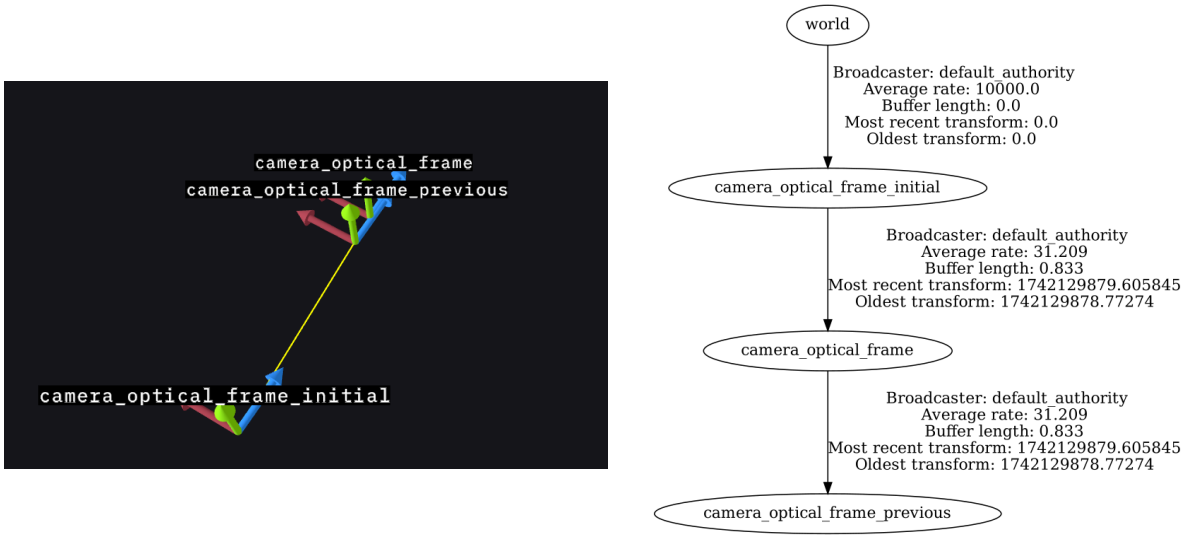


Figure 3.13: Coordinate frames and ROS2 TF tee. Left: Coordinate frames visualized using Foxglove. Right: ROS2 Transform Tree of the project generated using tf2_tools. Source: own work

Visual odometry algorithm

The visual odometry algorithm implemented in this work follows a stereo vo and feature-based stereo approach. It estimates the relative motion of the camera by detecting and matching visual features between consecutive frames, reconstructing their 3D positions using the depth image, and solving the camera pose using a Perspective-n-Point (PnP) algorithm [58] with RANSAC for outlier rejection.

A schematic overview of the algorithm steps is presented in Algorithm 1.

Algorithm 1 Stereo Visual Odometry Pipeline

Require: RGB images I_{k-1}, I_k, and depth image D_{k-1}
Ensure: Estimated relative pose (R, t) between frames

1: Detect ORB features in I_{k-1} and I_k using GPU acceleration
2: Match ORB features between I_{k-1} and I_k
3: Reproject matched keypoints from I_{k-1} into 3D using D_{k-1}
4: Solve PnP with RANSAC using 3D-2D correspondences
5: Refine the pose estimation with standard solvePnP using inliers
6: Estimate covariance based on reprojection residuals
7: Apply statistical filtering and optional exponential smoothing
8: Output (R, t), covariance matrix, and debug images

The code of the algorithm is implemented without any ROS2 dependencies, allowing it to be easily reused in other projects. In the next sections, the main steps of the algorithm are described in detail.

Feature detection and matching

Feature detection and matching constitute the first step in the visual odometry pipeline. In this phase, some points, that will serve to compute the desired transformation, will be detected in two consequtive frames. For this, the ORB (Oriented FAST and Rotated BRIEF) [59] algorithm will be used.

ORB is a binary feature descriptor that builds upon two classical methods: FAST (Features from Accelerated Segment Test) [60] for keypoint detection and BRIEF (Binary Robust Independent Elementary Features) [61] for descriptor computation.

In this context, a keypoint is a point in the image that is distinctive and repeatable (typically corners or blobs), that can be reliably identified across multiple frames. Around each keypoint, a descriptor is computed: a compact representation of the visual appearance in its local neighborhood. These descriptors allow the system to match corresponding features between frames, even under variations in viewpoint.

FAST is a corner keypoint detection algorithm that quickly identifies points in the image with significant intensity change compared to their surroundings. BRIEF, on the other hand, generates a binary string descriptor by comparing the intensity of random pairs of pixels in a small patch around the keypoint.

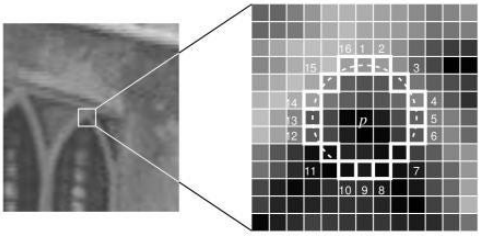


Figure 3.14: FAST algorithm. Source: [62]

ORB enhances these methods by introducing rotation and scale invariance. The orientation of each keypoint is computed to make BRIEF descriptors rotation-invariant, and a multi-scale pyramid is used to handle changes in scale.

The feature extraction and matching process is performed on the GPU using OpenCV's CUDA modules. This allows real-time processing even at high image resolutions.

The steps involved are:

1. Detect ORB features separately in the previous and current grayscale images.
2. Compute binary descriptors for each detected feature.
3. Match descriptors between the previous and current frames using brute-force Hamming distance.
4. Extract the coordinates of matched keypoints for subsequent motion estimation.

3D-2D correspondences

Once the feature matching step is completed, the next task is to establish 3D-2D correspondences between the previous and current frames. This is necessary for estimating the camera motion through Perspective-n-Point (PnP) techniques.

For each matched keypoint in the previous frame, the corresponding depth value is retrieved from the depth image associated with that frame. If the depth value is within a valid range, the 2D keypoint is reprojected into a 3D point using the intrinsic parameters of the stereo camera.

The reprojection follows the standard pinhole camera model:

$$X = (u - c_x) \cdot \frac{d}{f_x}, \quad Y = (v - c_y) \cdot \frac{d}{f_y}, \quad Z = d \tag{3.15}$$

where (u, v) are the pixel coordinates of the keypoint, (c_x, c_y) are the coordinates of the principal point, (f_x, f_y) are the focal lengths in pixels, and d is the depth value.

This process results in a set of valid 3D points extracted from the previous frame, and their corresponding 2D projections detected in the current frame. These 3D-2D pairs are later used to estimate the relative pose of the camera.

If the depth value is invalid (e.g., missing or outside the valid range), the corresponding match is discarded to avoid introducing noise into the motion estimation process. In the followign figure, a set of detected keypoint can be seen. The ones with valid depth are shown in green, while the ones with invalid depth are shown in red.

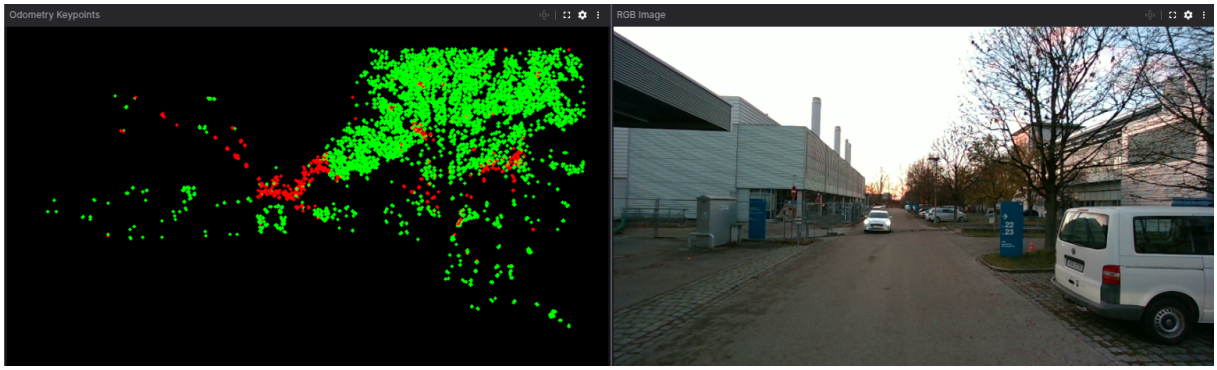


Figure 3.15: Odometry Keypoints with depth filtering and RGB Image. Source: own work

Motion estimation using PnP and RANSAC

Once 3D-2D correspondences are available, the relative camera motion between two frames can be estimated by solving the Perspective-n-Point (PnP) problem [58]. The goal is to compute the rotation $\mathbf{R}$ and translation $\mathbf{t}$ that map the 3D points observed in the previous frame to their 2D projections in the current image.

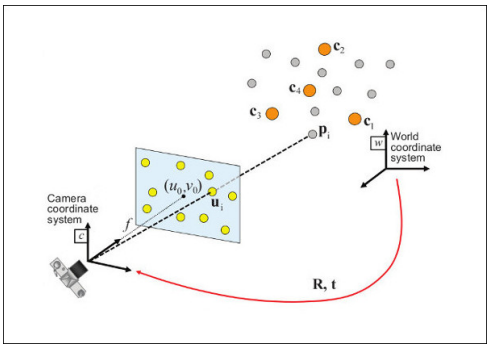


Figure 3.16: Perspectective-n-Point (PnP) problem. Source: [63]

Given a set of n correspondences $\{(\mathbf{X}_i, \mathbf{x}_i)\}$, where $\mathbf{X}_i$ are 3D points in the previous frame and $\mathbf{x}_i$ their 2D image projections in the current frame, PnP finds a transformation that minimizes the reprojection error under the pinhole camera model:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \sim \mathbf{K}\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{3.16}$$

Here, $\mathbf{K}$ is the camera intrinsic matrix, and $\mathbf{R}, \mathbf{t}$ represent the relative pose of the current frame with respect to the previous one. The symbol $\sim$ denotes equality up to scale in homogeneous coordinates.

In practice, 3D-2D correspondences may include mismatches or noisy observations. To robustly estimate the pose in the presence of such outliers, RANSAC (Random Sample Consensus) is used in conjunction with PnP.

RANSAC operates by randomly selecting minimal subsets of correspondences, solving the PnP problem for each subset, and evaluating how many correspondences agree with the estimated pose. The solution with the highest number of inliers is selected, and the pose is then refined using all inliers to minimize the overall reprojection error.

In this work, the function `cv::solvePnPRansac` from OpenCV is used to perform motion estimation. This function takes as input the 3D points from the previous frame, their 2D projections in the current frame, and the camera intrinsics, and returns the estimated rotation and translation vectors.

The steps involved are:

1. Project 3D points from the previous frame into the current image using candidate pose.
2. Measure the reprojection error against actual 2D keypoints.
3. Select inliers whose error is below a given threshold.
4. Repeat for several random subsets and select the best solution.
5. Refine the pose using all inliers.

The resulting output consists of:

- `rvec` a rotation vector in axis-angle form.
- `tvec` a 3D translation vector.

The `rvec` returned by `solvePnPRansac` is a Rodrigues rotation vector or axis-angle rotation vector, a compact 3×1 representation of rotation. The direction of the vector indicates the axis of rotation, while its magnitude represents the rotation angle in radians around that axis. This vector can be converted to a rotation matrix using the Rodrigues formula.

This matrix describes the transformation from the previous camera frame to the current one, and can be used for pose chaining, trajectory estimation, or visual odometry back-end integration.

Covariance estimation

After estimating the camera pose using PnP and RANSAC, it is often useful to quantify the uncertainty associated with this estimate. This is particularly relevant when integrating the pose into filtering frameworks, such as an Extended Kalman Filter (EKF), where a covariance is required.

In this work, the pose covariance is approximated based on the reprojection residuals of the inlier correspondences identified by RANSAC. Once the best pose has been selected, all 3D points are projected onto the image using the estimated transform, and the residual reprojection errors are computed as:

$$\mathbf{e}_i = \mathbf{x}_i^{\text{observed}} - \mathbf{x}_i^{\text{projected}}(R, t) \tag{3.17}$$

The residuals are stacked and used to compute a sample covariance matrix:

$$\Sigma = \frac{1}{N-1} \sum_{i=1}^{N} (\mathbf{e}_i - \bar{\mathbf{e}})(\mathbf{e}_i - \bar{\mathbf{e}})^T \tag{3.18}$$

Although this approach does not yield an exact pose covariance, it provides a reasonable and fast approximation that reflects the quality of the inlier set and the geometric configuration of the matched points.

Motion filtering and smoothing

The pose estimates obtained via PnP are subject to noise due to imperfect keypoint matching, depth errors, or small instabilities in the feature geometry. To improve the consistency of the estimated trajectory, a filtering and smoothing step is applied. Two approaches are combined:

- **Statistical filtering** To detect and remove outliers in both translation and rotation components based on deviation from a local history.
- **Exponential smoothing** To smooth the motion estimate over time using a weighted moving average.

Every component of the translation and rotation vector is separately. A sliding window is stored for each component to compute its mean and standard deviation. New values that exceed a fixed threshold or deviate significantly from the mean are replaced with the mean.

Once filtered, the values are smoothed using exponential smoothing:

$$s_t = \alpha x_t + (1-\alpha)s_{t-1} \tag{3.19}$$

where s_t is the smoothed value at time t, x_t is the current value, and α is a smoothing factor between 0 and 1.

Rotation vectors are smoothed component-wise in Rodrigues form. Although this is a simplification compared to quaternion-based smoothing on $SO(3)$, it performs well in practice for small motions.

After filtering and smoothing, the pose is converted to a 4x4 transformation matrix and stored as the final motion estimate for the current frame.

3.4.3 ROS2 Implementation

The previous algorithm was implemented in c++ class called `VisualOdometry`. This class contains all the methods and attributes required to perform the visual odometry algorithm. It is used in a package called `visual_odometry` that performs the ego motion estimation.

The ROS2 node subscribes to the depth image topic, the camera info topic that contains the camera intrinsic parameters, and the color image topic. It uses the VisualOdometry class to compute the rvec and tvec transformation as well as the covariance matrix and the debug image (figure 3.15).

Once the computation is finished, the node publishes the results on the following topics:

- `/perception_pipeline/odom` Odometry topic with the camera pose in the camera initial frame.
- `/perception_pipeline/camera_frame_tf` Camera frame transform topic with the camera pose in the previous frame relative to the current frame. It is published as a separate topic because in the Kalman filter steo, this information is required with an exact frame syncronization.
- `/perception_pipeline/path` Path topic with the camera pose in the world frame (figure 3.17). This topic is used to visualize the camera trajectory.
- `/debug/odometry_keypoints` Debug topic with the detected keypoint. In red the keypoints with invalid depth and in green the keypoints with valid depth.
- `/tf` The camera frame transform is published as a ROS2 transform. This allows to visualize the camera pose in the world frame using tools like RVIZ or Foxglove.

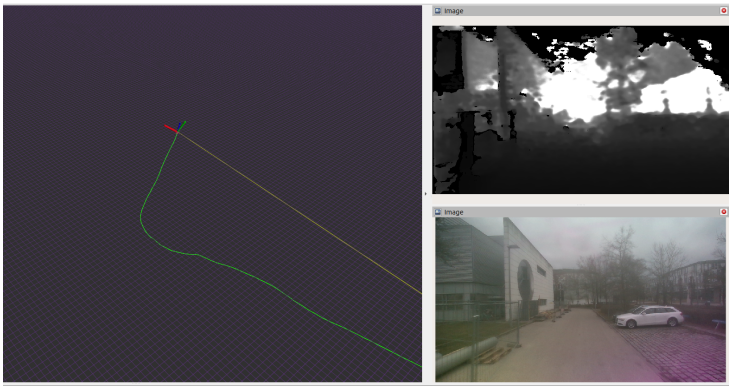


Figure 3.17: Path and depth camera topics. Source: own work

To configure the node, the following ROS2 parameters were added:

Table 3.2: ROS 2 parameters used in visual_odometry package

ROS2 Parameter	**Definition**
color_image_topic	Topic name for the input color image.
depth_image_topic	Topic name for the input depth image.
camera_info_topic	Topic providing intrinsic camera parameters.
odometry_debug_topic	Topic where debug keypoints for odometry are published.
camera_frame_tf_topic	Topic containing camera pose as a transform (TF).
max_depth_odom	Maximum valid depth (in meters) used for odometry.
min_depth_odom	Minimum valid depth (in meters) used for odometry.
odometry_debug_image	Enables publishing of debug images with tracked keypoints.
publish_odom	Enables publishing of odometry estimates.
odom_topic	Topic where odometry data is published.
publish_path	Enables publishing of the full trajectory path.
path_topic	Topic where the estimated path is published.
use_sim_time	Uses simulation time instead of system time (for Gazebo, etc.).

3.5 Perception pipeline: Kalman filter

After introducing the modules responsible for the depth computation (**??**), optical flow computation (**??**), and ego-motion estimation (**??**), this section addresses the core component of the perception pipeline: the estimation of a dense six-dimensional field using a Kalman filter.

The objective of this stage is to compute, for a subset of selected pixels of the image, both their 3D position and 3D velocity over time. This results in a per-pixel six-dimensional vector that captures the spatial and dynamic state of the scene.

This idea aligns with the concept of 6D vision (see Section 2.2), which was initially explored at the FTM Chair by Bauer [20] and later validated in a ROS1 implementation by Pastor [22].

Each point is represented by a `WorldPoint`, which maintains its own Kalman filter and a 6D state vector composed of 3D position and velocity in world coordinates. At each timestep, the point's state is updated using three main sources of input:

- Optical flow, providing pixel-level motion information between frames.
- Depth data, used to reconstruct the 3D location of the point via reprojection.
- Ego-motion, which accounts for the movement of the camera relative to the environment.

The Kalman filter follows a standard constant-velocity model in 3D space. Predicted states are projected back into the image frame for association with new measurements, producing real-time filtered estimates of position and velocity at pixel level resolution.

This approach differs from object-based or grid-based tracking by focusing on local, independent point estimation. Points are initialized or removed based on visibility, depth consistency, and motion reliability. A regular grid helps initialize new points, and the use of OpenMP [64] allows for paralellizing the updates of all active filters tracked points, leading to a significant speedup in the computation.

The system architecture separates the Kalman logic into a core module that operates independently of ROS, and a ROS2 node that manages communication, timing, and visualization. The output includes a 6D image representation, debug data, and visualization markers for use in tools such as RViz and Foxglove.

3.5.1 Kalman filter algorithm

In this module, each selected pixel or `WorldPoint` is associated with its own Kalman Filter instance that estimates their local state. The filters operate independently and are updated at each timestep using new input from the previous perception modules. This approach allows the system to maintain a dense spatial resolution while exploit paralell processing capabilities.

The Kalman Filter, first introduced by R.E. Kalman in 1960 [65], is a recursive estimation algorithm, designed for linear, discrete-time dynamical systems with Gaussian noise. It estimates the internal state of a system by predicting its evolution over time and correcting the prediction using new measurements. Taking this into account, the first thing that is necessary to implement a Kalman filter is to have the called **system model** and the **measurement model**.

The system model is a state space representation of the system, which describes how the state evolves over time. It is defined by the following equations:

$$\mathbf{x}_k = \mathbf{A} \cdot \mathbf{x}_{k-1} + \mathbf{B} \cdot \mathbf{u}_k + \boldsymbol{\omega}_k \tag{3.20}$$

Here, $\mathbf{x}_k$ is the state vector at time k, $\mathbf{A}$ is the state transition matrix, $\mathbf{u}_k$ is the control input, $\mathbf{B}$ is the control input matrix, and $\boldsymbol{\omega}_k$ is the process noise, assumed to be zero-mean Gaussian with covariance $\mathbf{Q}$:

$$\boldsymbol{\omega}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \tag{3.21}$$

The measurement model describes how the measurements are related to the state. It is defined by the following equations:

$$\mathbf{z}_k = \mathbf{H} \cdot \mathbf{x}_k + \boldsymbol{\nu}_k \tag{3.22}$$

where $\mathbf{z}_k$ is the measurement vector, $\mathbf{H}$ is the measurement matrix, and $\boldsymbol{\nu}_k$ is the measurement noise, assumed to be zero-mean Gaussian with covariance $\mathbf{R}$:

$$\boldsymbol{\nu}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \tag{3.23}$$

When both models are linear, as in the equations above, the standard Kalman filter can be used. However, if either model is nonlinear, the Extended Kalman Filter (EKF) [66] must be applied. The EKF approximates nonlinear functions by linearizing them around the current state estimate using the Jacobian (first order Taylor series approximation).

Standard Kalman filter

The standard Kalman filter operates in two main phases at each time step: prediction and correction. Each phase consists of a sequence of matrix operations that update both the estimated state and its associated covariance based on the system and measurement models.

Prediction step The prediction step starts by propagating the previous filtered state $\mathbf{x}_{k-1}$ forward in time using the system model. This results in the predicted state:

$$\hat{\mathbf{x}}_k = \mathbf{A} \cdot \mathbf{x}_{k-1} + \mathbf{B} \cdot \mathbf{u}_k \tag{3.24}$$

where $\mathbf{A}$ is the state transition matrix, $\mathbf{B}$ the control input matrix, and $\mathbf{u}_k$ the control vector at time k. The predicted covariance $\hat{\mathbf{P}}_k$ is then computed as:

$$\hat{\mathbf{P}}_k = \mathbf{A} \cdot \mathbf{P}_{k-1} \cdot \mathbf{A}^\top + \mathbf{Q} \tag{3.25}$$

where $\mathbf{P}_{k-1}$ is the previous covariance and $\mathbf{Q}$ is the process noise covariance.

Correction step Once the state and covariance have been predicted, the correction step begins and the expected measurement is computed using the measurement model:

$$\hat{\mathbf{z}}_k = \mathbf{H} \cdot \hat{\mathbf{x}}_k \tag{3.26}$$

The predicted measurement covariance is then:

$$\hat{\mathbf{T}}_k = \mathbf{H}_k \cdot \hat{\mathbf{P}}_k \cdot \mathbf{H}_k^\top \tag{3.27}$$

When a new measurement $\mathbf{z}_k$ is available, it is compared with the predicted measurement $\hat{\mathbf{z}}_k = \mathbf{h}(\hat{\mathbf{X}}_{w|k})$ to evaluate how consistent the prediction is with the actual observation. The difference between them is known as the innovation vector:

$$\mathbf{s}_k = \mathbf{z}_k - \hat{\mathbf{z}}_k \tag{3.28}$$

This vector represents the residual between what the filter expected to observe and what was actually measured. If the model and previous state are accurate, the innovation should be small and centered around zero. Large values in $\mathbf{s}_k$ may indicate unexpected changes in the system or incorrect predictions.

To properly weight the innovation in the update step, its uncertainty must be quantified. This is given by the innovation covariance matrix:

$$\mathbf{S}_k = \hat{\mathbf{T}}_k + \mathbf{T}_k \tag{3.29}$$

Here, $\hat{\mathbf{T}}_k$ is the predicted measurement covariance derived from the current state estimate, and $\mathbf{T}_k$ is the measurement noise covariance, defined by the sensor model. The matrix $\mathbf{S}_k$ thus quantifies the total expected uncertainty in the innovation vector, combining both predicted state uncertainty projected into measurement space and known sensor noise.

The innovation vector and its covariance are crucial for the correction step of the filter: they determine how strongly the measurement should influence the state update. A large $\mathbf{S}_k$ implies high uncertainty, which results in a smaller correction; conversely, a small $\mathbf{S}_k$ leads to a more aggressive update.

Once the innovation and its covariance are computed, the Kalman gain $\mathbf{K}_k$ is calculated. The Kalman gain is a matrix that determines how much the predicted state should be adjusted based on the new measurement. It is computed as:

$$\mathbf{K}_k = \hat{\mathbf{P}}_k \cdot \mathbf{H}^\top \cdot \mathbf{S}_k^{-1} \tag{3.30}$$

Finally, the state and covariance are updated using the Kalman gain and the innovation vector. The updated state is given by:

$$\mathbf{x}_k = \hat{\mathbf{x}}_k + \mathbf{K}_k \cdot \mathbf{s}_k \tag{3.31}$$

and the updated covariance becomes:

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \cdot \mathbf{H}) \cdot \hat{\mathbf{P}}_k \tag{3.32}$$

This completes one full iteration of the standard Kalman filter. The filtered state $\mathbf{x}_k$ and covariance $\mathbf{P}_k$ serve as the input for the next prediction step.

Extended Kalman filter

When the system or measurement model is nonlinear, the standard Kalman filter cannot be applied directly. In such cases, the Extended Kalman Filter (EKF) is used. The EKF extends the classical Kalman filter by linearizing the nonlinear functions around the current estimate using a first-order Taylor expansion.

Let $\mathbf{f}(\cdot)$ represent the nonlinear system model, and $\mathbf{h}(\cdot)$ the nonlinear measurement model. The EKF performs the prediction step by applying the nonlinear dynamics to the previous state:

$$\hat{\mathbf{x}}_k = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k) \tag{3.33}$$

The predicted covariance is obtained using the Jacobian $\mathbf{F}_k$ of $\mathbf{f}(\cdot)$ with respect to the state:

$$\hat{\mathbf{P}}_k = \mathbf{F}_k \cdot \mathbf{P}_{k-1} \cdot \mathbf{F}_k^\top + \mathbf{Q} \tag{3.34}$$

The predicted measurement is computed by evaluating the nonlinear measurement function:

$$\hat{\mathbf{z}}_k = \mathbf{h}(\hat{\mathbf{x}}_k) \tag{3.35}$$

and its Jacobian with respect to the state is:

$$\mathbf{H}_k = \left. \frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_k} \tag{3.36}$$

The predicted measurement covariance becomes:

$$\hat{\mathbf{T}}_k = \mathbf{H}_k \cdot \hat{\mathbf{P}}_k \cdot \mathbf{H}_k^\top \tag{3.37}$$

Given a new measurement $\mathbf{z}_k$, the innovation is:

$$\mathbf{s}_k = \mathbf{z}_k - \hat{\mathbf{z}}_k \tag{3.38}$$

and the innovation covariance is:

$$\mathbf{S}_k = \hat{\mathbf{T}}_k + \mathbf{R} \tag{3.39}$$

Once the innovation and its covariance are computed, the Kalman gain is calculated as:

$$\mathbf{K}_k = \hat{\mathbf{P}}_k \cdot \mathbf{H}_k^\top \cdot \mathbf{S}_k^{-1} \tag{3.40}$$

Finally, the updated state and covariance are:

$$\mathbf{x}_k = \hat{\mathbf{x}}_k + \mathbf{K}_k \cdot \mathbf{s}_k \tag{3.41}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \cdot \mathbf{H}_k) \cdot \hat{\mathbf{P}}_k \tag{3.42}$$

This completes one full iteration of the Extended Kalman Filter. It retains the recursive structure of the classical filter while enabling the integration of nonlinear system or measurement models by local linearization.

3.5.2 Mathematical model

Now that the Kalman filter algorithm has been introduced, we can define the mathematical model used in this work. First, the system model is defined and then the measurement model will be defined.

System model

For the system model, the state vector $\mathbf{x}_{k|w}$ is defined as a 6×1 vector that contains the position and velocity of the tracked point in world coordinates (w):

$$\mathbf{x}_{k|w} = \begin{bmatrix} X \\ Y \\ Z \\ \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} \tag{3.43}$$

This vector is propagated over time using a constant-velocity motion model, assuming no acceleration between frames. When no ego-motion compensation is applied, the evolution of the state is defined by the following linear state model system:

$$\hat{\mathbf{x}}_{k|w} = \mathbf{A}_{k|w} \cdot \mathbf{x}_{k-1|w} + \boldsymbol{\omega}_{k|w} \tag{3.44}$$

where $\hat{\mathbf{x}}_{k|w}$ is the predicted state at time k, $\mathbf{A}_{k|w}$ is the transition matrix that models constant velocity over the time step Δt, and $\boldsymbol{\omega}_{k|w}$ is the process noise.

The state transition matrix $\mathbf{A}_{k|w}$ of the system is defined as:

$$\mathbf{A}_{k|w} = \begin{bmatrix} \mathbf{I}_3 & \Delta t \cdot \mathbf{I}_3 \\ \mathbf{0}_3 & \mathbf{I}_3 \end{bmatrix} \tag{3.45}$$

where Δt is the time elapsed between frames, representing the linear motion model. The process noise is assumed to be zero-mean Gaussian:

$$\boldsymbol{\omega}_{k|w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_{k|w}) \tag{3.46}$$

Where the process noise covariance matrix $\mathbf{Q}_{k|w}$ captures the uncertainty introduced during the state prediction. It is derived from the continuous-time system noise model [67] assuming white noise acceleration, and discretized using standard integration over the time interval Δt. The resulting structure is:

$$\mathbf{Q}_{k|w} = \begin{pmatrix} \frac{1}{3}\Delta t^2 \cdot \text{diag}(\sigma_{\dot{X}}^2, \sigma_{\dot{Y}}^2, \sigma_{\dot{Z}}^2) & \frac{1}{2}\Delta t \cdot \text{diag}(\sigma_{\dot{X}}^2, \sigma_{\dot{Y}}^2, \sigma_{\dot{Z}}^2) \\ \frac{1}{2}\Delta t \cdot \text{diag}(\sigma_{\dot{X}}^2, \sigma_{\dot{Y}}^2, \sigma_{\dot{Z}}^2) & \text{diag}(\sigma_{\dot{X}}^2, \sigma_{\dot{Y}}^2, \sigma_{\dot{Z}}^2) \end{pmatrix} \tag{3.47}$$

This formulation assumes that velocity is affected by independent white noise processes in X, Y, and Z, and that position integrates this uncertainty over time. As a result, the top-left block of $\mathbf{Q}_{k|w}$ corresponds to accumulated uncertainty in position, and the bottom-right to direct uncertainty in velocity.

To introduce the ego-motion compensation, we have to modify the system model. For this we assume, that the WorldPoints are transformed with the rotation matrix $\mathbf{R}_k$ and the translation vector $\mathbf{t}_k$, that model the movement of the camera between two frames (3.4.2). The position and velocity components of the state vector x_k are transformed as follows:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix}_k = \mathbf{R}_k \cdot \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}_{k-1|w} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}_k \tag{3.48}$$

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix}_k = \mathbf{R}_k \cdot \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix}_{k-1|w} \tag{3.49}$$

which leads to the following updated system state model:

$$\mathbf{x}_k = \mathbf{A}_k \cdot \mathbf{x}_{k-1} + \mathbf{u}_k + \boldsymbol{\omega}_k \tag{3.50}$$

Where the individual components are defined as:

$$\mathbf{A}_k = \mathbf{D}_k \cdot \mathbf{A}_{k|w}, \in \mathbb{R}^{6\times 6} \tag{3.51}$$

$$\mathbf{D}_k = \begin{bmatrix} \mathbf{R}_k & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{R}_k \end{bmatrix} \in \mathbb{R}^{6\times 6} \tag{3.52}$$

$$\mathbf{u}_k = \begin{bmatrix} \mathbf{t}_k \\ \mathbf{0}_3 \end{bmatrix} \in \mathbb{R}^{6\times 1} \tag{3.53}$$

$$\boldsymbol{\omega}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k) \tag{3.54}$$

Now that the ego-motion was introduced in the system model, the covariace matrix $\mathbf{Q}_k$ must be defined. This matrix is not only dependant on the process noise $\sigma^2_{\dot{X}}$, $\sigma^2_{\dot{Y}}$, and $\sigma^2_{\dot{Z}}$ like the covarianze matrix without ego-motion $\mathbf{Q}_{k|w}$, but also on the ego-motion parameters. For this reason a parameter vector containing the ego-motion must be defined.

In previous works like [67], [20], and [22], the ego-motion parameters were defined by the translation components and the Euler angles:

$$\mathbf{p}_k = \begin{bmatrix} t_{k|x} & t_{k|y} & t_{k|z} & \theta_k & \phi_k & \psi_k \end{bmatrix}^T \tag{3.55}$$

This approach has the big disadvantage that the Euler angles are not continuous and can lead to discontinuities in the covariance matrix. Moreover, its necessary to know beforehand which Euler angle convention was used to build the rotation matrix $\mathbf{R}_k$. This can lead to confusion and errors in the implementation and also limits the modularity of the code.

For this reason, in this work, the ego-motion rotation components are defined using the Rodrigues vector $\mathbf{r}_k$ that is computed from the rotation matrix $\mathbf{R}_k$:

$$\mathbf{p}_k = \begin{bmatrix} t_{k|x} & t_{k|y} & t_{k|z} & r_{k|x} & r_{k|y} & r_{k|z} \end{bmatrix}^T \tag{3.56}$$

The Euler-rodrigues vector, as introduced in the chapter 3.4.2, is a continuous representation of the rotation matrix as an vector in $\mathbb{R}^3$ where the length of the vector is the angle of rotation and the direction of the vector is the axis of rotation [68, 69] as seen in the figure 3.18. This representation is continuous and also enables the correct working of the algorithm regardless of how the rotation matrix was computed.

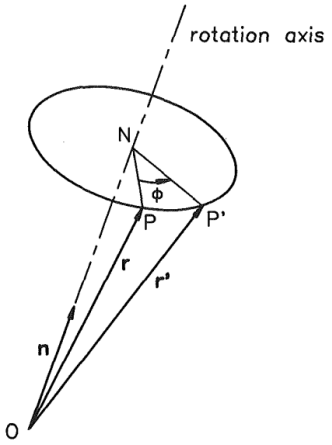


Figure 3.18: Rodrigues vector representation of the rotation matrix. Source: [68]

Now that the p_k vector is defined, we can state that the rotation matrix $\mathbf{R}_k$ and the translation vector $\mathbf{t}_k$ are build as follows:

$$\mathbf{R}_k = \mathbf{R}(\mathbf{p}_k + \zeta_k) \tag{3.57}$$

$$\mathbf{t}_k = \mathbf{t}(\mathbf{p}_k + \zeta_k) \tag{3.58}$$

Where ζ_k is the additive white noise of p_k and follows a Gaussian distribution with zero mean and covariance $\mathbf{L}$:

$$\zeta_k \sim \mathcal{N}(\mathbf{0}, \mathbf{L}), \quad \mathbf{L} = \text{diag}(\sigma^2_{t|x}, \sigma^2_{t|y}, \sigma^2_{t|z}, \sigma^2_{r|x}, \sigma^2_{r|y}, \sigma^2_{r|z}) \in \mathbb{R}^{6\times6} \tag{3.59}$$

Taking this into account, the covariance matrix $\mathbf{Q}_k$ is defined as:

$$\mathbf{Q}_k = \mathbf{D}_k \cdot \mathbf{Q}_{k|w} \cdot \mathbf{D}_k^\top + \mathbf{J}_k \cdot \mathbf{L} \cdot \mathbf{J}_k^\top \tag{3.60}$$

In the previous equation, the term J_k is the Jacobian of the state vector with respect to the ego-motion parameter vector p_k:

$$\mathbf{J}_k = \left.\frac{\partial \mathbf{x}}{\partial \mathbf{p}}\right|_{(\mathbf{x}_{k-1}, \mathbf{p}_k)} \in \mathbb{R}^{6\times6} \tag{3.61}$$

This Jacobian can be extended as follows:

$$\mathbf{J}_k = \left.\frac{\partial \mathbf{x}}{\partial \mathbf{p}}\right|_{(\mathbf{x}_{k-1}, \mathbf{p}_k)} = \left[\left.\frac{\partial \mathbf{x}}{\partial \mathbf{t}_k}\right|_{(\mathbf{x}_{k-1}, \mathbf{p}_k)} \quad \left.\frac{\partial \mathbf{x}}{\partial \mathbf{r}_k}\right|_{(\mathbf{x}_{k-1}, \mathbf{p}_k)} \right] \tag{3.62}$$

The first term, the partial derivative with respect to $\mathbf{t}_k$, corresponds to the direct effect of translation on the predicted state. Since the translation vector only affects the position and not the velocity, the derivative is:

$$\frac{\partial \mathbf{x}_k}{\partial \mathbf{t}_k} = \begin{bmatrix} \mathbf{I}_3 \\ \mathbf{0}_3 \end{bmatrix} \tag{3.63}$$

The second term, the derivative with respect to the rotation vector $\mathbf{r}_k$, is more complex. The rotation matrix $\mathbf{R}_k$ is computed from $\mathbf{r}_k$ using the Euler-Rodrigues formula. In addition to the matrix itself, OpenCV provides the Jacobian $\frac{\partial \mathbf{R}_k}{\partial \mathbf{r}_k} \in \mathbb{R}^{9\times 3}$, which encodes the derivative of each element of $\mathbf{R}_k$ with respect to the rotation parameters.

Let us denote the intermediate propagated state before ego-motion as:

$$\mathbf{p}_k = \mathbf{A}_{k|w} \cdot \mathbf{x}_{k-1|w} = \begin{bmatrix} \mathbf{p}_k^{\text{pos}} \\ \mathbf{p}_k^{\text{vel}} \end{bmatrix} \tag{3.64}$$

where $\mathbf{p}_k^{\text{pos}}$ and $\mathbf{p}_k^{\text{vel}}$ are the position and velocity parts of the propagated state. Then, the rotated position is given by $\mathbf{R}_k \cdot \mathbf{p}_k^{\text{pos}}$, and its derivative with respect to $\mathbf{r}_k$ is:

$$\frac{\partial (\mathbf{R}_k \cdot \mathbf{p}_k^{\text{pos}})}{\partial \mathbf{r}_k} = \sum_{i=1}^{3} \left(\frac{\partial \mathbf{R}_k}{\partial r_i} \cdot \mathbf{p}_k^{\text{pos}} \right) \tag{3.65}$$

The same procedure applies to the velocity component. The full Jacobian is then:

$$\frac{\partial \mathbf{x}_k}{\partial \mathbf{r}_k} = \begin{bmatrix} \frac{\partial (\mathbf{R}_k \cdot \mathbf{p}_k^{\text{pos}})}{\partial \mathbf{r}_k} \\ \frac{\partial (\mathbf{R}_k \cdot \mathbf{p}_k^{\text{vel}})}{\partial \mathbf{r}_k} \end{bmatrix} \in \mathbb{R}^{6\times 3} \tag{3.66}$$

Combining both components, the full Jacobian with respect to the motion parameters is:

$$\mathbf{J}_{k|w} = \begin{bmatrix} \mathbf{I}_3 & \frac{\partial (\mathbf{R}_k \cdot \mathbf{p}_k^{\text{pos}})}{\partial \mathbf{r}_k} \\ \mathbf{0}_3 & \frac{\partial (\mathbf{R}_k \cdot \mathbf{p}_k^{\text{vel}})}{\partial \mathbf{r}_k} \end{bmatrix} \tag{3.67}$$

This expression captures how the uncertainty in the ego-motion parameters propagates into the predicted state covariance, and allows the filter to adapt to the quality of the visual odometry estimates in a principled way. The Jacobian $\mathbf{J}_k$ is re-evaluated at each time step based on the current predicted state and ego-motion estimate.

Now that the full state prediction model—including ego-motion and its uncertainty—has been defined, we turn our attention to the measurement model, which relates the predicted state to the observations available from the stereo camera and optical flow pipeline.

Measurement model

The measurement model relates the predicted state to the observed measurements. In this case, the measurements are obtained from the stereo camera and the optical flow computation. The measurements are defined as a 3×1 vector:

$$\mathbf{z}_k = \begin{bmatrix} u_k \\ v_k \\ d_k \end{bmatrix} \tag{3.68}$$

where u_k and v_k are the pixel coordinates of the tracked point in the image, and d_k is the depth. In this project, the depth was considered for the measurment model, instead of the disparity like the previous works, since most of the modern stereo cameras provide the depth directly as output.

Unlike the system model, the measurement model is nonlinear. The relationship between the predicted state and the measurements is given by the pinhole camera model, which maps the 3D world coordinates to 2D pixel coordinates. The measurement model is defined as:

$$\mathbf{z}_k = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_3 \end{bmatrix} \cdot \frac{1}{w} \cdot \mathbf{\Pi} \cdot \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} \tag{3.69}$$

where $\mathbf{\Pi}$ is the intrinsic projection matrix of the camera, defined as:

$$\mathbf{\Pi} = \begin{bmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{3.70}$$

This projection transforms a 3D point in camera coordinates into homogeneous pixel coordinates. The final measured pixel location is obtained by normalizing the first two components by the third (perspective division), and taking the depth Z directly from the position vector. Therefore, the nonlinear measurement function $\mathbf{h}(\mathbf{x}_k)$ is defined as:

$$\mathbf{h}(\mathbf{x}_k) = \begin{bmatrix} \frac{f_x X}{Z} + c_x \\ \frac{f_y Y}{Z} + c_y \\ Z \end{bmatrix} \tag{3.71}$$

This maps the predicted 3D point to the expected image coordinates (u, v) and depth $d = Z$. Since this model is nonlinear, the Extended Kalman Filter is used. To apply the correction step of the EKF, the Jacobian of the measurement model with respect to the state vector is required.

$$\mathbf{H}_k = \left. \frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_k} \tag{3.72}$$

Only the position part of the state vector affects the measurement, so the Jacobian matrix $\mathbf{H}_k$ has non-zero entries only in the first three columns. The matrix $\mathbf{H}_k \in \mathbb{R}^{3\times 6}$ is computed as:

$$\mathbf{H}_k = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & 0 & 0 & 0 \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \tag{3.73}$$

This Jacobian is evaluated at the predicted state $\hat{\mathbf{x}}_k$ and used in the computation of the innovation covariance.

3.5.3 Implementation

Regarding the implementation of the filter, that will later be used by the corresponding `KalmanFilterNode` ROS2 node, it is separated into two main classes as introduces before:

- `WorldPoint`: This class represents a single tracked point in the scene and manages its own Kalman filter.
- `KalmanCore`: This class manages the list of all active filters and handles their initialization and updates.

In this subsection, we will describe the high level software architecture of both of them.

WorldPoint class

The `WorldPoint` class is the core of the Kalman filter implementation. It encapsulates all the logic for managing a single tracked point in the scene, including its Kalman filter, state prediction, measurement update, and error code handling. In the following figure 3.19 the class diagram of the `WorldPoint` class is shown.

Figure 3.19: WorldPoint class. Source: own work

The initialization takes place through the method `initKalmanFilter()`, which takes a pixel location and a depth value, reprojects it to 3D world coordinates, and initializes the state vector and covariance. A regular grid ensures that only one point is initialized per cell, maintaining a uniform spatial distribution as will be explained in the following section.

The main function for updating each filter is `computeKalmanStep()`. It takes as input the depth image, optical flow, and the current ego-motion estimation. It begins by updating the measurement vector using optical flow, then retrieves the new depth from the depth image. If the new pixel lies outside the image boundaries or the depth is invalid, the function returns an error. Otherwise, it proceeds to the prediction step.

Algorithm 2 Kalman Filter Step (WorldPoint)

```
Get optical flow at previous pixel position
Compute new pixel and read new depth
if new pixel is outside bounds or depth is invalid then
    return Error
end if
Predict new state using motion model
if using ego-motion then
    Apply rotation and translation
    Compute updated uncertainty using Jacobians
end if
Project predicted state into image
Compute innovation vector and covariance
if 3-sigma test fails then
    return Error
end if
Compute Kalman gain
Update state and covariance
return Success
```

If ego-motion compensation is enabled, the predicted state is rotated and translated based on the estimated motion of the camera. Additionally, the associated uncertainty is propagated using Jacobians of the rotation matrix with respect to the Rodrigues vector. These Jacobians are computed using OpenCV, and allow precise propagation of uncertainty due to camera motion.

After predicting the state, the measurement is predicted by projecting the 3D point into the image using the pinhole camera model. This expected pixel and depth are then compared to the actual measurement, and an innovation vector is calculated.

Before applying the correction step, a 3-sigma test is used to validate the measurement. This test ensures that the observed measurement is statistically consistent with the predicted measurement given the uncertainty. Specifically, it checks whether the Mahalanobis distance of the innovation vector $\mathbf{s}_k$ is below a threshold:

$$\mathbf{s}_k^\top \cdot \mathbf{S}_k^{-1} \cdot \mathbf{s}_k < \tau \tag{3.74}$$

where $\mathbf{S}_k$ is the innovation covariance and τ is the threshold corresponding to a 99.7% confidence interval in a chi-squared distribution with 3 degrees of freedom. If the condition is not met, the measurement is considered an outlier and the correction step is skipped to maintain filter robustness.

If the update is accepted, the Kalman gain is computed, and the state and covariance are updated accordingly. The filter also checks whether the updated 3D point lies within acceptable height bounds. If the point is too high or too low, it is discarded.

KalmanCore class

The `KalmanCore` class is responsible for managing the full set of active `WorldPoint` filters in the scene. It handles the initialization, update, removal, and memory management of each tracked point, ensuring a scalable and spatially uniform filtering process across the image.

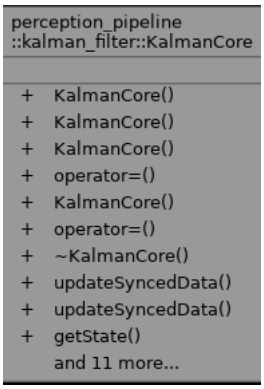


Figure 3.20: KalmanCore class.

To guarantee an even distribution of tracked points, a grid-based approach is employed (figure 3.21). The image plane is divided into a fixed number of rectangular cells, controlled by the parameters `grid_cols` and `grid_rows`. Each cell holds at most one active `WorldPoint`. This ensures that high-texture regions do not accumulate redundant points, and low-texture areas are still covered, improving robustness across the entire field of view.

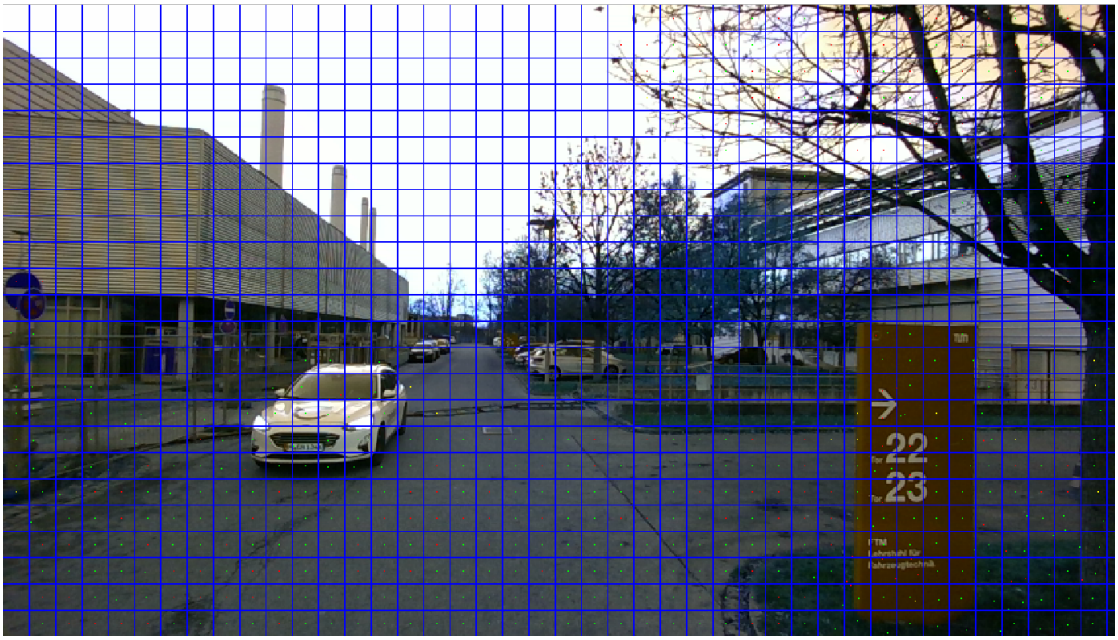


Figure 3.21: KalmanCore grid.

The core logic of each frame is executed in the `updateSyncedData()` method. This method is responsible for updating all active filters, removing invalid ones, and reinitializing new points where necessary. The complete process is shown in Algorithm 3.

Algorithm 3 Frame Processing (`KalmanCore::updateSyncedData`)

Input: depth image, optical flow, ego-motion estimate
Output: updated set of WorldPoints
for all grid cells (i, j) **do**
    if cell has an active WorldPoint **then**
        run `computeKalmanStep()` for the point
        if step fails or point outside bounds **then**
            remove WorldPoint from cell
        end if
    end if
end for
for all empty cells (i, j) **do**
    search candidate pixels in image region
    if valid depth and point found **then**
        initialize new WorldPoint and store in grid
    end if
end for

bb

The algorithm is divided into two stages. In the first stage, each existing `WorldPoint` is updated. If the prediction or correction fails, the point is discarded. This may occur due to missing depth, invalid image location, or high innovation error detected by the three-sigma test.

In the second stage, the algorithm checks for empty grid cells. For each unoccupied cell, a new point is initialized by scanning its corresponding image region for valid pixels with usable depth. The pixel is reprojected to 3D and passed to `initKalmanFilter()` to start a new filter instance.

The grid ensures that at least one point exists per region at any time, preventing overlapping estimates and enabling scalable parallel execution. In addition, the class supports multithreading: the outer loop is parallelized with OpenMP, allowing all `WorldPoints` to be updated simultaneously. This results in a significant performance gain, especially when tracking hundreds of points across large resolutions.

3.5.4 ROS2 integration

The `KalmanFilterNode` is the central ROS 2 component that connects the Kalman filter logic with the perception pipeline. It handles all data flow by subscribing to synchronized data input streams, applying the filter update loop, and publishing processed outputs and debug information.

The node subscribes to three synchronized input topics: the depth image, the optical flow, and the ego-motion transform. Synchronization is performed using `message_filters::Synchronizer` to guarantee time alignment across all data modalities. Accurate synchronization is crucial to prevent inconsistencies in the filter prediction and correction steps.

In addition, runtime behavior is fully configurable via ROS 2 parameters. These include the grid size, filter parameters such height limits to consider, and whether ego-motion should be used. It also allows configuring all relevant noise sources via scalar parameters, including the standard deviations of optical flow, depth,

process model, and ego-motion (rotational and translational) noise. These values are internally used to construct the covariance matrices required by the Kalman update equations.

The output of the node is a 2D image encoding the full 6D state field over the image plane. This image uses the `CV_32FC(7)` encoding, where each pixel contains a seven-channel float vector.

- Channels 0–2: estimated position (X, Y, Z)
- Channels 3–5: estimated velocity $(\dot{X}, \dot{Y}, \dot{Z})$
- Channel 6: binary validity flag (1.0 if valid, 0.0 if empty)

For qualitative evaluation and debugging, additional topics are published. These include:

- A RGB image with the WorldPoint positions overlaid on the original image
- A `MarkerArray` with 3D arrows, representing the estimated velocity field

The output of the Kalman Filter is shown in the figure 3.22.

Figure 3.22: Kalman Filter output. Source: own work

3.6 Perception pipeline: Object detection

The final stage of the perception pipeline is the object detection component. It is responsible for transforming the dense 6D motion field, produced by the Kalman filter, into a compact set of object-level representations. Each detected object is described by its spatial extent and estimated velocity, enabling further reasoning in downstream modules such as criticality assessment.

The object detection component performs two main tasks. First, it filters and selects reliable motion vectors from the raw state field. Then, it applies a clustering algorithm to group consistent points into distinct objects. This approach allows the system to transition from point-based to object-centric perception, which is better suited for interpreting dynamic scenes.

Like the Kalman filter module, the object detection logic is implemented in a ROS2-independent source library, and a ROS2 node handles communication and integration with the rest of the system.

3.6.1 Clustering

The goal of the clustering stage is to aggregate motion vectors into object-level groups. Each vector, defined by a 3D position and velocity, represents a localized motion hypothesis estimated by the Kalman filter. Clustering serves to identify sets of points that likely belong to the same moving object in the environment.

This task is challenging due to the sparsity and variability of the input data. Objects may vary in size, shape, and velocity; some points may be isolated or unreliable due to sensor noise or tracking loss. As a result, the clustering method must be robust, flexible, and capable of handling outliers.

Several clustering techniques have been proposed in literature for similar tasks. Among the most common are:

- **K-Means [70]**: a partition-based algorithm that assigns points to a fixed number of clusters. It assumes convex cluster shapes and equal density, which is often not the case in real-world perception problems.
- **Hierarchical clustering [71]**: a tree-based method that builds a cluster hierarchy by recursively merging or splitting groups. It does not require prior knowledge of the number of clusters, but has high computational cost and is sensitive to noise.
- **Mean Shift [72]**: a mode-seeking algorithm that clusters based on density peaks. It can discover arbitrarily shaped clusters, but tends to be computationally expensive and sensitive to bandwidth selection.
- **DBSCAN [37]**: a density-based algorithm that identifies clusters as connected regions of high point density. It does not require the number of clusters in advance and handles noise explicitly.

Given the requirements of this project—unstructured input, unknown number of objects, and frequent outliers—DBSCAN was selected. It is well suited to segment point clouds where object boundaries are defined by density rather than geometry. Moreover, DBSCAN can be extended with a custom distance function, enabling the integration of both spatial and motion cues into the clustering process.

In this implementation, the algorithm considers two points as neighbors if they are close in space and exhibit similar velocity vectors. This joint criterion is more discriminative than position alone, especially in dynamic scenes where multiple objects may temporarily overlap in 3D space.

3.6.2 DBSCAN clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a non-parametric clustering algorithm introduced by Ester et al. [37]. It defines clusters as areas of high point density and treats sparse regions as separators or noise. This density-based definition allows DBSCAN to discover clusters of arbitrary shape and to operate without requiring the number of clusters in advance.

The algorithm operates on a dataset of points and requires two parameters:

- ε: the neighborhood radius around each point
- $minPts$: the minimum number of points within the radius to form a dense region

The process begins by iterating over each point in the dataset:

1. For a given point, its ε-neighborhood is computed.
2. If the number of neighbors is greater than or equal to $minPts$, the point is marked as a *core point*.
3. A new cluster is initiated and the neighborhood of the core point is expanded recursively to include all density-connected points.
4. If a point is not a core point and not reachable from any cluster, it is marked as noise.

Two points are considered *density-connected* if there exists a chain of core points such that each is within ε of the next. This enables clusters to grow organically around dense regions and stop at natural boundaries.

Figure 3.23 illustrates the basic idea of DBSCAN. Core points have dense neighborhoods and initiate clusters. Border points lie within the neighborhood of a core point but lack enough neighbors to be core points themselves. Noise points do not belong to any cluster.

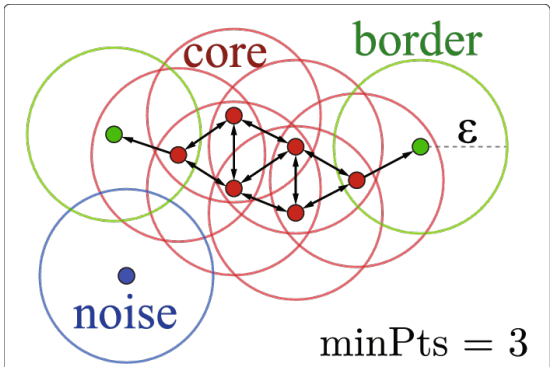


Figure 3.23: Illustration of DBSCAN clustering: core, border, and noise points. Source: [73]

This structure makes DBSCAN robust to spurious detections and suitable for real-world scenarios with irregular object shapes. In dynamic environments, it is common for moving objects to generate clusters of varying density. DBSCAN handles this without the need to fit a global model or impose strict geometric assumptions.

In our application, the algorithm is applied not in pure Euclidean space, but using a distance function that incorporates both spatial and motion similarity. The definition of this distance and its implementation are described in the following sections.

Implementation

The `ObjectDetector` class is the central component responsible for identifying dynamic objects from the 6D state field computed in the Kalman filter stage. It is designed as a self-contained module that can be reused and evaluated independently of the ROS2 framework. The main purpose of this class is to convert a dense field of motion vectors into a sparse list of detected object representations suitable for higher-level reasoning.

The class exposes a public method, `detect()`, which performs the complete processing pipeline. The input to this method is an OpenCV matrix of type `CV_32FC(7)`, where each pixel encodes a 6D motion estimate and a validity flag. The output is a list of `WorldEntity` instances, each corresponding to a cluster of consistent motion vectors.

Internally, the pipeline consists of three sequential stages. These are described below in detail.

1. CUDA-based filtering The first stage is responsible for identifying valid and informative motion vectors from the full image. Given that the input may contain a large number of pixels, including invalid or static ones, it is essential to reduce the dataset before clustering. To achieve this efficiently, a custom CUDA kernel is used. Each thread in the kernel processes one pixel, applying a sequence of checks:

- Validity: The flag channel must indicate a valid tracked point.
- Motion: The magnitude of the velocity vector must exceed a minimum threshold.

- Depth: The 3D position must be finite and within a configured spatial range.

Points that pass all filters are copied into a device buffer. After kernel execution, the filtered data is transferred from GPU to CPU memory. This parallel approach allows processing large images at interactive rates, even when the proportion of valid points is low.

2. DBSCAN clustering The second stage takes the filtered 6D points and performs density-based clustering using DBSCAN. This is implemented using the `mlpack` C++ library, which provides an extensible interface for clustering with custom distance functions. In this case, a metric combining spatial distance and velocity similarity is used. The motivation and details of this distance function are presented in the following subsection.

DBSCAN is configured with parameters ε and $minPts$, which are specified as ROS parameters. These control the local density required to initiate a cluster and the radius used to define neighborhoods. Clusters with fewer than the minimum size are discarded as noise. This helps remove isolated points or spurious detections that may pass the filtering stage but do not correspond to any coherent object.

3. Cluster-to-object conversion For each valid cluster found by DBSCAN, a new `WorldEntity` object is created. This class encapsulates the aggregate properties of the cluster, including its 3D bounding box, average velocity, and center position. The conversion is straightforward: all points in the cluster are passed to the constructor of the `WorldEntity`, which then computes the relevant statistics. This separation of concerns improves modularity and simplifies testing.

The high-level flow of the detection pipeline is outlined in Algorithm 4.

Algorithm 4 Object detection pipeline (`ObjectDetector::detect`)

Input: 6D motion field (OpenCV matrix)
Output: List of `WorldEntity` objects

Launch CUDA kernel to extract valid motion vectors
Transfer filtered points from GPU to CPU
if number of valid points $<$ minimum **then**
    return empty list
end if
Instantiate DBSCAN with custom distance function
Run clustering on the filtered point set
for all output clusters **do**
    if cluster size $<$ minimum allowed **then**
        discard as noise
    else
        construct `WorldEntity` from cluster
    end if
end for
return list of detected entities

A structural overview of the class is presented in Figure 3.24. The figure shows the main attributes, public interface, and key internal methods used during the detection process.

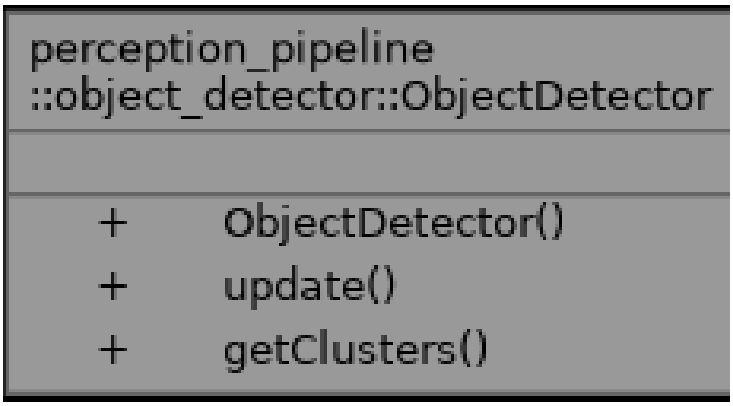


Figure 3.24: UML diagram of the `ObjectDetector` class. Source: own work

This architecture ensures that the computational workload is kept minimal on the CPU and that the more expensive filtering operations benefit from parallel execution on the GPU. The use of `mlpack` further abstracts the clustering logic, while still allowing full control over the distance metric and clustering parameters.

The following section describes the custom distance function in detail.

Custom distance function The clustering stage uses a custom distance metric that combines spatial proximity with velocity coherence, enabling the detection of objects not only based on their location but also on their motion behavior. The implementation is provided in the `WeightedPosVelDistance` struct defined in `point_distance.hpp`. Each input point is a six-dimensional vector that encodes position (x, y, z) and velocity (v_x, v_y, v_z).

The distance between two points $\mathbf{a}$ and $\mathbf{b}$ is computed as:

$$d(\mathbf{a}, \mathbf{b}) = w_p \cdot d_p + w_v \cdot \theta \tag{3.75}$$

where w_p and w_v are the position and velocity weights, d_p is the Euclidean distance between the 3D positions, and θ is the angle between the velocity vectors of the two points.

The Euclidean position distance is computed as:

$$d_p = \|\mathbf{p}_a - \mathbf{p}_b\|_2 = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2 + (z_a - z_b)^2} \tag{3.76}$$

The angle θ between velocity vectors is defined as:

$$\theta = \arccos\left(\frac{\mathbf{v}_a \cdot \mathbf{v}_b}{\|\mathbf{v}_a\| \cdot \|\mathbf{v}_b\|}\right) \tag{3.77}$$

To ensure numerical stability, the angle is clamped to the interval $[0, \pi]$. If either velocity vector has near-zero magnitude, the angle is set to π, indicating maximum dissimilarity. This design penalizes static or noisy points when compared to moving ones, encouraging clusters to form around coherent velocity fields.

This formulation enables the algorithm to separate objects that are close in space but move in different directions. For example, two cars driving past each other at close distance will be assigned to different clusters due to their opposing velocity vectors. Conversely, spatially disjoint points moving in the same direction may still be grouped if their positions lie within the specified range and the velocity angle is small.

The distance weights w_p and w_v are passed to the constructor of the distance functor and exposed as ROS2 parameters, allowing the clustering behavior to be tuned at runtime. Internally, the function avoids unnecessary allocations and uses precomputed dot products and norms to ensure efficiency during large-scale clustering.

This hybrid distance formulation provides a principled way to fuse geometric and motion information and is a key enabler for robust clustering in dynamic environments.

WorldEntity class

Each cluster produced by the DBSCAN algorithm is converted into a `WorldEntity` object, which encapsulates its geometric and kinematic properties in a compact representation. The class is initialized with a list of 3D points and corresponding velocity vectors, from which it computes the centroid, average velocity, and an oriented bounding box. These quantities are stored internally and accessed via public getter methods. The centroid is computed as the arithmetic mean of all 3D positions, and the average velocity is calculated analogously from the velocity vectors. These two features provide a representative location and motion estimate for the object.

The bounding box is computed using a Principal Component Analysis (PCA), which enables an orientation-aware description of the cluster's shape. The procedure consists of the following steps:

1. Center the input data by subtracting the mean position from each point.
2. Form a $3 \times N$ matrix containing all centered 3D points.
3. Compute the covariance matrix of the centered data.
4. Solve the eigenvalue decomposition to obtain three orthogonal basis vectors.
5. Project all points into this PCA-aligned coordinate system.
6. Determine the minimum and maximum bounds along each principal axis.
7. Reconstruct the eight corners of the bounding box in world coordinates by transforming the axis-aligned box in PCA space back to the original frame.

This results in an oriented bounding box that tightly encloses the cluster. The three axes of the box correspond to the directions of maximum variance in the data, and the box dimensions reflect the spatial spread along those axes. If the number of points is too low to perform PCA reliably, a fallback axis-aligned bounding box is computed directly from the minimum and maximum values in each coordinate dimension.

This bounding box serves not only as a geometric representation of the object, but also for the computation of the criticallity metric. Thanks to the bounding boxes, its not necessary anymore to assume the objects to be point-like, but they can be represented as 3D boxes. This allows for a more accurate

In addition to geometric information, the class assigns a unique identifier to each entity and includes utility methods for converting its contents into ROS-compatible messages. These include a method for generating a visualization marker and another for publishing the entity as part of a message array. The class diagram shown in Figure 3.25 summarizes the internal structure and public interface of the implementation.

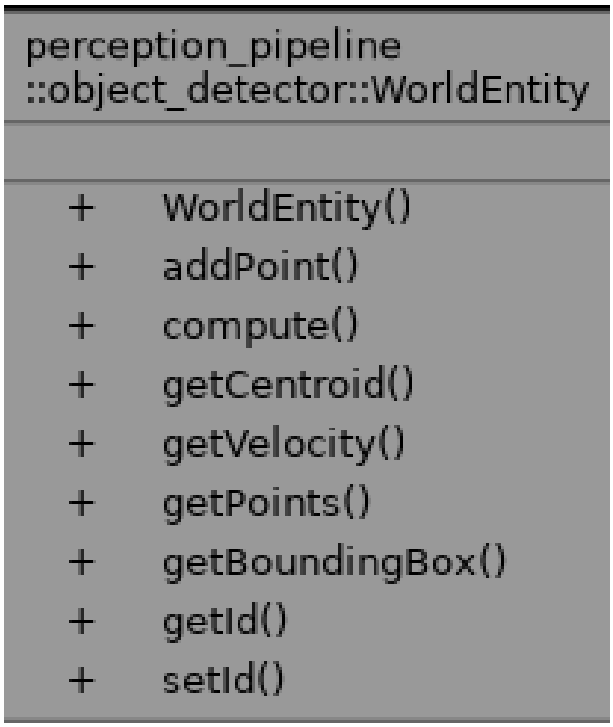


Figure 3.25: UML diagram of the `WorldEntity` class. Source: own work

This class provides a consistent abstraction between the clustering algorithm and the next stages of the perception pipeline

ROS2 integration

The `ObjectDetectorNode` serves as the ROS2 interface to the object detection module. It connects the independent detection logic with the rest of the perception pipeline and is responsible for receiving motion data, triggering clustering, and publishing object-level representations.

The node subscribes to a single topic, containing the 6D motion field output from the Kalman filter stage. This field is published as a `sensor_msgs::msg::Image` message with encoding `CV_32FC(7)`. Each pixel in the image stores a seven-dimensional float vector: three values for position, three for velocity, and one flag indicating if the data pixel contains a tracked point.

Upon receiving a new message, the callback function `callback6dImage()` converts the input to a `cv::Mat`, which is then passed to the detector. The `ObjectDetector` instance processes the image and returns a list of clustered entities. These clusters are then published in two forms: as RViz visualization markers and as structured messages for downstream consumption.

The structured output is published on the `/object_clusters` topic using a custom message type `ClusteredObjectArray`. This message encapsulates all detected objects for a given frame. It contains a header, the number of clusters, and a vector of `ClusteredObject` messages. Each `ClusteredObject` includes:

- `id`: a unique integer identifier for the object
- `position`: a `geometry_msgs::Point` representing the centroid
- `velocity`: a `geometry_msgs::Vector3` with average motion
- `bb_marker`: a `visualization_msgs::Marker` for visualization

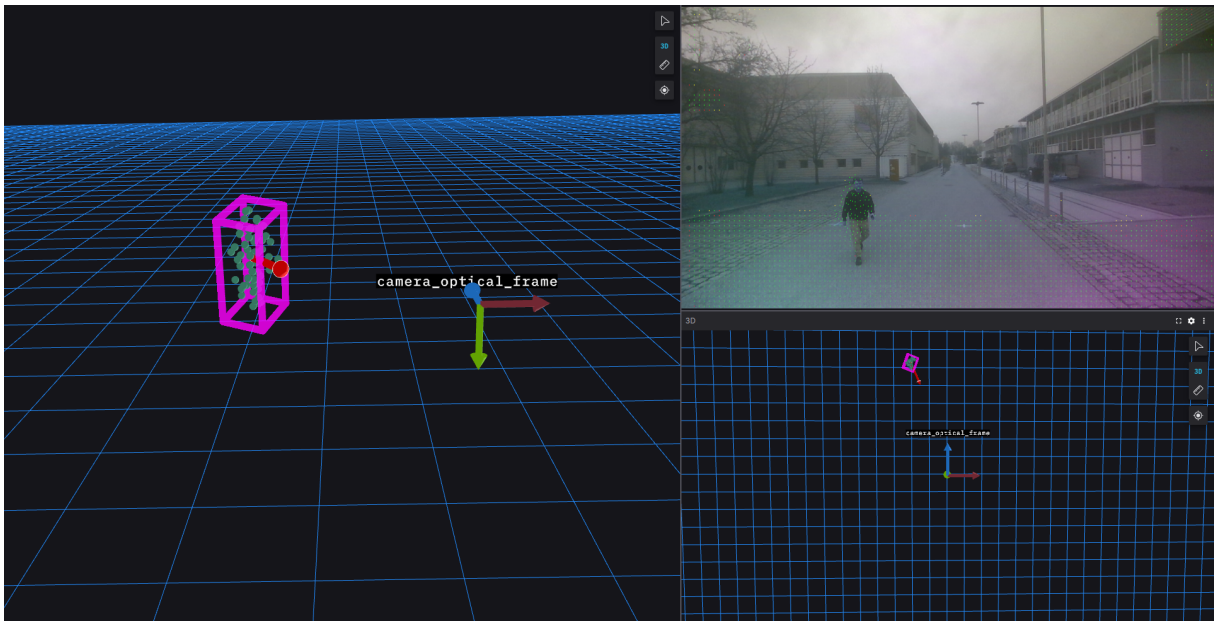


Figure 3.26: Object detection output. Source: own work

- `bounding_box`: a custom message of type `BoundingBox`

The `BoundingBox` message defines the geometry of the oriented bounding box as a list of eight 3D corners, stored as an array of `geometry_msgs::Point`. This explicit representation of the box enables further geometric reasoning in later stages of the pipeline, such as the criticality assessment.

In parallel to structured outputs, the node publishes markers for visualization. These include the bounding box as a wireframe, the motion vector as an arrow, and the clustered points as spheres, all in the `camera_optical_frame`. This visualization output is shown in the following figure 3.26.

All relevant parameters of the node are loaded from the ROS2 parameter server. These include the clustering radius ε, the minimum number of points `minPts`, velocity threshold, and the weights `pos_weight` and `vel_weight` for the custom distance metric. The input and output topics can also be configured via parameters to ensure compatibility with different integration setups. This can be included in a launch file like in the following example:

Code 3.3: Stereo computation launch

```
launch_ros.actions.Node(
        package='object_detector',
        executable='object_detector_node',
        name='object_detector_node',
        output='screen',
        parameters=[
          {'input_6d_topic': '/perception_pipeline/output_6d'},
          {'output_markers_topic' : '/perception_pipeline/
 output_markers'},
          {'output_clusters_topic' : '/perception_pipeline/
 output_clusters'},
          {'eps': 0.3},
          {'minPts': 7},
          {'pos_weight': 1.0},
          {'vel_weight': 0.02},
          {'vel_threshold': 0.2},
          {'use_sim_time': use_sim_time}
```

```
16                 ],
17             )
```

With this structure, the object detection node forms a robust interface layer that bridges raw motion estimation with object-level perception, providing consistent, interpretable outputs for the next stages of the pipeline.

3.7 Criticality pipeline: TTC computation

This section addresses the final stage of the perception pipeline: the computation of Time-to-Collision (TTC) as a criticality indicator. TTC quantifies the temporal proximity between the ego vehicle and external dynamic entities, assuming a constant relative velocity. It is widely used in automated driving due to its simplicity, computational efficiency, and direct interpretability.

Beyond TTC, various criticality metrics have been proposed in the literature. Westhofen et al. [38] provide a comprehensive taxonomy, grouping them into time-based, distance-based, probabilistic, and hybrid categories. Several of these metrics directly extend TTC, enriching its semantics or adapting it to specific contexts. Among the most relevant TTC-based metrics are the following:

- **Time-to-Brake (TTB)**: estimates the time remaining until a braking maneuver is required to avoid collision, based on a constant deceleration model. It refines TTC by considering the ego braking capacity.
- **Time-to-Steer (TTS)**: measures the time available before a steering maneuver becomes necessary to avoid an obstacle. It introduces lateral dynamics and is particularly relevant in evasive scenarios.
- **Time-to-React (TTR)**: incorporates human or system reaction time into the TTC estimation, often used in driver monitoring and shared control contexts.
- **Time-to-Avoidance (TTA)**: models the time available until an avoidance maneuver (brake or steer) becomes ineffective. It compares TTC with the time needed to complete the avoidance maneuver successfully.
- **Acceleration-based metrics (AM)**: estimate the required acceleration to avoid a collision within a given time window, assuming dynamic constraints. These are useful in risk-aware motion planning.

All these metrics maintain a direct conceptual dependency on TTC, which serves as the temporal reference for evaluating risk. Figure 3.28 shows a dependency graph of commonly used criticality metrics, with TTC at its core.

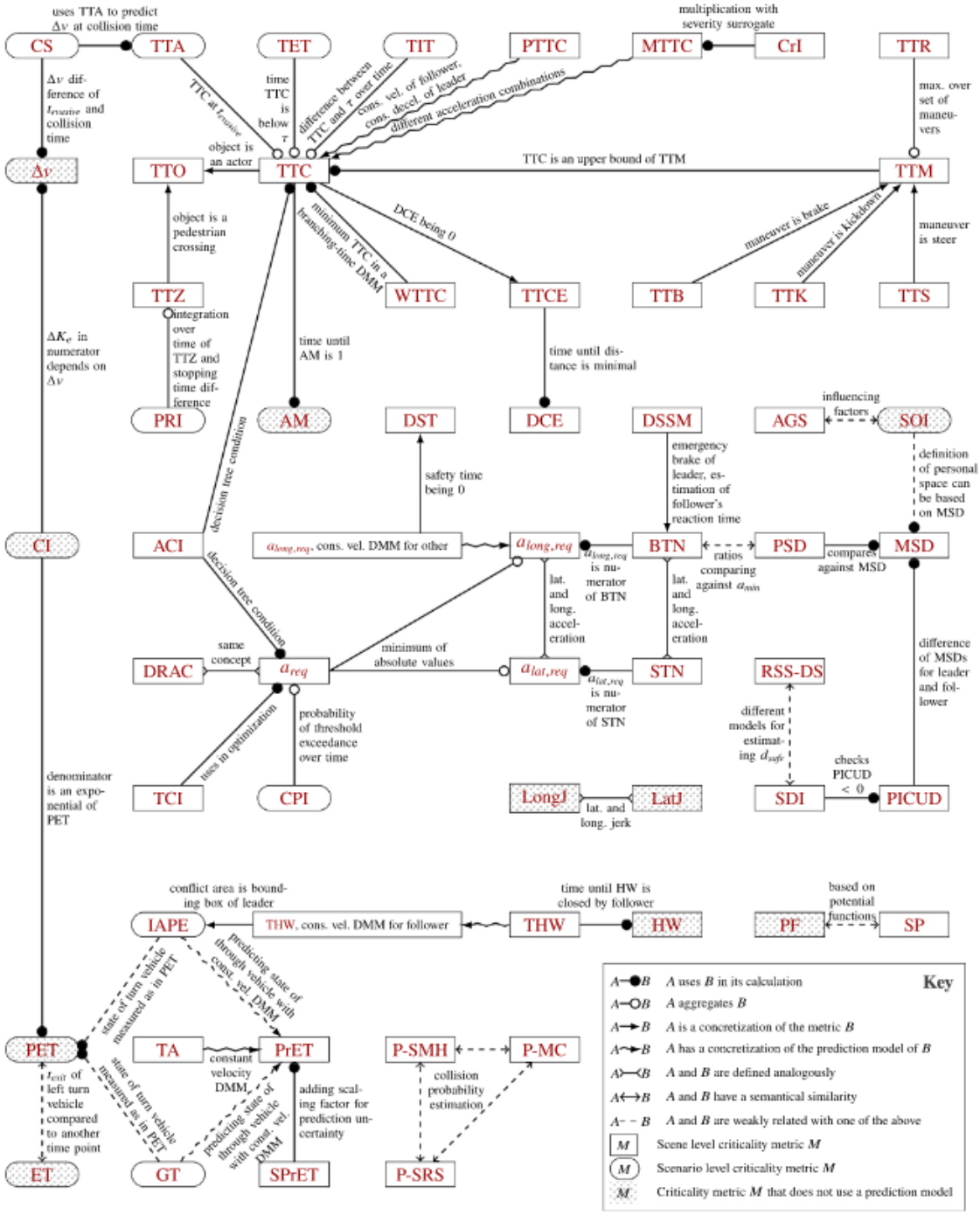


Figure 3.27: Dependency graph of criticality metrics based on TTC. Adapted from [38].

While TTC is often computed under the assumption of point-like agents, this approximation neglects object geometry and orientation. In structured urban environments, this simplification can lead to under- or overestimation of risk, especially in dense traffic or intersection scenarios.

A more accurate strategy considers object dimensions through oriented bounding boxes. Jiménez et al. [74] propose a method that models each vehicle as a rectangular body and evaluates potential collisions by tracking corner-to-edge interactions over time. Their formulation handles various motion configurations and computes the TTC based on geometric overlap rather than idealized point intersections.

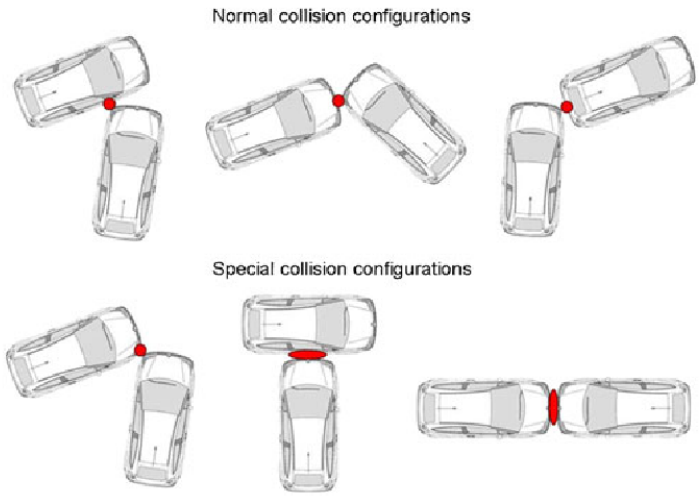


Figure 3.28: Improved TTC computation based on rectangle-based vehicles. Adapted from [74].

Such approaches could be integrated into future iterations of the criticality pipeline, leveraging the bounding box information already available from the object detection stage (Section 3.6.2). This would enable more realistic contact prediction and improve the robustness of the TTC computation under complex spatial arrangements.

3.7.1 Time-to-Collision metric

After introducing the conceptual role of TTC in the broader context of criticality estimation, this subsection presents a more detailed mathematical and algorithmic formulation of the metric.

The objective is to formalize its computation, analyze the assumptions underlying its definition, and discuss how it is adapted and extended in this work to support trajectory-aware reasoning.

The Time-to-Collision (TTC) is a fundamental metric for estimating collision risk in autonomous driving scenarios. It expresses the time remaining until a potential collision occurs, assuming that both agents continue moving with constant velocity. Its main advantage lies in its simplicity, low computational requirements, and direct interpretability.

In its basic formulation, the Time-to-Collision (TTC) represents the time it would take for two objects to collide if they continue moving at constant velocity. It is computed as the current distance between the two agents divided by the difference in their velocities projected onto the line connecting them.

Let $\mathbf{p}_{\text{rel}} \in \mathbb{R}^3$ be the relative position vector from the ego vehicle to the object, and $\mathbf{v}_{\text{rel}} \in \mathbb{R}^3$ the relative velocity between them. The TTC is then defined as:

$$\text{TTC} = \frac{\|\mathbf{p}_{\text{rel}}\|}{-\mathbf{v}_{\text{rel}}^{\top} \cdot \hat{\mathbf{p}}_{\text{rel}}} \tag{3.78}$$

Here, $\hat{\mathbf{p}}_{\text{rel}} = \mathbf{p}_{\text{rel}}/\|\mathbf{p}_{\text{rel}}\|$ is the unit vector pointing from the ego vehicle towards the object. The denominator in Equation (3.78) corresponds to the projection of the relative velocity onto this direction vector, i.e., the component of $\mathbf{v}_{\text{rel}}$ that is aligned with the separation vector. This projection represents how quickly the distance between the two agents is decreasing along the line of sight. If this projection is negative, it indicates

that the object is approaching the ego vehicle. A positive projection implies divergence, in which case TTC is undefined or infinite.

Thus, the TTC can be interpreted as:

$$\text{TTC} = \frac{distance}{rate\ of\ approach} = \frac{\|\mathbf{p}_{\text{rel}}\|}{-\mathbf{v}_{\text{rel}}^{\top} \cdot \hat{\mathbf{p}}_{\text{rel}}} \tag{3.79}$$

The numerator is the Euclidean distance between the object and the ego vehicle, while the denominator is the closing velocity.

This geometric interpretation is illustrated in Figure 3.29, where the TTC is calculated as the ratio between the relative distance and the rate at which that distance shortens along the collision axis.

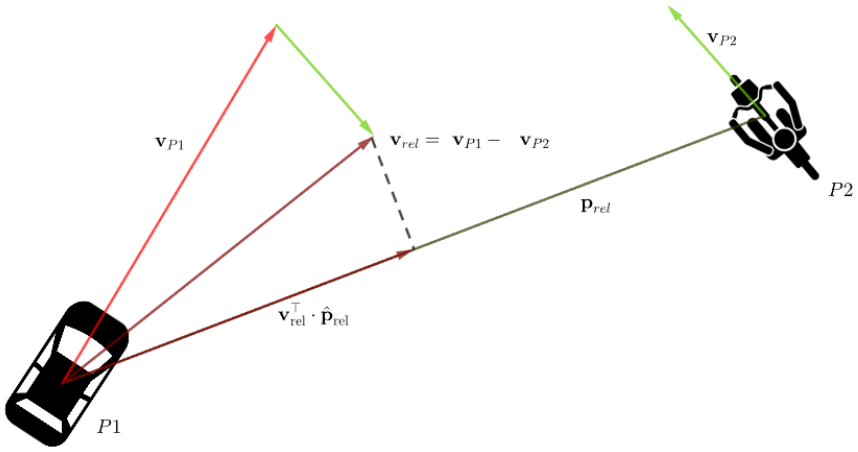


Figure 3.29: Geometric interpretation of time-to-collision.

Equation (3.78) assumes rectilinear motion at constant velocity, with both agents approximated as point masses. It does not account for lateral motion, acceleration, or non-holonomic constraints. Moreover, the formulation implicitly assumes that a collision occurs when the object reaches the ego vehicle's current position, ignoring their respective shapes and orientations.

To overcome some of these limitations, a second approach is implemented based on forward simulation over the ego vehicle's planned trajectory (figure 3.30). Let $\mathcal{P} = \{\mathbf{p}_1, \ldots, \mathbf{p}_N\}$ denote a discrete sequence of future ego poses, sampled at a fixed rate $1/\Delta t$, and let $\mathbf{o}_0$ and $\mathbf{v}_o$ be the current position and velocity of the object.

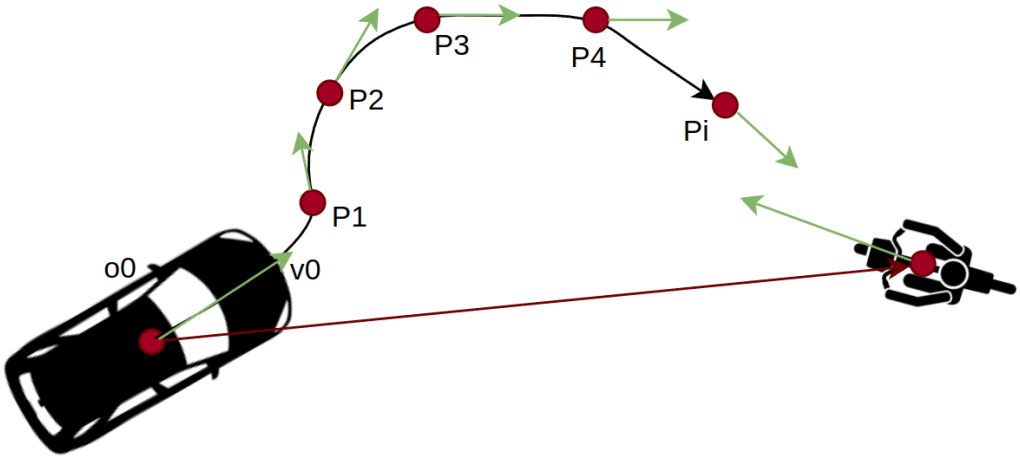


Figure 3.30: Time-to-collision with ego path consideration. Source: own work

Assuming constant object velocity, its future position at time $t_i = i \cdot \Delta t$ is predicted as:

$$\mathbf{o}_i = \mathbf{o}_0 + i \cdot \Delta t \cdot \mathbf{v}_o \tag{3.80}$$

The TTC is then defined as the first t_i such that the distance between the ego vehicle and the object falls below a collision threshold d_{th}:

$$\|\mathbf{p}_i - \mathbf{o}_i\| < d_{\text{th}} \tag{3.81}$$

The threshold d_{th} accounts for the physical dimensions of the ego vehicle and a minimal buffer. A typical value ranges from 0.5 to 1.5 meters, depending on system resolution and safety requirements.

This method leverages existing trajectory information and enables a more realistic estimation of collision likelihood in structured environments. It is particularly relevant when the ego vehicle is navigating a curve or executing complex maneuvers where the twist-based approximation becomes inaccurate.

Despite this improvement, both approaches treat vehicles as dimensionless points. In practice, real objects occupy space and have non-negligible extent. Ignoring object geometry may lead to conservative or optimistic estimates, especially when bounding boxes partially overlap without their centers being aligned.

To address this, an extended TTC formulation could incorporate bounding box geometry into the distance computation. Jiménez et al. [74] propose a method where each vehicle is modeled as a rectangle, and TTC is calculated based on the earliest contact between corners and edges. Their approach enumerates the possible collision configurations and accounts for both relative motion and orientation.

In the current pipeline, each object is already associated with an oriented bounding box derived from clustering (see Section 3.6.2). This enables a future extension where the collision threshold d_{th} is replaced by geometric contact prediction between the ego vehicle's bounding box and that of the object. This enhancement would lead to more accurate criticality assessments, particularly in dense urban scenarios or during turning maneuvers.

3.7.2 Implementation: TTC detection module

The TTC estimation described above is implemented as a dedicated ROS2 node named `ttc_calculator_node`. This component serves as the bridge between the object-level output of the perception pipeline and the criticality estimation required for downstream decision-making modules.

The node operates in two modes: twist-based or path-based TTC, depending on a configurable parameter. In both cases, it receives as input the list of detected objects, each represented as a `ClusteredObject` message containing position, velocity, and bounding box information. The output is a list of the same objects, each annotated with a TTC estimate.

The node subscribes to the following input topics: `/object_clusters` (`ClusteredObjectArray`), which contains the list of detected objects; `/ego_twist` (`geometry_msgs::Twist`), which provides the linear velocity of the ego vehicle; and optionally `/ego_path` (`nav_msgs::Path`), which defines the planned trajectory in path-based mode.

The output topics are: `/object_ttc`, publishing the list of objects with updated TTC values; `/min_ttc`, containing the minimum TTC observed in the current frame; and `/ego_marker`, which publishes RViz markers for visualizing the ego vehicle geometry and velocity vector.

When a new object array is received, the node computes the TTC for each object based on the selected mode. In path mode, the future position of the object is compared to the ego trajectory as described in Section 3.7.1. In twist mode, the relative velocity is projected onto the direction vector connecting the agents. The result is stored in the `ttc` field of the output message. The minimum TTC value is also computed and published separately.

The node is fully configurable through parameters. These include topic names for input and output; the dimensions of the ego vehicle (`ego_length`, `ego_width`, `ego_height`); the reference frame (`frame_id`); and flags for enabling path-based mode (`use_path`) and RViz visualization (`publish_ego_marker`). All parameters can be defined via a launch file, allowing flexible integration into larger systems.

The node optionally publishes visualization markers in the `camera_optical_frame`, showing the ego vehicle as a green cube and its velocity as a red arrow. These markers provide an intuitive representation of motion in the scene and assist in debugging and validation.

The design ensures real-time operation, with each TTC computation performed independently for each object. This allows for future parallelization and the integration of more advanced models, such as bounding box contact estimation or probabilistic motion prediction.

4 Experiments and Results

This chapter presents the experimental validation of the perception and criticality pipelines developed in this thesis. The evaluation addresses three main goals. First, it assesses the suitability of the selected stereo hardware for 6D motion estimation, including the calibration procedures required for integration with the perception pipeline. In case of insufficient performance, alternative hardware is evaluated and re-integrated accordingly. Second, the computational performance of the full pipeline is analyzed to determine whether real-time constraints can be fulfilled on representative embedded platforms. Third, the output of the deterministic system is compared against the learned perception stack currently deployed on the EDGAR research vehicle, with a particular focus on object detection quality and Time-to-Collision estimation.

All experiments are designed to evaluate compliance with the functional and technical requirements introduced in Chapter 2.5. These include stereo-only sensing, semantic independence, real-time capability, modular implementation, and robustness under partial occlusion. Additional criteria involve 6D motion tracking, consistent egomotion compensation, clustering of dynamic points, and runtime computation of criticality metrics. The objective is not only to validate the proposed design, but also to quantify its reliability, accuracy, and deployability under realistic operational conditions.

The remainder of this chapter is structured as follows. Section 4.1 describes the camera hardware evaluation and calibration procedures. Section 4.2 outlines the evaluation methodology and experimental setup. Sections 4.3 and 4.4 present the results of the perception pipeline assessment and runtime performance analysis, respectively. Finally, Section 4.5 summarizes the main findings.

4.1 Camera hardware selection and calibration

The perception pipeline developed in this thesis is based exclusively on stereo vision. Consequently, selecting a suitable camera system was a critical initial step. Rather than introducing external hardware, the evaluation started with the camera systems already available on the EDGAR research platform. These included: (i) a pair of Framos D455e depth cameras, (ii) six Basler acA1920-50gc cameras mounted for surround view and long-range sensing, and (iii) a set of fisheye cameras dedicated to vehicle teleoperation.

Each system met the integration and software compatibility requirements of EDGAR, such as ROS2 support and Precision Time Protocol (PTP) synchronization. However, their suitability for stereo-based depth estimation varied significantly. Table 4.1 summarizes the most relevant technical characteristics for stereo vision evaluation.

Table 4.1: Comparison of available camera systems on EDGAR

Camera System	FoV (H)	Stereo-ready	Notes
Basler + 16 mm lens	38.6°	No	Long-range; narrow FoV
Basler + 6/4.7 mm lenses	85–100°	No	Non-parallel; fisheye config.
Framos D455e	87°	Yes	Active IR stereo; ROS2 support

The long-range Basler setup, equipped with 16 mm lenses, was designed for distant object detection, such as traffic signs. However, its narrow field of view and lack of stereo configuration made it unsuitable for this application. The surround-view system, composed of six Basler cameras with 6 mm and 4.7 mm lenses, provided wide-angle coverage but was intended for monocular perception. The cameras were mounted at non-parallel orientations around the vehicle and would require complex extrinsic calibration to enable stereo vision. Furthermore, the use of fisheye lenses violated the rectilinear geometry assumptions of the disparity computation module.

The Framos D455e camera was ultimately selected for the first phase of pipeline development. It integrates a stereo pair with a 95 mm baseline and computes depth maps on board using active infrared projection. The camera offers VGA resolution at 30 Hz, includes an IMU, and is natively compatible with ROS 2. These characteristics allowed rapid integration and made it possible to collect initial datasets without extensive driver development.

The goal of this phase was not geometric calibration of the sensor itself, but rather the tuning of internal parameters in the perception pipeline. In particular, the system required calibration of:

- the observation noise of the Kalman filter, including depth and optical flow uncertainty;
- the covariance of the egomotion transform;
- filtering thresholds for optical flow;
- and parameters for motion consistency and point reinitialization.

A series of static experiments were designed and executed to collect the necessary data. These included synthetic motion generation through alpha-angle tests and depth range validation using planar objects at known distances. The controlled nature of the setup allowed for repeatability and easier parameter tuning through offline inspection and visualization in RViz.

The data and methodology of this calibration phase are described in Section 4.1.2. The system limitations encountered when the camera was tested in dynamic outdoor conditions are discussed in Section 4.1.3.

4.1.1 Initial tests with Framos camera

The initial evaluation phase consisted of two test sessions designed to characterize the performance of the Framos D455e stereo camera and to calibrate the parameters of the perception pipeline. The first session focused on direct visual inspection of the raw depth output, while the second was based on structured rosbag recordings integrated with the full pipeline.

Session 1: Visual depth quality inspection

In this session, the camera was operated outside the ROS 2 pipeline. A human subject walked laterally with respect to the camera's optical axis at fixed distances of 2, 4, 6, 8, 10, and 12 meters. For each distance, a pair of frames was recorded: one showing the raw 2D depth image, and one showing the corresponding 3D point cloud visualization. These data were used to assess how depth quality and spatial consistency degrade as the distance increases.

Figure 4.1 and Figure 4.2 show the results at 2 meters. The subject is clearly defined, and the depth structure is sharp. In contrast, Figure 4.3 and Figure 4.4 illustrate the camera output at 10 meters. In this case, the IR-based disparity estimation produces noisy and incomplete results, making it unsuitable for reliable motion tracking.

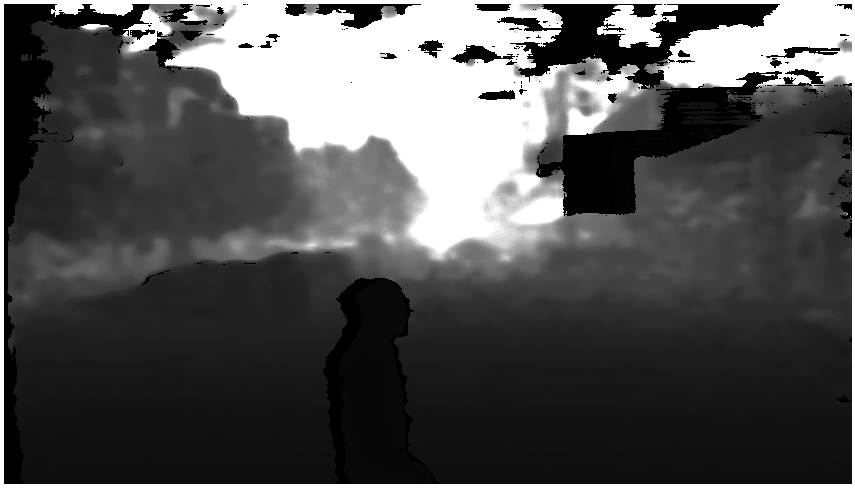


Figure 4.1: Raw 2D depth image at 2 m distance.

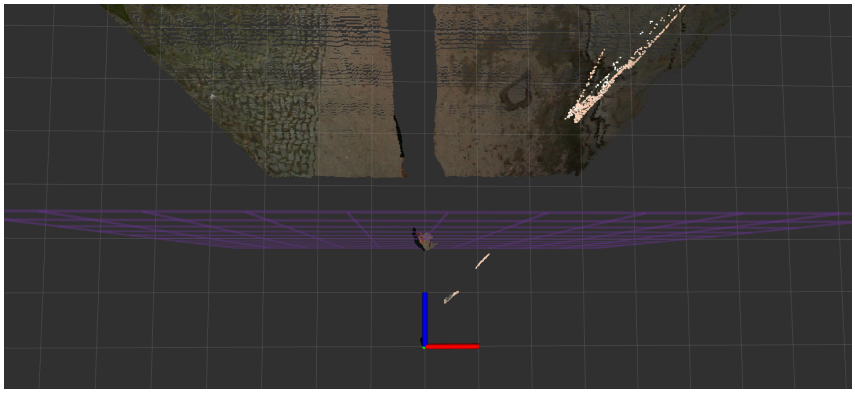


Figure 4.2: 3D depth visualization at 2 m distance.

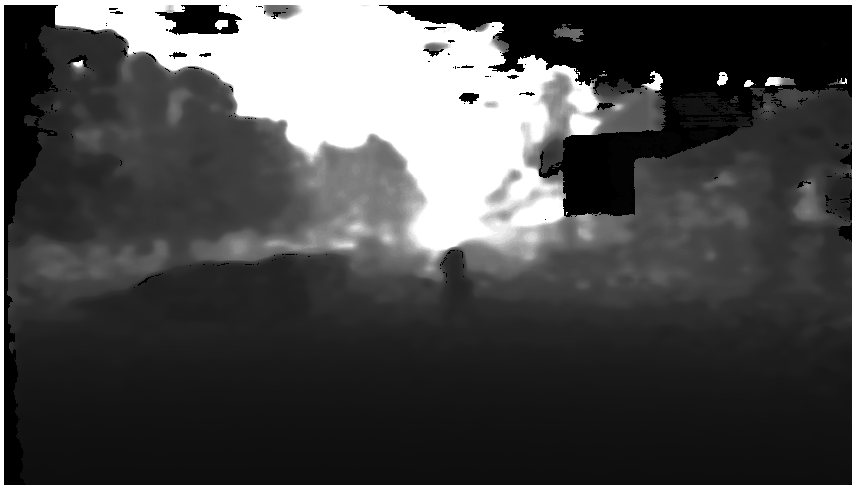


Figure 4.3: Raw 2D depth image at 10 m distance.

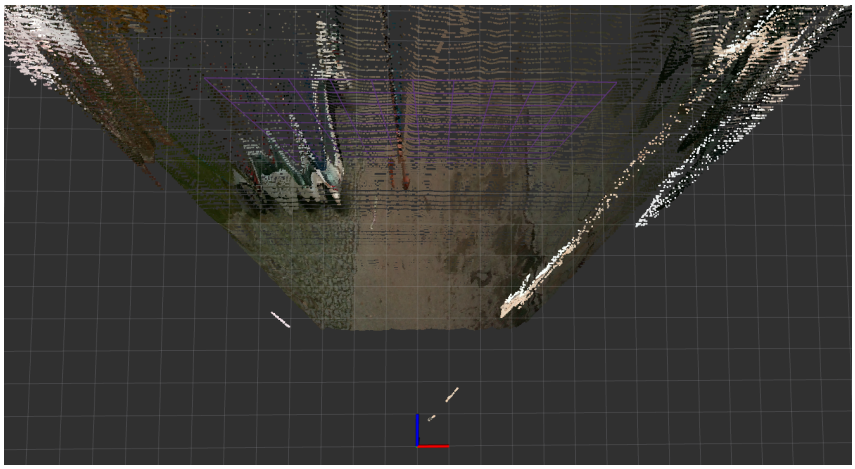


Figure 4.4: 3D depth visualization at 10 m distance.

The complete set of 2D and 3D depth images from this session, including intermediate distances, is available in Appendix A.

From these tests, it was concluded that the Framos D455e camera provides usable depth information up to approximately 8 m in outdoor daylight conditions. Beyond this range, depth quality degrades rapidly, introducing noise and outliers that affect the reliability of the perception modules.

This session also revealed an important phenomenon related to the motion of objects across the image plane. When a tracked `WorldPoint` lies on the border of a moving object—such as a person walking laterally across the field of view—it can be erroneously reassigned to the background in the following frame if the object moves past it. As a result, the Kalman filter receives an invalid depth measurement from a distant surface, causing a sharp discontinuity in the estimated position.

This discontinuity leads to the creation of an artificial velocity vector in the opposite direction of the motion. The resulting ghost vectors appear visually as a "depth shadow" that trails the object and can introduce false positives in the clustering stage. An example of this effect is shown in Figure **??**, where the depth discontinuity causes a spurious motion estimate along the wall.

Add a figure showing the depth shadow effect.

This observation motivated the need for controlled experiments with known motion trajectories, leading to the second session described below.

Session 2: Structured rosbag recording

The objective of this recording session was to acquire structured datasets under repeatable and controlled conditions, using only raw sensor data from the Framos stereo camera. Unlike the first session, where depth quality was evaluated directly, this phase focused on generating rosbags that would later be used for offline parameter tuning and performance validation.

To this end, three types of scenarios were recorded:

- **range test**, in which a subject walked directly towards the camera, enabling later analysis of detection range and point initialization behavior.
- **alpha-angle test**, where the subject followed predefined diagonal trajectories at known angles, intended to study how trajectory orientation affects tracking quality and velocity estimation.
- **dynamic test**, conducted with the EDGAR vehicle in motion, to assess the perception pipeline's robustness under real driving conditions.

All scenarios were recorded with the vehicle in a fixed or controlled setting, and no processing was performed online. The pipeline was executed offline after recording to allow parameter sweeps and diagnostic visualization in RViz. These datasets form the basis for the calibration process described in Section 4.1.2, and the evaluation of results in Section 4.1.3.

Range test This test was carried out in front of the Leichtbauhalle at the Garching Forschungszentrum campus, with the goal of evaluating at which distance the subject becomes detectable and consistently trackable by the perception pipeline. A series of ground markers were placed between 30 m and 1 m in front of the EDGAR vehicle, defining a straight walking path aligned with the optical axis of the Framos camera.

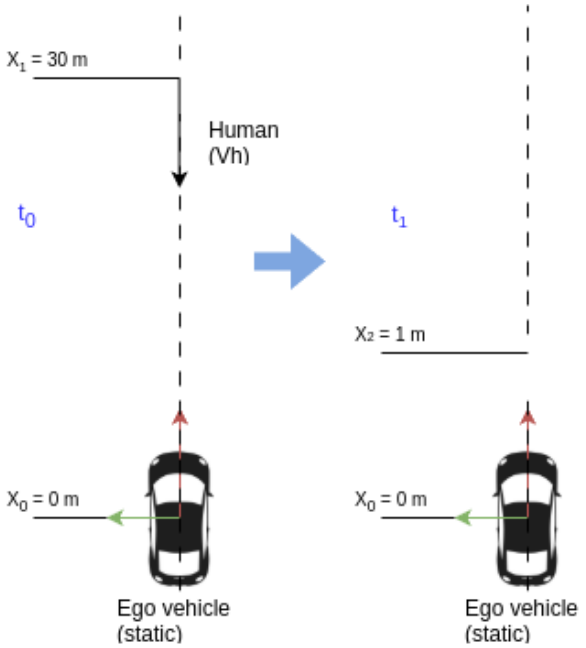


Figure 4.5: Range test experiment setup. Source: own work

During the test, a person walked toward the vehicle at constant speed, passing through each marker. The camera was mounted on top of EDGAR in its standard forward-facing configuration. No processing was performed online; instead, the raw sensor data were recorded for offline analysis and calibration.

The following topics were captured:

- Color image: `/device_0/sensor_1/Color_0/image/data`
- Depth image: `/device_0/sensor_0/Depth_0/image/data`
- Camera info: `/device_0/sensor_0/Depth_0/info/camera_info`

This dataset enabled later evaluation of Kalman filter initialization behavior at different distances, and provided ground evidence to assess how depth quality and measurement stability degrade as the object approaches. A schematic of the test setup is shown in Figure **??**.

Alpha-angle test In this test, a human subject walked along six predefined trajectories forming known angles α with respect to the optical axis of the Framos camera. The objective was to evaluate the impact of trajectory orientation on the stability and accuracy of the estimated 6D velocity vectors. This setup also enabled partial ground truth estimation by comparing the known motion direction with the one inferred by the Kalman filter.

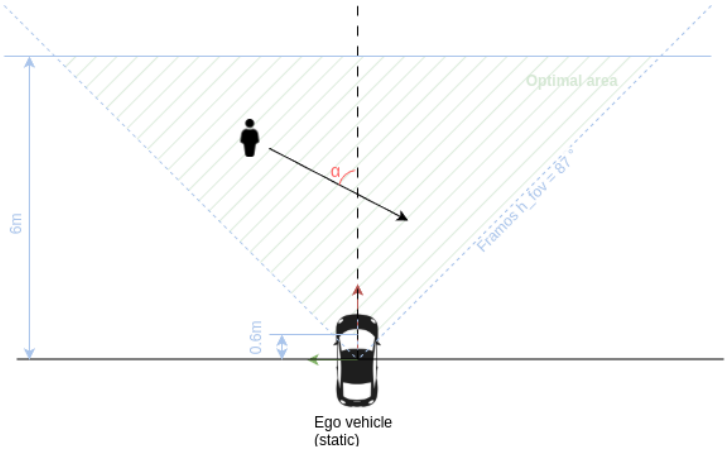


Figure 4.6: Angle test experiment setup. Source: own work

The camera was mounted on the EDGAR vehicle in a fixed position, and the subject's path was defined by marking the start and end points on the ground using measuring tape. These positions were chosen to produce α values ranging from 14° to over 80°, as summarized in Table 4.2. The subject maintained approximately constant speed during all recordings.

In some trajectories, the motion crossed the center of the field of view, while in others it remained off-center. This variety helped evaluate the sensitivity of the perception pipeline to viewing angle and occlusion effects.

As in the previous test, only raw sensor data were recorded. The following topics were captured:

- Color image: `/device_0/sensor_1/Color_0/image/data`
- Depth image: `/device_0/sensor_0/Depth_0/image/data`
- Camera info: `/device_0/sensor_0/Depth_0/info/camera_info`
- Odometry (for reference frame tracking)

The dataset was later used to calibrate the directional sensitivity of the velocity estimation module and to verify the consistency of the motion model under varying geometric conditions.

A summary of all trajectories is shown in Figure 4.7, and their corresponding coordinates and angles are listed in Table 4.2.

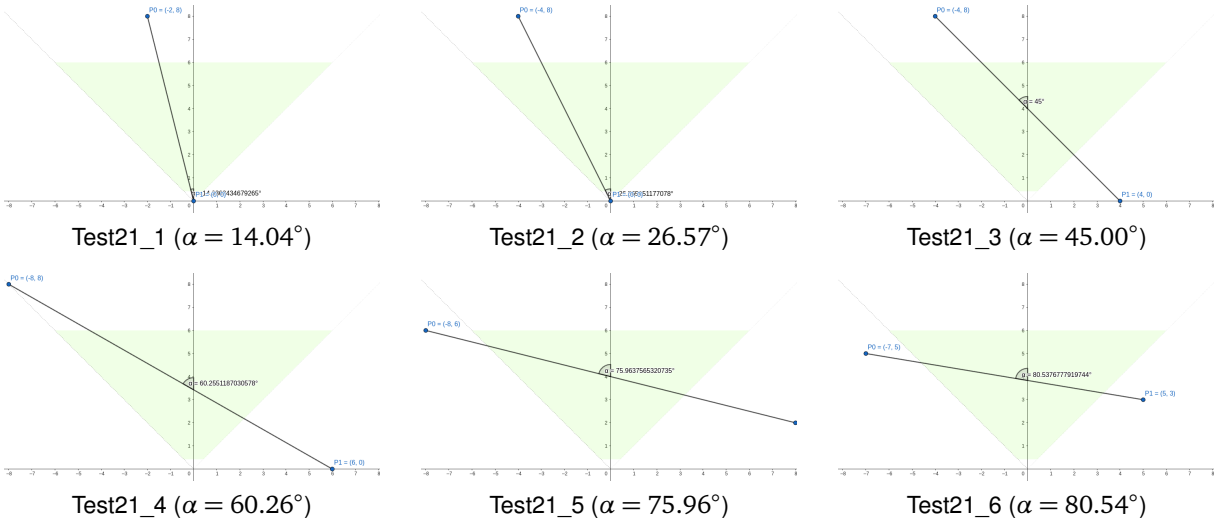


Figure 4.7: Overview of the six recorded trajectories used in the alpha-angle test.

Table 4.2: Coordinates and angles of recorded trajectories.

Recording	P0 (Xw, Yw)	P1 (Xw, Yw)	α (deg)
Test21_1	(-2, 8)	(0, 0)	14.04
Test21_2	(-4, 8)	(0, 0)	26.57
Test21_3	(-4, 8)	(4, 0)	45.00
Test21_4	(-8, 8)	(6, 0)	60.26
Test21_5	(-8, 6)	(8, 2)	75.96
Test21_6	(-7, 5)	(5, 3)	80.54

This test provided valuable insight into how trajectory orientation affects the geometry of observations and the robustness of temporal tracking. The variety of angles and lateral offsets allowed for systematic tuning of the motion model parameters and velocity estimation logic. These data were later used as part of the pipeline calibration procedure described in Section 4.1.2.

Dynamic test The third recording scenario consisted of a short autonomous drive with EDGAR around the perimeter of the Leichtbauhalle building at Garching Forschungszentrum. The objective was to evaluate the behavior of the perception pipeline under motion-induced perturbations, such as camera vibration, parallax variation, and changing egocentric viewpoint. Unlike the previous tests, this scenario included continuous egomotion of the camera platform.

The vehicle followed a predefined loop at a speed of approximately 10–15 km/h, maintaining a smooth trajectory without abrupt accelerations or stops. The total duration of the recording was around two minutes. Although the environment included road infrastructure and occasional traffic participants, few dynamic objects were present in the recorded scene. Nonetheless, the data served as a valuable baseline for evaluating the robustness of perception modules under non-static conditions.

As in the other scenarios, only raw camera data were recorded. The topics included:

- Color image: `/device_0/sensor_1/Color_0/image/data`
- Depth image: `/device_0/sensor_0/Depth_0/image/data`
- Camera info: `/device_0/sensor_0/Depth_0/info/camera_info`

This dataset was later used to calibrate the egomotion compensation module and to evaluate the pipeline's performance under real driving conditions. The full route is illustrated in Figure **??**.

Add route illustration as Figure **??**

4.1.2 Calibration of the perception pipeline

Describe the parameter calibration session with the Framos camera.

4.1.3 Results, limitations and motivation for replacement

Discuss the limitations of the Framos stereo camera.

4.1.4 Selection of the Carnegie S30 stereo camera

Describe the selection criteria for the Carnegie S30 camera.

4.1.5 Calibration and dataset recording with Carnegie

4.2 Evaluation methodology

Detail the experiments conducted with the Carnegie S30 camera.

4.2.1 Evaluation modes

Introduce the evaluation methodology.

4.2.2 Recording scenarios and conditions

Define qualitative and quantitative evaluation strategies.

4.3 Perception pipeline evaluation

Describe where and how the evaluation data was collected.

4.3.1 Qualitative results

Introduce the pipeline evaluation section.

4.3.2 Quantitative TTC comparison

Present the qualitative comparison between your pipeline and Autoware.

4.4 Performance benchmarking

Analyze and compare TTC values between pipelines.

4.4.1 Tested hardware platforms

Introduce the performance benchmarking section.

4.4.2 Execution time and resource usage

4.5 Summary of results

List and describe the platforms used for benchmarking.

Present runtime and system load results for all platforms

5 Summary and Conclusion

Write a summary of the thesis, including the main contributions, the results obtained, and the implications for future work. Reflect on the limitations of the current implementation and suggest possible improvements.

List of Figures

List of Tables

Bibliography

[1] J. Van Brummelen, M. O'Brien, D. Gruyer and H. Najjaran, "Autonomous vehicle perception: The technology of today and tomorrow," *Transportation Research Part C: Emerging Technologies*, vol. 89, pp. 384–406, 2018, DOI: 10.1016/j.trc.2018.02.012.

[2] BMW. "*Level 3 highly automated driving available in the new BMW 7 Series from next spring.*" en. 2023. Available: https://www.press.bmwgroup.com/global/article/detail/T0438214EN/level-3-highly-automated-driving-available-in-the-new-bmw-7-series-from-next-spring?language=en [visited on 02/26/2025].

[3] Mercedes-Benz. "*Drive Pilot*," 2025. Available: https://www.mercedes-benz.de/passengercars/technology/drive-pilot.html [visited on 02/26/2025].

[4] Waymo. "*Meet the 6th-generation Waymo Driver: Optimized for costs, designed to handle more weather, and coming to riders faster than before*," 2024. Available: https://waymo.com/blog/2024/08/meet-the-6th-generation-waymo-driver [visited on 02/26/2025].

[5] Y. Li and J. Ibanez-Guzman, "Lidar for Autonomous Driving: The Principles, Challenges, and Trends for Automotive Lidar and Perception Systems," *IEEE Signal Processing Magazine*, vol. 37, no. 4, pp. 50–61, 2020, DOI: 10.1109/MSP.2020.2973615.

[6] J. Kim, B.-j. Park and J. Kim, "Empirical Analysis of Autonomous Vehicle's LiDAR Detection Performance Degradation for Actual Road Driving in Rain and Fog," *Sensors*, vol. 23, no. 6, 2023, DOI: 10.3390/s23062972. Available: https://www.mdpi.com/1424-8220/23/6/2972.

[7] J. Leonard, J. How, S. Teller, M. Berger, S. Campbell, et al., "A perception-driven autonomous urban vehicle," *Journal of Field Robotics*, vol. 25, no. 10, pp. 727–774, 2008, DOI: 10.1002/rob.20262.

[8] R. H. Rasshofer and K. Gresser, "Automotive Radar and Lidar Systems for Next Generation Driver Assistance Functions," in *Advances in Radio Science*, 2005, pp. 205–209, DOI: 10.5194/ars-3-205-2005.

[9] K. N. Maanyu, D. G. Raj, R. V. Krishna and D. S. B. Choubey, "A Study on Tesla Autopilot," vol. 71, no. 171, 2020.

[10] R. Szeliski, *Computer Vision: Algorithms and Applications*, (Texts in Computer Science), Cham, Springer International Publishing, 2022, ISBN: 978-3-030-34371-2 978-3-030-34372-9. DOI: 10.1007/978-3-030-34372-9.

[11] X. Zhou, V. Koltun and P. Krähenbühl, "Tracking Objects as Points," *CoRR*, vol. abs/2004.01177, 2020. arXiv: 2004.01177. Available: https://arxiv.org/abs/2004.01177.

[12] B. Shuai, A. G. Berneshawi, D. Modolo and J. Tighe, "Multi-Object Tracking with Siamese Track-RCNN," *CoRR*, vol. abs/2004.07786, 2020. arXiv: 2004.07786. Available: https://arxiv.org/abs/2004.07786.

[13] S. Shi, X. Wang and H. Li, "PointRCNN: 3D Object Proposal Generation and Detection from Point Cloud," *CoRR*, vol. abs/1812.04244, 2018. arXiv: 1812.04244. Available: http://arxiv.org/abs/1812.04244.

[14] Y. Li, A. W. Yu, T. Meng, B. Caine, J. Ngiam, et al. "*DeepFusion: Lidar-Camera Deep Fusion for Multi-Modal 3D Object Detection*," 2022. arXiv: 2203.08195 [cs.CV]. Available: https://arxiv.org/abs/2203.08195.

[15] L.-H. Wen and K.-H. Jo, "Deep learning-based perception systems for autonomous driving: A comprehensive survey," *Neurocomputing*, vol. 489, pp. 255–270, 2022, DOI: 10.1016/j.neucom.2021.08.155.

[16] B. Lucas and T. Kanade, "An Iterative Image Registration Technique with an Application to Stereo Vision (IJCAI)," 1981.

[17] B. K. P. Horn and B. G. Schunck, "Determining optical flow," *Artificial Intelligence*, vol. 17, no. 1, pp. 185–203, 1981, DOI: 10.1016/0004-3702(81)90024-2.

[18] U. Franke, C. Rabe, H. Badino and S. Gehrig, "6D-Vision: Fusion of Stereo and Motion for Robust Environment Perception," in *Pattern Recognition*, 2005, pp. 216–223, ISBN: 978-3-540-31942-9. DOI: 10.1007/11550518_27.

[19] P. Karle, T. Betz, M. Bosk, F. Fent, N. Gehrke, et al. "*EDGAR: An Autonomous Driving Research Platform – From Feature Development to Real-World Application*," arXiv:2309.15492 [cs]. Jan. 2024. DOI: 10.48550/arXiv.2309.15492.

[20] S. Bauer, "Objektdetektion und Bewegungsprädiktion mittels eines Stereokamerasystems beim teleoperierten Fahren," MA thesis, TUM, Munich, 2014.

[21] A. Rotar, "Weiterentwicklung und echtzeitfähige Umsetzung eines Stereo-Vision-basierten Bewegungsprädiktionssystems für die aktive Sicherheit von teleoperierten Fahrzeugen," MA thesis, TUM, Munich, 2016.

[22] C. Pastor, "Validation of an Obstacle Detection Approach for Teleoperation of Automated Vehicles," BSc. Thesis, TUM, 2021.

[23] ros. "*3D Camera Survey — ROS-Industrial — rosindustrial.org*," https://rosindustrial.org/3d-camera-survey. [Accessed 12-05-2025]. 2025.

[24] A. O' Riordan, T. Newe, D. Toal and G. Dooly, "Stereo Vision Sensing: Review of existing systems," 2018, DOI: 10.1109/ICSensT.2018.8603605.

[25] M. K. Gjerde, K. Nasrollahi, T. B. Moeslund and J. B. Haurum, "Enhancing Direct Visual Odometry with Deblurring and Saliency Maps," in *ICMVA*, 2024, pp. 154–161. Available: https://doi.org/10.1145/3653946.3653970.

[26] L. O. Rojas-Perez and J. Martinez-Carranza, "Towards Autonomous Drone Racing without GPU Using an OAK-D Smart Camera," *Sensors*, vol. 21, no. 22, 2021, DOI: 10.3390/s21227436. Available: https://www.mdpi.com/1424-8220/21/22/7436.

[27] P. Hübner, J. Hou and D. Iwaszczuk, "EVALUATION OF INTEL REALSENSE D455 CAMERA DEPTH ESTIMATION FOR INDOOR SLAM APPLICATIONS," *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLVIII-1/W2-2023, pp. 1207–1214, 2023, DOI: 10.5194/isprs-archives-XLVIII-1-W2-2023-1207-2023.

[28] L. Rustler, V. Volprecht and M. Hoffmann, "Empirical Comparison of Four Stereoscopic Depth Sensing Cameras for Robotics Applications," *IEEE Access*, vol. 13, pp. 67564–67577, 2025, DOI: 10.1109/access.2025.3560810. Available: http://dx.doi.org/10.1109/ACCESS.2025.3560810.

[29] F. Afzal Maken, S. Muthu, C. Nguyen, C. Sun, J. Tong, et al., "Improving 3D Reconstruction Through RGB-D Sensor Noise Modeling," *Sensors*, vol. 25, no. 3, 2025, DOI: 10.3390/s25030950. Available: https://www.mdpi.com/1424-8220/25/3/950.

[30] B. H. Wang, C. Diaz-Ruiz, J. Banfi and M. E. Campbell, “Detecting and Mapping Trees in Unstructured Environments with a Stereo Camera and Pseudo-Lidar," *CoRR*, vol. abs/2103.15967, 2021. arXiv: 2103.15967. Available: https://arxiv.org/abs/2103.15967.

[31] A. Asvadi, C. Premebida, P. Peixoto and U. Nunes, “3D Lidar-Based Static and Moving Obstacle Detection in Driving Environments: An Approach Based on Voxels and Multi-Region Ground Planes," *Journal of Field Robotics*, vol. 33, no. 10, pp. 1230–1252, 2016, DOI: 10.1002/rob.21661. Available: https://www.researchgate.net/publication/304247843 [visited on 05/12/2025].

[32] D. J. Yoon, T. Y. Tang and T. D. Barfoot, “Mapless Online Detection of Dynamic Objects in 3D Lidar," *arXiv preprint arXiv:1809.06972*, 2018, DOI: 10.48550/arXiv.1809.06972. Available: http://arxiv.org/abs/1809.06972 [visited on 05/12/2025].

[33] T. Eppenberger, G. Cesari, M. Dymczyk, R. Siegwart and R. Dubé, “Leveraging Stereo-Camera Data for Real-Time Dynamic Obstacle Detection and Tracking," *arXiv preprint arXiv:2007.10743*, 2020, DOI: 10.48550/arXiv.2007.10743. Available: http://arxiv.org/abs/2007.10743 [visited on 05/12/2025].

[34] A. Ess, B. Leibe, K. Schindler and L. Van Gool, “Moving Obstacle Detection in Highly Dynamic Scenes," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 56–63, DOI: 10.1109/ROBOT.2009.5152884. Available: https://www.researchgate.net/publication/221071574 [visited on 05/12/2025].

[35] W. Shi, M. Liu and Y. Liu, “Dynamic Obstacles Rejection for 3D Map Simultaneous Updating," *IEEE Access*, vol. 6, pp. 42939–42951, 2018, DOI: 10.1109/ACCESS.2018.2836192. Available: https://www.researchgate.net/publication/325137826 [visited on 05/12/2025].

[36] G. Farnebäck, “Two-Frame Motion Estimation Based on Polynomial Expansion," 2003, pp. 363–370, ISBN: 978-3-540-40601-3. DOI: 10.1007/3-540-45103-X_50.

[37] M. Ester, H.-P. Kriegel, J. Sander and X. Xu, “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," in *Knowledge Discovery and Data Mining*, 1996. Available: https://api.semanticscholar.org/CorpusID:355163.

[38] L. Westhofen, C. Neurohr, T. Koopmann, M. Butz, B. Schütt, et al., “Criticality Metrics for Automated Driving: A Review and Suitability Analysis of the State of the Art," *Archives of Computational Methods in Engineering*, vol. 30, no. 1, pp. 1–35, 2023, DOI: 10.1007/s11831-022-09788-7.

[39] nvidia. “*NVIDIA CUDA GPU Compute Capability — developer.nvidia.com*," https://developer.nvidia.com/cuda-gpus. [Accessed 09-05-2025]. 2025.

[40] S. Heidari, M. Rogers and P. J. Delmas, “(PDF) An Improved Quantum Solution for the Stereo Matching Problem," in *ResearchGate*, DOI: 10.1109/IVCNZ54163.2021.9653310. Available: https://www.researchgate.net/publication/357424395_An_Improved_Quantum_Solution_for_the_Stereo_Matching_Problem [visited on 04/26/2025].

[41] J. Karathanasis, D. Kalivas and J. Vlontzos, “Disparity estimation using block matching and dynamic programming," in *Proceedings of Third International Conference on Electronics, Circuits, and Systems*, 1996, 728–731 vol.2, DOI: 10.1109/ICECS.1996.584465. Available: https://ieeexplore.ieee.org/abstract/document/584465 [visited on 04/26/2025].

[42] H. Hirschmuller, “Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, DOI: 10.1109/TPAMI.2007.1166.

[43] R. A. Hamzah and H. Ibrahim, “Literature Survey on Stereo Vision Disparity Map Algorithms," *Journal of Sensors*, vol. 2016, no. 1, p. 8742920, 2016, DOI: 10.1155/2016/8742920. Available: https://onlinelibrary.wiley.com/doi/abs/10.1155/2016/8742920 [visited on 04/26/2025].

[44] D. C. Niehorster, “Optic Flow: A History," *i-Perception*, vol. 12, no. 6, p. 20416695211055766, 2021, DOI: 10.1177/20416695211055766. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8652193/.

[45] G. Boesch. “*Optical Flow - Everything You Need to Know in 2025*," en-US. Oct. 2024. Available: https://viso.ai/deep-learning/optical-flow/ [visited on 11/12/2024].

[46] D. Fortun, P. Bouthemy and C. Kervrann, “Optical flow modeling and computation: A survey," *Computer Vision and Image Understanding*, vol. 134, pp. 1–21, 2015, DOI: 10.1016/j.cviu.2015.02.008. Available: https://linkinghub.elsevier.com/retrieve/pii/S1077314215000429 [visited on 11/13/2024].

[47] O. Zvorișteanu, S. Caraiman and V. Manta, “Speeding Up Semantic Instance Segmentation by Using Motion Information," *Mathematics*, vol. 10, p. 2365, 2022, DOI: 10.3390/math10142365.

[48] P. Fischer, A. Dosovitskiy, E. Ilg, P. Häusser, C. Hazirbas, et al., “FlowNet: Learning Optical Flow with Convolutional Networks," *CoRR*, vol. abs/1504.06852, 2015. arXiv: 1504.06852. Available: http://arxiv.org/abs/1504.06852.

[49] Z. Teed and J. Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," *CoRR*, vol. abs/2003.12039, 2020. arXiv: 2003.12039. Available: https://arxiv.org/abs/2003.12039.

[50] A. Alfarano, L. Maiano, L. Papa and I. Amerini, “Estimating optical flow: A comprehensive review of the state of the art," *Computer Vision and Image Understanding*, vol. 249, p. 104160, 2024, DOI: 10.1016/j.cviu.2024.104160.

[51] J. L. Barron, D. J. Fleet and S. S. Beauchemin, “Performance of optical flow techniques," *International Journal of Computer Vision*, vol. 12, no. 1, pp. 43–77, 1994, DOI: 10.1007/BF01420984. Available: https://doi.org/10.1007/BF01420984.

[52] S. Lucey, R. Navarathna, A. B. Ashraf and S. Sridharan, “Fourier Lucas-Kanade Algorithm," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 6, pp. 1383–1396, 2013, DOI: 10.1109/TPAMI.2012.220.

[53] J. Sánchez, E. Llopis and G. Facciolo, “TV-L1 optical flow estimation," *Image Processing On Line*, vol. 3, pp. 137–150, 2013, DOI: 10.5201/ipol.2013.26.

[54] R. Szeliski, “Computer Vision: Algorithms and Applications," 2021.

[55] H. Moravec, “Obstacle Avoidance and Navigation in the Real World by a Seeing Robot Rover," Pittsburgh, PA, 1980.

[56] D. Nister, “An efficient solution to the five-point relative pose problem," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 26, no. 6, pp. 756–770, 2004, DOI: 10.1109/TPAMI.2004.17.

[57] K. L. Lim and T. Bräunl. “*A Review of Visual Odometry Methods and Its Applications for Autonomous Driving*," arXiv:2009.09193 [cs]. Sept. 2020. DOI: 10.48550/arXiv.2009.09193.

[58] E. Marchand, H. Uchiyama and F. Spindler, “Pose Estimation for Augmented Reality: A Hands-On Survey | Request PDF," *ResearchGate*, 2024, DOI: 10.1109/TVCG.2015.2513408. Available: https://www.researchgate.net/publication/287348989_Pose_Estimation_for_Augmented_Reality_A_Hands-On_Survey [visited on 04/29/2025].

[59] OpenCV. “*OpenCV: ORB (Oriented FAST and Rotated BRIEF)*," Available: https://docs.opencv.org/4.x/d1/d89/tutorial_py_orb.html [visited on 04/27/2025].

[60] OpenCV. “*OpenCV: FAST Algorithm for Corner Detection*," Available: https://docs.opencv.org/3.4/df/d0c/tutorial_py_fast.html [visited on 04/27/2025].

[61] OpenCV. “*OpenCV: BRIEF (Binary Robust Independent Elementary Features)*," Available: https://docs.opencv.org/3.4/dc/d7d/tutorial_py_brief.html [visited on 04/27/2025].

[62] Deep. "*Introduction to FAST (Features from Accelerated Segment Test) — deepanshut041*," https://medium.com/@deepanshut041/introduction-to-fast-features-from-accelerated-segment-test-4ed33dde6d65. [Accessed 29-04-2025].

[63] OpenCV. "*OpenCV: Perspective-n-Point (PnP) pose computation*," 2024. Available: https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html [visited on 04/29/2025].

[64] OpenMP. "*Reference Guides - OpenMP — openmp.org*," https://www.openmp.org/resources/refguides/. [Accessed 02-05-2025]. 2025.

[65] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Transactions of the ASME–Journal of Basic Engineering*, vol. 82, no. Series D, pp. 35–45, 1960.

[66] M. Ribeiro and I. Ribeiro. "*Kalman and Extended Kalman Filters: Concept, Derivation and Properties*," Apr. 2004.

[67] C. Rabe, "Detection of Moving Objects by Spatio-Temporal Motion Analysis," 2011.

[68] H. Cheng and K. C. Gupta, "An Historical Note on Finite Rotations," *Journal of Applied Mechanics*, vol. 56, no. 1, pp. 139–145, 1989, DOI: 10.1115/1.3176034. Available: https://doi.org/10.1115/1.3176034.

[69] J. S. Dai, "Euler–Rodrigues formula variations, quaternion conjugation and intrinsic connections," *Mechanism and Machine Theory*, vol. 92, pp. 144–152, 2015, DOI: 10.1016/j.mechmachtheory.2015.03.004. Available: https://www.sciencedirect.com/science/article/pii/S0094114X15000415 [visited on 05/02/2025].

[70] J. B. MacQueen, "Some Methods for Classification and Analysis of MultiVariate Observations," in *Proc. of the fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1967, pp. 281–297.

[71] F. Murtagh and P. Legendre, "Ward's Hierarchical Agglomerative Clustering Method: Which Algorithms Implement Ward's Criterion?," *Journal of Classification*, vol. 31, no. 3, pp. 274–295, 2014, DOI: 10.1007/s00357-014-9161-z. Available: http://dx.doi.org/10.1007/s00357-014-9161-z.

[72] D. Comaniciu and P. Meer, "Mean shift: a robust approach toward feature space analysis," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 5, pp. 603–619, 2002, DOI: 10.1109/34.1000236.

[73] A. Mehle, B. Likar and D. Tomaževič, "In-line recognition of agglomerated pharmaceutical pellets with density-based clustering and convolutional neural network," *IPSJ Transactions on Computer Vision and Applications*, vol. 9, 2017, DOI: 10.1186/s41074-017-0019-2.

[74] F. Alonso, J. Naranjo and F. Garcia, "An Improved Method to Calculate the Time-to-Collision of Two Vehicles," *International Journal of Intelligent Transportation Systems Research*, vol. 11, 2011, DOI: 10.1007/s13177-012-0054-4.

Appendix

A Raw Depth and 3D Visualizations from Framos Camera

This appendix provides the full set of raw 2D depth images and corresponding 3D point cloud visualizations obtained during the first test session with the Framos D455e camera. These results demonstrate how depth quality deteriorates progressively with distance in outdoor conditions.

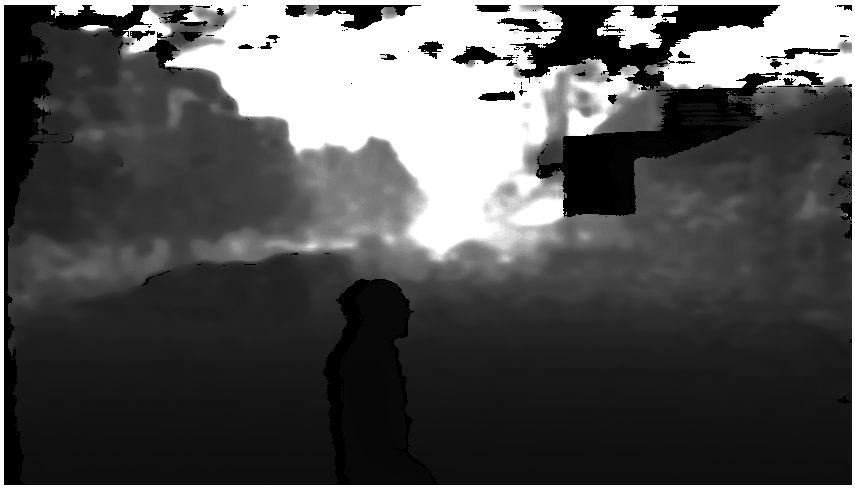


Figure A.1: Raw 2D depth image at 2 m distance.

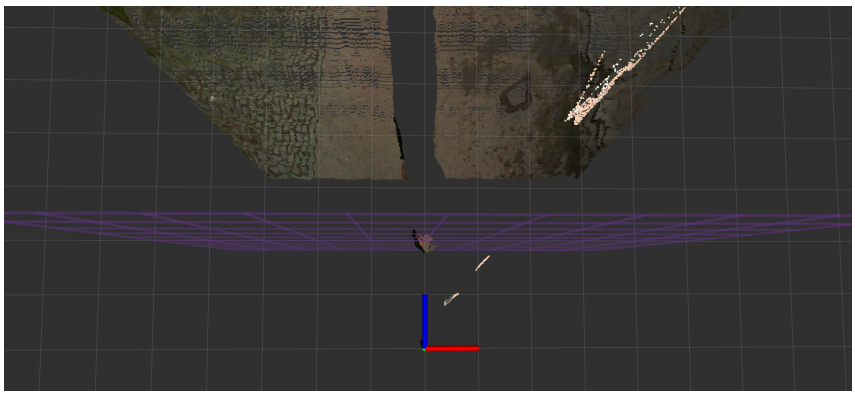


Figure A.2: 3D depth visualization at 2 m distance.

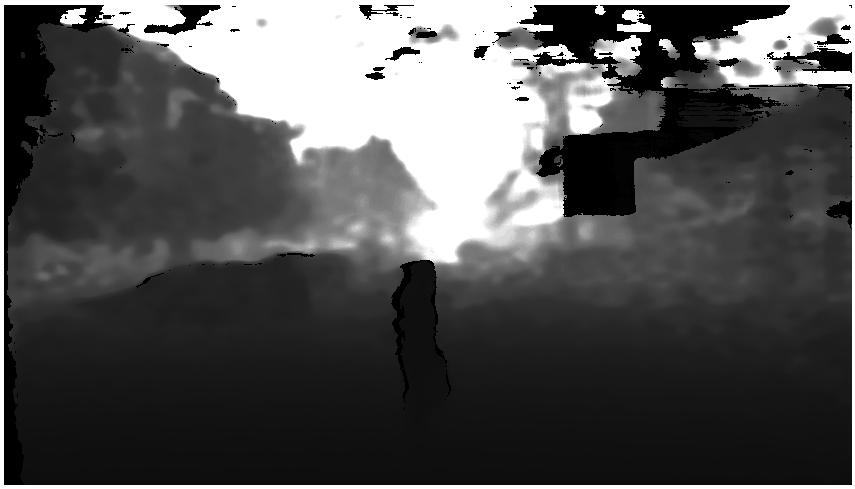


Figure A.3: Raw 2D depth image at 4 m distance.

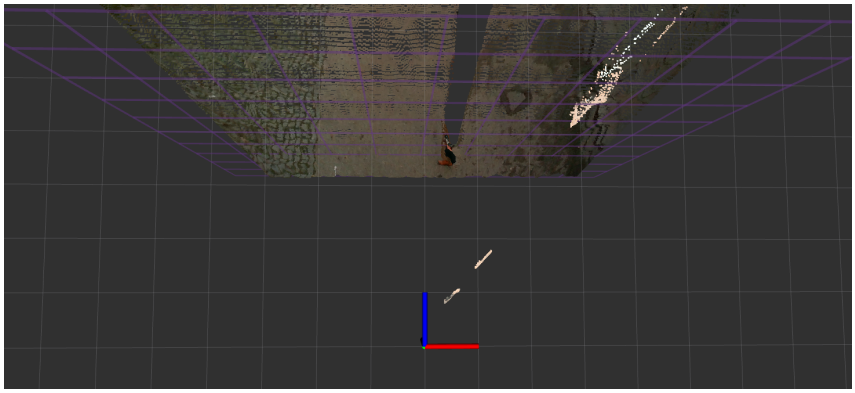


Figure A.4: 3D depth visualization at 4 m distance.

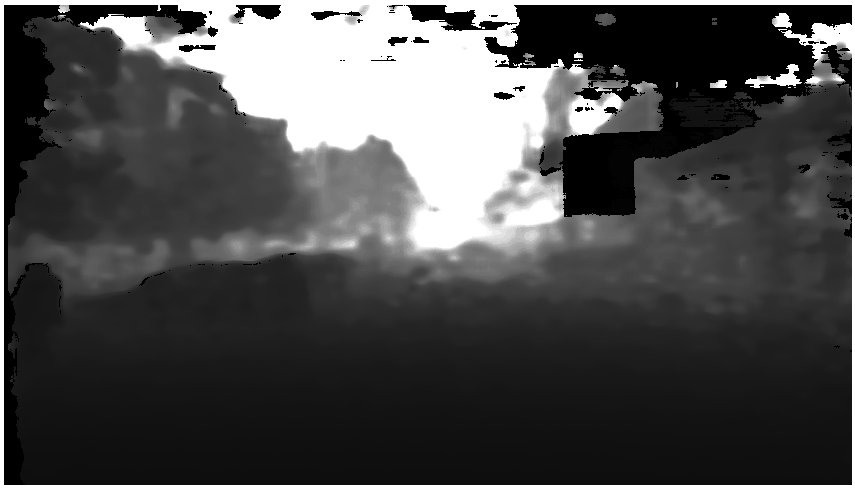


Figure A.5: Raw 2D depth image at 6 m distance.

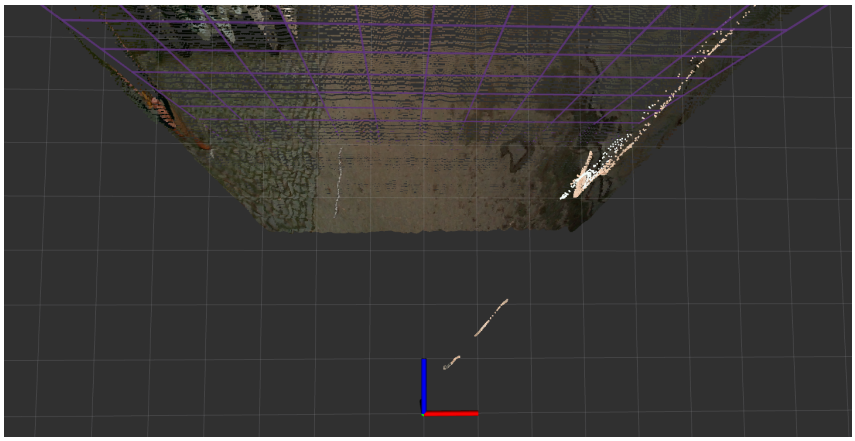


Figure A.6: 3D depth visualization at 6 m distance.

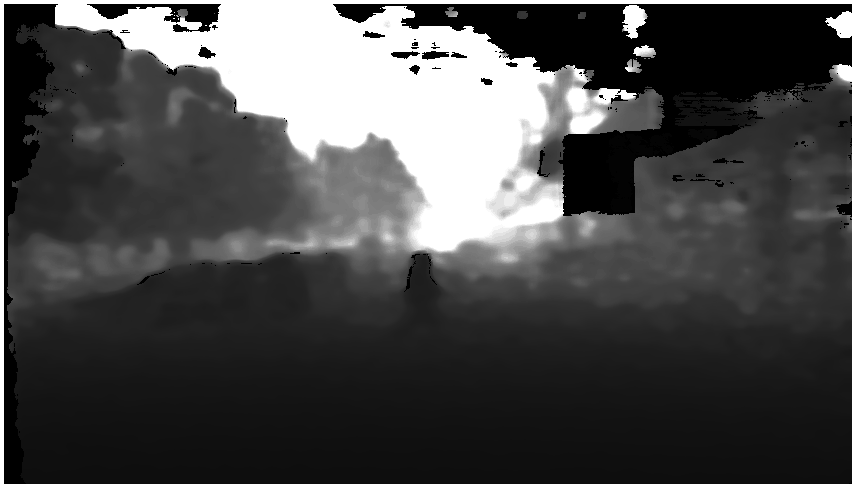


Figure A.7: Raw 2D depth image at 8 m distance.

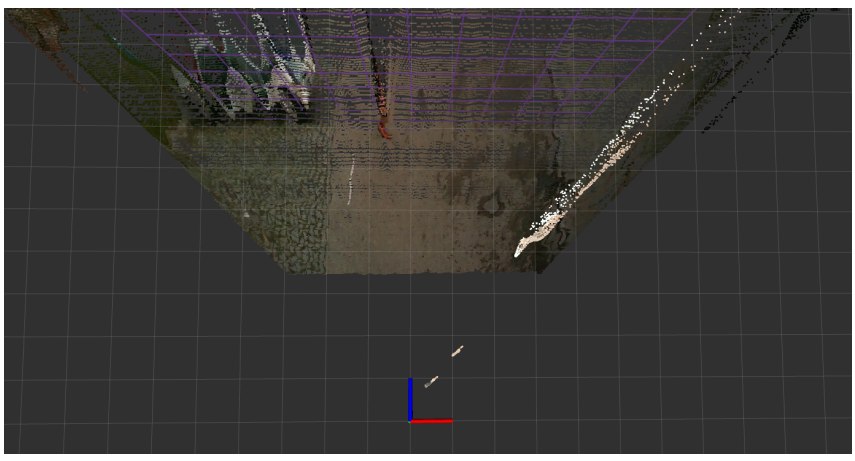


Figure A.8: 3D depth visualization at 8 m distance.

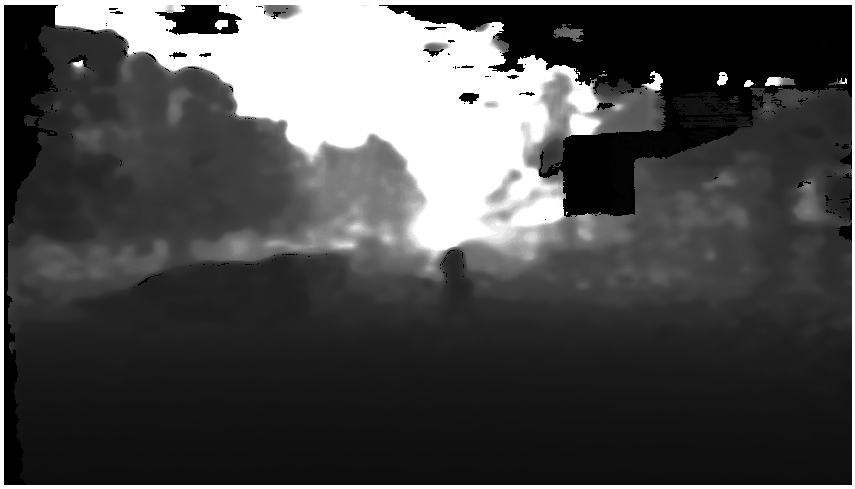


Figure A.9: Raw 2D depth image at 10 m distance.

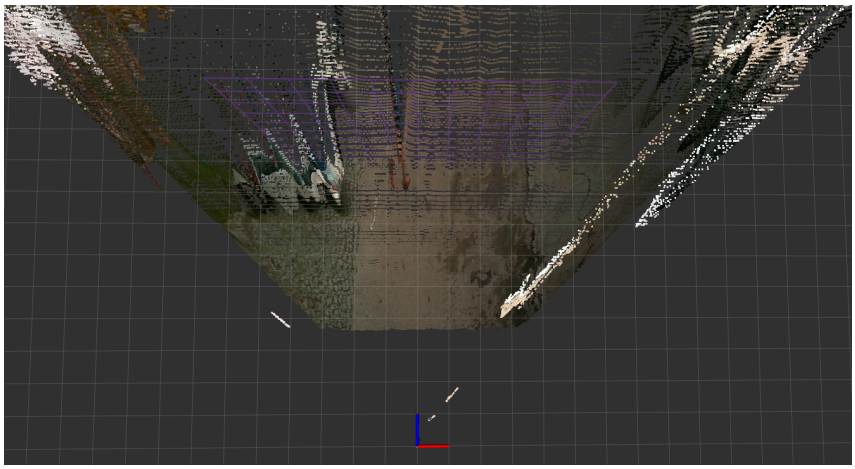


Figure A.10: 3D depth visualization at 10 m distance.

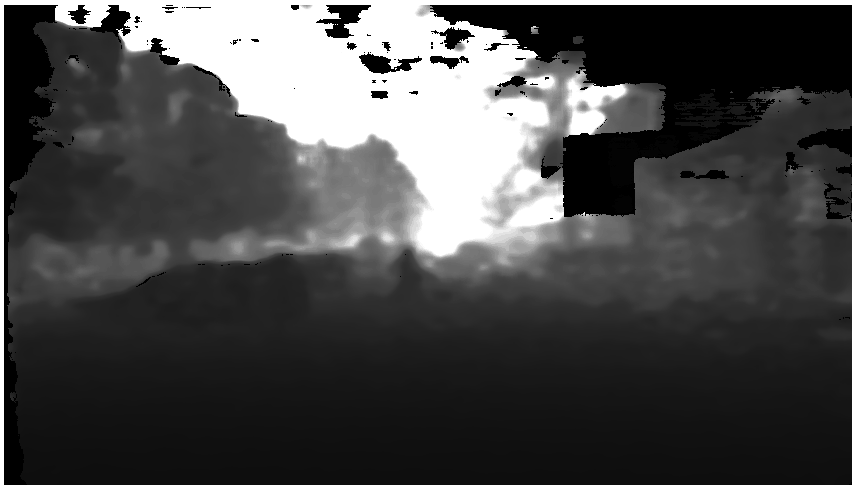


Figure A.11: Raw 2D depth image at 12 m distance.

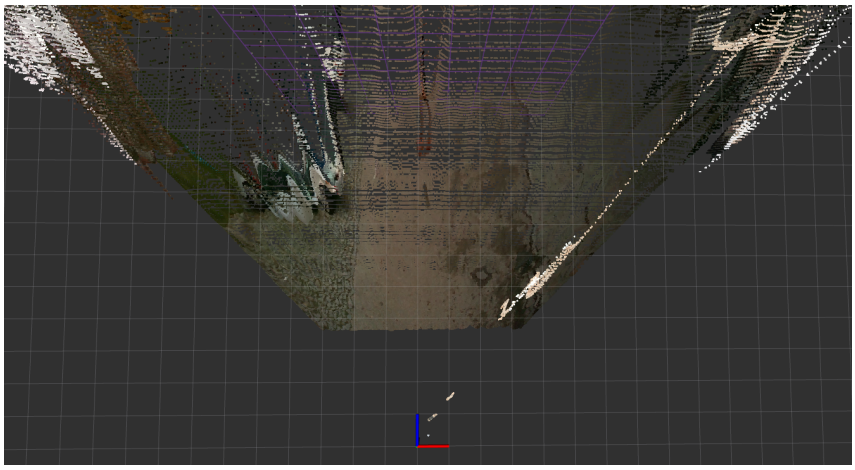


Figure A.12: 3D depth visualization at 12 m distance.